

CORE COMPUTER SCIENCE · DATA STRUCTURES · VOLUME 2

Everything About Queues

FIFO, ring buffers, deques, heaps, and the systems built on them.

86 topics in twelve parts · C++17 throughout · every topic with runnable code, an ASCII diagram, a trap and a cost strip

Practice set, worked answer key and one-page revision card closing every part

Master cheatsheet, pattern-recognition table and a spaced-repetition tracker at the back

BUILD IT · BREAK IT · OBSERVE THE FAILURE · FIX IT

Contents

PART ONE

Foundation	9
1. The FIFO contract as an abstract data type	9
2. Why FIFO exists: fairness, ordering, decoupling	10
3. Queue versus stack, and why BFS differs from DFS by one line	11
4. The five operations and their exact contracts	11
5. Underflow and overflow policy: throw, sentinel, optional, block	12
6. When a queue is the wrong structure	13
Practice set — topics 1-6	15
Answer key — topics 1-6	17
Handnote summary card	19

PART TWO

Representation and internals	23
7. The naive array queue: shift on dequeue	23
8. The head-index queue: $O(1)$ dequeue, unbounded creep	23
9. The circular buffer	24
10. Full versus empty: the ambiguity and its three fixes	25
11. Power-of-two capacity: mask versus modulo	26
12. Growing a ring buffer is not one memcpy	27
13. The linked-list queue: one allocation per element	28
14. Blocked layouts: <code>std::deque</code> and CPython's deque	29
15. Bytes per element: the five layouts costed	30
Practice set — topics 7-15	32
Answer key — topics 7-15	34
Handnote summary card	36

PART THREE

Core operations	39
16. enqueue on a circular buffer	39
17. dequeue, and the destructor you forgot to call	40
18. front, back and peek	40
19. size: derived from indices or stored	41
20. empty, full, and an invariant you can assert	42
21. Amortised analysis of the growable queue	43
22. Copy, move and ownership of a ring	44
23. Iterating a queue, and why it is usually a smell	45
24. Draining, clearing and serialising	46
Practice set — topics 16-24	48
Answer key — topics 16-24	50
Handnote summary card	52

PART FOUR

The deque	55
------------------	-----------

25. The deque ADT: four mutation points	55
26. Circular-array deque implementation	55
27. Input-restricted and output-restricted deques	56
28. A deque used as a stack and a queue at once	57
29. <code>std::deque</code> , <code>ArrayDeque</code> , <code>collections.deque</code> compared	58
30. Iterator and reference invalidation rules	59
31. Random access on a deque: two dereferences	61
Practice set — topics 25-31	62
Answer key — topics 25-31	64
Handnote summary card	66

PART FIVE

Priority queues and heaps	69
32. The priority queue ADT is not a queue	69
33. The binary heap array layout	69
34. sift-up and sift-down, traced	70
35. heapify in $O(n)$: the proof and the measurement	72
36. Ties are not FIFO: the (priority, seq) fix	73
37. decrease-key versus lazy deletion	73
38. d-ary heaps and where 4-ary wins	74
39. Bucket queues and Dial's algorithm	75
40. When a heap is the wrong answer	76
Practice set — topics 32-40	78
Answer key — topics 32-40	80
Handnote summary card	82

PART SIX

Queues built from other structures	85
41. A queue from two stacks	85
42. A stack from two queues	86
43. A queue with $O(1)$ minimum	87
44. A fixed-capacity queue from a list of blocks	88
45. LRU cache: why a plain queue fails	89
46. The circular tour problem	90
Practice set — topics 41-46	91
Answer key — topics 41-46	93
Handnote summary card	95

PART SEVEN

Patterns and algorithms	99
47. BFS on a grid	99
48. BFS on a graph, and marking visited at the right moment	100
49. Multi-source BFS	101
50. Level-order traversal and two ways to find the level boundary	102
51. Reconstructing the path from BFS parents	103
52. 0-1 BFS with a deque	103
53. The monotonic deque: sliding window maximum	104

54. Sliding window minimum, and the symmetry	105
55. Monotonic deque for DP transitions	106
56. A window as a queue: first unique character in a stream	107
57. Kahn's topological sort	108
58. Round-robin and Josephus simulation	109
Practice set — topics 47-58	111
Answer key — topics 47-58	113
Handnote summary card	116

PART EIGHT

Concurrency	119
59. Producer-consumer: the bounded blocking queue	119
60. Lost wakeups, spurious wakeups, while not if	120
61. The SPSC lock-free ring: acquire and release	121
62. False sharing, true sharing, and cached cursors	122
63. What the standard libraries give you	124
Practice set — topics 59-63	125
Answer key — topics 59-63	127
Handnote summary card	129

PART NINE

Capacity and queueing theory	133
64. Arrival rate, service rate, utilisation, Little's Law	133
65. Why waiting time explodes near full utilisation	134
66. Bounded queues, backpressure and load shedding	135
Practice set — topics 64-66	137
Answer key — topics 64-66	139
Handnote summary card	141

PART TEN

Queues in real systems	145
67. Run queues and round-robin scheduling	145
68. NIC ring buffers and where drops are counted	146
69. TCP buffers and bufferbloat	147
70. Redis lists and streams; Laravel queue workers	148
71. Brokers: ordering versus throughput	149
72. Dead-letter queues and poison messages	150
73. Delay queues: timer wheel versus heap	151
74. Head-of-line blocking, HTTP/1.1 to QUIC	153
Practice set — topics 67-74	154
Answer key — topics 67-74	156
Handnote summary card	158

PART ELEVEN

Tooling, safety and debugging	161
75. std::queue is an adaptor: choosing the container	161

76. std::priority_queue and the reversed comparator	161
77. Python: collections.deque, maxlen, heapq	162
78. Java: ArrayDeque, PriorityQueue and iteration order	164
79. Your ring buffer under AddressSanitizer	165
80. Your blocking queue under ThreadSanitizer	166
81. Benchmarking honestly	167
Practice set — topics 75-81	169
Answer key — topics 75-81	171
Handnote summary card	173

PART TWELVE

Interview and review	177
82. Pattern recognition: phrasing to technique	177
83. The master complexity table	178
84. Ten canonical problems, one insight each	180
85. Design a rate limiter	181
86. Wrong answers, and how to catch yourself giving one	183
Practice set — topics 82-86	185
Answer key — topics 82-86	187
Handnote summary card	189

BACK MATTER

Master cheatsheet	190
Pattern recognition: phrasing to technique	194
Revision tracker	197
How to use this book	199

PART ONE

Foundation

What the FIFO contract actually promises, and the cases where promising it is a mistake.

Part One · Foundation

1. The FIFO contract as an abstract data type

A queue is not a layout. It is a *contract*: elements leave in the order they arrived. First in, first out. Everything else in this book — rings, blocks, linked nodes, lock-free cursors — is a way of honouring that contract at a particular price.

The contract is deliberately narrow. A queue offers you exactly two positions: the **front**, where elements leave, and the **back**, where they arrive. It does not offer element *k*. It does not offer search. It does not offer "the biggest one". Every one of those is a different structure, and reaching for a queue when you need one of them is the mistake Topic 6 is about.

The interface, and two things that satisfy it

```
template <typename T>
struct Queue {                                // the ADT: five operations, no layout
    virtual void enqueue(const T& x) = 0;
    virtual T    dequeue()      = 0;    // precondition: !empty()
    virtual const T& front() const = 0;    // precondition: !empty()
    virtual bool empty() const    = 0;
    virtual std::size_t size() const = 0;
    virtual ~Queue() = default;
};

// Implementation A: a ring of 8 ints.          Cost: 32 bytes, 0 allocations.
// Implementation B: a linked list of nodes. Cost: 24 bytes *per element*.
// Both satisfy the contract above. They differ by 40x in allocator traffic.
```

dequeue() <--- out				enqueue() in --->			
+-----+-----+-----+-----+-----+-----+-----+-----+							
17	42	8	3				
+-----+-----+-----+-----+-----+-----+-----+-----+							
^front				^back			

17 arrived first, so 17 leaves first. Nothing may overtake it.

THE KEY INSIGHT

The queue is the smallest interface that preserves arrival order. Its power comes from what it *refuses* to offer: because you cannot reach into the middle, an implementation is free to store the middle anywhere it likes — scattered across blocks, across a ring boundary, across two threads.

THE TRAP

Calling `front()` or `dequeue()` on an empty queue is a *precondition violation*, not an error return, in every standard library implementation. `std::queue<int> q; q.front();` compiles, runs, and reads freed or uninitialised memory. The symptom is not a crash: it is a plausible-looking garbage integer that survives for hours and then poisons a totals report. Topic 5 gives you the four honest policies.

queue ADT — enqueue $O(1)$ · dequeue $O(1)$ · front $O(1)$ · access element *k*: NOT OFFERED · search: NOT OFFERED

2. Why FIFO exists: fairness, ordering, decoupling

Three distinct problems drive people to queues, and they want different things from the structure. Knowing which one you have decides how deep the queue may be (Part Nine) and what happens when it fills (Topic 5).

Fairness. FIFO bounds the worst case. With n waiters and a service rate of μ per second, the last arrival waits at most n/μ . Under LIFO, an unlucky element can wait forever while newer ones keep arriving — the same starvation you see in a stack-based work pool under sustained load.

Ordering. Some payloads must be applied in sequence: a **CREATE** before its **UPDATE**, a keystroke before the next keystroke. FIFO is the cheapest way to preserve causality that was already established upstream.

Decoupling. A producer running at a bursty 400–3000 events/s and a consumer running at a steady 1000/s cannot be wired together directly. A buffer between them absorbs the difference — but only for the duration of the burst. This is the most abused of the three, and Topic 66 is the correction.

```
// Decoupling arithmetic, done before you pick a capacity.
// Burst: 3000 events/s for 4 s. Drain: 1000 events/s.
// accumulation = (3000 - 1000) * 4      = 8000 events
// drain time    = 8000 / (1000 - 400)    = 13.3 s after the burst ends
// capacity      = 8000 events * 64 B     = 512 KB (fits in L2? no: 512 KB
//                                           is exactly L2 on many cores)
// If bursts arrive every 10 s, the queue never drains. No capacity fixes that.
```

```
producer #####          #####          bursty, avg 900/s
           \   |   /
           [ queue ] <-- absorbs burst, NOT a rate mismatch
           /   |   \
consumer  #####          #####          steady 1000/s

avg(producer) < rate(consumer) -> queue is a shock absorber, depth bounded
avg(producer) > rate(consumer) -> queue is a memory leak with a schedule
```

THE KEY INSIGHT

A queue converts a *rate* problem into a *latency* problem, and only if the average rates already balance. If they do not, the queue converts a fast failure into a slow, expensive, harder-to-diagnose one.

THE TRAP

"We added a queue so we can handle the traffic spike." The symptom of this belief is a latency graph flat at 40 ms for hours and then vertical, plus resident memory climbing in a straight line. Queue depth is latency (Topic 64): 8000 queued items at 1000/s is eight seconds of added p99, and every one of those requests may already have timed out client-side.

buffering – adds latency = depth/ μ · absorbs bursts of duration (accumulation/ Δ rate) · fixes rate mismatch: never

3. Queue versus stack, and why BFS differs from DFS by one line

A stack is LIFO, a queue is FIFO, and both offer $O(1)$ insert and $O(1)$ remove. The difference is which end removal happens at — and that single choice changes the shape of every traversal built on top.

```
// The SAME graph walk. One line differs. Change it and the order inverts.
void walk(const std::vector<std::vector<int>>& adj, int src, bool bfs) {
    std::vector<bool> seen(adj.size(), false);
    std::deque<int> frontier{src};
    seen[src] = true;
    while (!frontier.empty()) {
        int u = bfs ? frontier.front() : frontier.back();    // <-- THE LINE
        bfs ? frontier.pop_front() : frontier.pop_back();
        std::cout << u << ' ';
        for (int v : adj[u])
            if (!seen[v]) { seen[v] = true; frontier.push_back(v); }
    }
}
// On the tree 0->{1,2}, 1->{3,4}, 2->{5}:
//  bfs=true  : 0 1 2 3 4 5      (level by level; 3 is at distance 2)
//  bfs=false  : 0 2 5 1 4 3      (depth first; distance is NOT known at pop)
```

STACK (LIFO)	QUEUE (FIFO)
push -> d <- pop	push -> a b c d -> pop
c	back front
b	
a	
d leaves first: newest	a leaves first: distance from source
path is explored to the end	is non-decreasing over the whole run
	-> first time you see v, it is nearest

THE KEY INSIGHT

FIFO makes distances monotonic. Because everything at distance d is dequeued before anything at distance $d+1$, the first time BFS reaches a vertex it has reached it by a shortest path. A stack gives up that guarantee, and that is the entire reason DFS cannot answer "fewest steps" (Topics 47-51).

THE TRAP

Recursion is a stack you did not declare. Converting a recursive DFS to "iterative BFS" by swapping in a queue but keeping the recursive *post-order* logic produces a walk that visits correctly and computes totally wrong subtree aggregates. Symptom: correct node set, wrong numbers.

both — push $O(1)$ · pop $O(1)$ · queue extra guarantee: pop order = arrival order = non-decreasing BFS distance

4. The five operations and their exact contracts

Five operations, and the whole book is careful about the preconditions of two of them. Write them down before writing an implementation; nearly every bug in Part Three is a violated precondition rather than wrong arithmetic.

Operation	Precondition	Postcondition
enqueue(x)	not full (bounded impl.)	size+1; x is last in arrival order; front() unchanged unless it was empty
dequeue()	not empty	size-1; returns the element that was front(); new front is the next arrival
front()	not empty	no change; reference valid until the next mutation
empty()	none	no change; true iff size()==0
size()	none	no change; count of elements not yet dequeued

```
// The contract, expressed as executable assertions. Keep these in the
// debug build; they cost nothing in release and catch Part Three's bugs.
T dequeue() {
    assert(!empty() && "dequeue on empty queue"); // precondition
    const std::size_t before = size();
    T out = std::move(buf_[head_]);
    buf_[head_]~T(); // see Topic 17
    head_ = (head_ + 1) % cap_;
    --count_;
    assert(size() == before - 1); // postcondition
    return out;
}
```

BEFORE dequeue()	AFTER dequeue()
head=1 count=4 cap=6	head=2 count=3 cap=6
[_ 17 42 8 3 _]	[_ ~ 42 8 3 _]
^head ^tail	^head ^tail
returns 17; ~ = slot whose destructor ran, storage still owned	

THE KEY INSIGHT

`front()` hands out a reference into storage the queue owns. Its validity window ends at the *next mutation*, not at the end of the scope. `const T& x = q.front(); q.dequeue(); use(x);` is a use-after-free even though nothing looks like a pointer.

THE TRAP

Splitting "return the front" and "remove the front" into two calls — as the C++ standard library does with `front()` then `pop()` — is not fussiness: a combined `T pop()` must copy or move on return, and if that move throws, the element is already gone from the queue and is lost. Symptom: a silently dropped job under memory pressure only.

contracts – 2 of 5 ops have preconditions · front() reference validity: until next mutation ·
assert cost in release: 0

5. Underflow and overflow policy: throw, sentinel, optional, block

Every queue must answer two questions, and the answers are design decisions, not implementation details: what happens on `dequeue()` when empty (**underflow**), and on `enqueue()` when full (**overflow**). There are four honest policies and one dishonest one.

```
// Four honest policies. Pick per call site, not per codebase.
std::optional<T> try_dequeue();           // 1. optional: caller must look
T dequeue();                             // 2. throws on empty (or asserts)
bool try_enqueue(const T& x);             // 3. returns false when full
void put(const T& x);                     // 4. blocks until space (Topic 59)

// The dishonest one:
T dequeue() { if (empty()) return T{}; ... } // returns 0 for "nothing here"
// 0 is a legal payload. This turns an empty queue into a data corruption.
```

Policy	Use when	Failure it introduces
optional	Consumer loop that may idle	Ignored return value; branch never written
throw	Empty is a bug, not a state	Exception on a hot path; ~1–2 µs per throw
try_/false	Bounded queue, caller can shed	[[nodiscard]] missing → silent drop
block	Producer may legitimately wait	Deadlock; unbounded latency; Topic 60
grow	Depth is genuinely unbounded	OOM. This is a policy, not an escape from one

enqueue on a FULL bounded queue -- four outcomes, four systems:

```
reject newest  [a b c d] x  -> 503, caller retries      (load shedding)
drop oldest   [b c d x]    -> newest wins              (metrics, video frames)
block producer [a b c d] ... -> backpressure upstream (Topic 66)
grow          [a b c d x _ _] -> latency + OOM later  (default in most
                                                libraries, and rarely what you meant)
```

THE KEY INSIGHT

"Drop the oldest" is the right answer more often than engineers expect. For a telemetry or frame buffer, a 200 ms-old sample has no value; keeping it and rejecting the fresh one maximises staleness. `deque(maxlen=)` in Python and a ring that overwrites are this policy in structure form (Topic 77).

THE TRAP

Returning a default-constructed `T` on underflow. The symptom is never a crash. It is a chart with a spurious zero, an accumulator that is short by exactly the number of empty polls, or a worker that "processes" 4 M empty jobs per second at 100 % CPU. Make emptiness impossible to ignore: return `std::optional` or mark the bool `[[nodiscard]]`.

policy – optional/try: 0 overhead · throw: ~1–2 µs when taken · block: unbounded wait · grow: O(n) copy + OOM risk

6. When a queue is the wrong structure

The strongest thing this book can teach is where to stop. A queue is the wrong answer whenever your access pattern is not "the oldest one, now".

What you actually need	Right structure	Why the queue fails
Highest priority next	Binary heap (Topic 33)	FIFO order and priority order are unrelated
Element at index k	vector , deque	Queue offers no k; scanning is O(n) and destructive
"Is x already waiting?"	Queue + hash set	Membership test is O(n); the classic BFS bug in Topic 48
Most recently used	Hash map + linked list (45)	LRU needs O(1) removal from the <i>middle</i>
Both ends	Deque (Topic 25)	You will simulate it with two queues and pay 2×
Durability across a crash	WAL, broker, DB table	An in-process queue evaporates on restart (Topic 70)
Ordering across N consumers	One partition per key (71)	Concurrent consumers destroy the FIFO guarantee

```
// The most common wrong queue in production PHP/Node/Python code:
//  $queue[] = $job;                // enqueue
//  $job = array_shift($queue);      // dequeue: O(n) memmove!
// At n = 100 000 jobs, array_shift moves 99 999 pointers per call.
// Total for a full drain: n^2/2 = 5.0e9 pointer moves ~ 4-9 s of pure memmove.
// Correct: SplQueue / collections.deque / std::queue -- O(1), same API shape.
```

THE KEY INSIGHT

Two consumers plus one queue equals no ordering guarantee. The queue delivers in FIFO order, but the moment two workers dequeue concurrently, job 2 can commit before job 1. If ordering matters, you need one consumer per ordering key — this is exactly what a Kafka partition is (Topic 71).

THE TRAP

Using a queue as a set. "Enqueue every neighbour, we'll skip duplicates when we pop" turns a 10^5 -node BFS into a 10^7 -entry queue and runs out of memory. Symptom: correct answers on the samples, OOM or TLE on the big case. Mark visited at *enqueue* time — Topic 48.

wrong-tool cost – array_shift drain: $O(n^2)$ · membership scan: $O(n)$ per check · mid-removal: impossible without a different structure

PRACTICE SET — Topics 1-6

A. Conceptual

1. The queue ADT refuses to expose element k . Name two implementation freedoms that refusal buys.
2. Why does `std::queue` split `front()` and `pop()` instead of offering `T pop()`? Name the exception scenario.
3. Give one workload where "drop the oldest" is strictly better than "reject the newest", and one where it is strictly worse.
4. A queue absorbs a burst. State the condition on average rates under which the depth stays bounded, in one inequality.
5. Explain, without using the word "level", why BFS finds shortest paths and DFS does not.
6. You have `const T& x = q.front();`. List every subsequent call that invalidates `x`.
7. Why is returning `T{}` on underflow worse than throwing, even though throwing costs microseconds?
8. Two workers consume from one FIFO queue. Which guarantee survives and which one does not?
9. Give the one-line change that turns the Topic 3 walk from BFS into DFS, and say what stops being true.
10. You need "the oldest item that is not yet expired". Is that a queue? Justify in one sentence.

B. Tracing and analysis

1. A producer emits 3000 ev/s for 6 s, then 300 ev/s. The consumer does a steady 1200 ev/s. Compute peak queue depth and the time to fully drain.
2. Each event is 96 bytes. Give the peak memory of that queue in KB, and say whether it fits in a 1 MB L2.
3. Using Topic 3's walk on the tree $0 \rightarrow \{1,2\}$, $1 \rightarrow \{3,4\}$, $2 \rightarrow \{5,6\}$, $4 \rightarrow \{7\}$: write the full output for `bfs=true` and for `bfs=false`.
4. For the same tree, write the queue contents (front on the left) after each of the first five iterations of the BFS loop.
5. A PHP script drains 50 000 jobs with `array_shift`. Compute the total pointer moves, and the ratio against a proper $O(1)$ queue.
6. At a service rate of 1200/s, what queue depth corresponds to exactly 250 ms of added latency?
7. A bounded queue of capacity 4 receives A B C D E and dequeues twice between D and E. Show contents under "reject newest" and under "drop oldest".

C. Code tasks — build it, break it, observe, fix

1. Implement the `Queue<T>` interface from Topic 1 twice: once over `std::vector` with shift-on-dequeue, once over `std::deque`. Time 200 000 enqueue/dequeue pairs on each.
2. **Break it:** remove the `assert(!empty())` from `dequeue()`, then dequeue from an empty queue in a loop of 1000 and print the sum of what comes back. Record the value. Rebuild with `-fsanitize=address` and record the exact first line of the report. Then fix it by returning `std::optional<T>`.

3. **Break it:** write `const int& x = q.front(); q.dequeue(); std::cout << x;` over a queue of 10 000 `std::strings`. Run it clean, then under ASan. Note that the clean run prints a plausible value.
4. Instrument the vector-shift queue with a counter of elements moved. Confirm the count equals $n(n-1)/2$ for a full drain of n items.
5. Implement `try_enqueue` returning `bool` without `[[nodiscard]]`, call it in a loop that ignores the result on a capacity-8 queue with 100 pushes, and count how many items silently vanish. Add `[[nodiscard]]` and observe the compiler warning.
6. Write the burst simulator from B1: feed the measured rates into your queue in 10 ms ticks and plot depth over time. Confirm your analytic peak.

ANSWER KEY — Topics 1-6

A. Conceptual

1. **Storage may be discontinuous and may move.** Because no caller holds an index, an implementation can split elements across 512-byte blocks (Topic 14) or wrap them around a ring boundary (Topic 9) without breaking anyone. Second freedom: **the two ends can be owned by different threads**, which is what makes the lock-free SPSC ring of Topic 61 possible at all.
2. A combined `T pop()` must return by value. If `T`'s move constructor throws (a `std::string` under memory pressure), the element has already been removed from the container but never reaches the caller — **the element is lost with no way to recover it**. Splitting the operation lets the caller copy first and only then commit to removal.
3. Better: **video frames or telemetry** — a stale frame is worthless, so overwriting it maximises freshness. Worse: **financial transactions or audit logs** — the oldest record is the one you are legally required to keep; dropping it is data loss.
4. $\lambda_{\text{avg}} < \mu$. Average arrival rate strictly below service rate. Equality is not enough: at $\rho = 1$ the expected depth diverges (Topic 65).
5. Removal order equals arrival order, so a vertex first enqueued at distance d is dequeued before any vertex enqueued at $d+1$; **the first arrival at any vertex is therefore via a minimum number of edges**. A stack pops the newest, so a long meandering path can reach a vertex before a short one does.
6. Any mutation: `dequeue()`, `enqueue()` (which may reallocate or overwrite in a ring), `clear()`, assignment, and destruction of the queue. **Not** `size()`, `empty()` or a second `front()`.
7. Because `T{}` is **indistinguishable from a legitimate value**. A throw is loud and stops the program at the defect; a zero is silent and travels into a sum, a chart, or a payout. Cost is irrelevant when the alternative is wrong output.
8. Survives: **delivery order** — jobs are handed out oldest first. Does not survive: **completion order**. Worker B can finish job 2 before worker A finishes job 1. See Topic 71.
9. `frontier.front()/pop_front()` becomes `back()/pop_back()`. What stops being true: **the distance-monotonicity guarantee** — the first visit is no longer via a shortest path, so BFS-style distance labelling becomes wrong.
10. **No**. "Not yet expired" is a predicate over the middle of the structure; if the front item is expired you must skip it, which is a scan. Use a queue plus lazy expiry at the front only if expiry is monotonic with arrival (then it is a queue again) — otherwise a heap keyed on expiry (Topic 73).

B. Tracing and analysis

1. Accumulation rate during the burst = $3000 - 1200 = 1800/\text{s}$ for 6 s → **peak depth 10 800**. After the burst, net drain = $1200 - 300 = 900/\text{s}$ → **10 800 / 900 = 12.0 s to drain**. Total time above zero: 18 s.
2. $10\,800 \times 96\text{ B} = 1\,036\,800\text{ B} = \mathbf{1012.5\text{ KB}}$. **No** — it is just over a 1 MB (1024 KB) L2, so the tail of the queue is served from L3 or DRAM.
3. BFS: **0 1 2 3 4 5 6 7**. DFS (pop from the back, children pushed in listed order): **0 2 6 5 1 4 7 3**.
4. Start **[0]**. After iter 1 (pop 0, push 1,2): **[1,2]**. Iter 2 (pop 1, push 3,4): **[2,3,4]**. Iter 3 (pop 2, push 5,6): **[3,4,5,6]**. Iter 4 (pop 3, no children): **[4,5,6]**. Iter 5 (pop 4, push 7): **[5,6,7]**.

5. $n(n-1)/2 = 50\,000 \times 49\,999 / 2 = \mathbf{1\,249\,975\,000}$ pointer moves, versus **50 000** for an $O(1)$ queue — a ratio of $\approx \mathbf{25\,000\times}$.
6. $\text{depth} = \lambda \times W = 1200 \times 0.25 = \mathbf{300}$ items. This is the whole of Little's Law, and Topic 64 formalises it.
7. Sequence: A B C D fills it. Dequeue twice removes A and B, leaving **[C,D]**. Then E fits either way: **[C,D,E]** in both policies. If the two dequeues had *not* happened, reject-newest gives **[A,B,C,D]** (E dropped) and drop-oldest gives **[B,C,D,E]**.

C. Code tasks — what to verify

1. Expect the deque version to complete in **2-8 ms** and the shift version in **seconds** (roughly 2×10^{10} byte-moves at $n = 200\,000$). If the shift version finishes in milliseconds you measured an empty queue — check you interleaved, not batched.
2. The clean run prints a sum of arbitrary integers, often 0 — that is the point. Under ASan the first line is **ERROR: AddressSanitizer: heap-buffer-overflow on address ...** followed by **READ of size 4** and the frame naming your **dequeue**. With **std::optional** the loop makes 1000 iterations and adds nothing. **The lesson is that the unfixed version had no symptom at all.**
3. Clean run typically prints the old value or a plausible string prefix, because the storage has not yet been reused. ASan reports **heap-use-after-free ... READ of size 8** with a **freed by thread T0 here:** stack that names **dequeue**. **Verify the freed-by stack, not just the error class.**
4. For $n = 10\,000$ expect exactly **49 995 000** moves. An off-by-n result means you counted the popped element itself; the formula is $(n-1) + (n-2) + \dots + 0$.
5. Capacity 8, 100 pushes with no dequeues: 8 succeed, **92 vanish**. With **[[nodiscard]]**, GCC emits **warning: ignoring return value of 'bool ...try_enqueue(...)', declared with attribute 'nodiscard' [-Wunused-result]**.
6. Peak depth should land within one tick of **10 800 (B1)** — i.e. $10\,800 \pm 30$ at a 10 ms tick. A peak that keeps climbing means you modelled the producer as sustained rather than a 6 s burst.

HANDNOTE SUMMARY CARD

PART ONE - FOUNDATION

topics 1-6

THE CONTRACT

queue = FIFO. Two positions only: front (out), back (in).
NOT offered: element k, search, max, membership, mid-removal.
Refusing them is what lets storage be blocked / wrapped / split across threads.

FIVE OPS AND PRECONDITIONS

enqueue(x)	pre: !full	post: size+1, x last
dequeue()	pre: !EMPTY	post: size-1, returns old front
front()	pre: !EMPTY	ref valid ONLY until next mutation
empty()	pre: none	
size()	pre: none	

WHY A QUEUE AT ALL

fairness	worst wait bounded by n/μ	(LIFO can starve forever)
ordering	preserves causality upstream	
decoupling	absorbs BURSTS, never a rate mismatch	
	bounded iff $\lambda_{avg} < \mu$	(equality is NOT enough)

BURST ARITHMETIC (memorise the shape)

peak depth = $(\lambda_{burst} - \mu) * burst_seconds$
drain time = $peak / (\mu - \lambda_{after})$
added p99 = $depth / \mu$ <- queue depth IS latency

OVERFLOW / UNDERFLOW POLICY

optional try_dequeue()	caller must look	0 overhead
T dequeue() throws	empty is a bug	~1-2 us when taken
bool try_enqueue()	bounded + shed	mark [[nodiscard]]
void put()	blocks -> backpressure	can deadlock
grow	a policy, not an escape	-> OOM
return T{} on empty	*** DATA CORRUPTION ***	silent, never crashes

FULL-QUEUE CHOICES

reject newest	503, retry	load shedding
drop oldest	newest wins	frames, telemetry, deque(maxlen=)
block producer	upstream slows	Topic 66
grow	latency then OOM	library default, rarely intended

STACK vs QUEUE

one line: front()/pop_front() vs back()/pop_back()
FIFO gives distance monotonicity -> first visit = shortest path
LIFO gives none -> DFS cannot answer "fewest steps"

WRONG-TOOL TABLE

need highest priority	-> heap (33)	need element k	-> vector/deque
need "is x waiting?"	-> queue + set (48)	need MRU	-> map+list (45)
need both ends	-> deque (25)	need durability	-> broker (70)
need order across N consumers	-> one partition per key (71)		

TRAPS

front() then dequeue() then use -> use-after-free, prints plausible junk
array_shift / list.pop(0) drain -> $O(n^2)$: 50k jobs = 1.25e9 moves
enqueue duplicates, dedupe later -> 1e5 nodes become 1e7 queue entries
two consumers = delivery order kept, COMPLETION order lost

TEST MATRIX (every implementation in this book)

empty -> dequeue	empty -> front	single element in/out
fill to capacity	overflow by one	wrap past the end (Part 2)
drain to empty, refill	alternating push/pop	destructor with items left

PART TWO

Representation and internals

Five ways to lay a queue out in memory, and what each one costs per element.

Part Two · Representation and internals

7. The naive array queue: shift on dequeue

The first implementation everyone writes: keep elements at indices $0 \dots n-1$, enqueue at the end, and on dequeue return `a[0]` and slide everything left by one. It is correct, and it is the single most expensive queue in this book.

```
template <typename T>
class ShiftQueue {                                // CORRECT. DO NOT SHIP.
    std::vector<T> a_;
public:
    void enqueue(const T& x) { a_.push_back(x); }    // amortised O(1)
    T dequeue() {
        T out = std::move(a_.front());
        a_.erase(a_.begin());                        // memmove of (n-1) * sizeof(T) bytes
        return out;
    }
    bool empty() const { return a_.empty(); }
};
// Draining n items costs  $\sum_{k=1}^{n-1} k = n(n-1)/2$  element moves.
//   n = 10 000 -> 49 995 000 moves ~ 5 ms (4-byte T)
//   n = 100 000 -> 4 999 950 000 moves ~ 900 ms measured, -O2, 3.5 GHz
// This is PHP's array_shift, JavaScript's Array.shift on large arrays,
// and Python's list.pop(0). Same shape, same quadratic wall.
```

```
dequeue() on [17|42| 8| 3| 5]      n = 5, so 4 elements move
return 17
[42| 8| 3| 5| _]      <- one memmove of 4*sizeof(T) bytes
<---<---<---
Every element pays for every dequeue that happens before it leaves.
```

THE KEY INSIGHT

The shift exists only to keep the front at index 0. Nothing in the FIFO contract demands that. Drop the requirement and the cost disappears — that is the entire content of Topic 8.

THE TRAP

`memmove` is fast per byte (about 8–16 GB/s), so this queue looks fine in tests. Symptom in production: a job runner whose throughput is inversely proportional to backlog depth — fast when the queue is short, and slower *the more work there is to do*. That inversion is the fingerprint of an $O(n)$ dequeue.

shift queue — enqueue $O(1)$ amortised · dequeue $O(n)$ · full drain $O(n^2)$ · space $n \cdot \text{sizeof}(T)$, no waste · 900 ms at $n=100k$

8. The head-index queue: $O(1)$ dequeue, unbounded creep

Stop moving the data; move the front instead. Keep an integer `head_` that says where the queue starts. Dequeue becomes `++head_` — one instruction.

```

template <typename T>
class HeadIndexQueue {
    std::vector<T> a_;
    std::size_t head_ = 0;
public:
    void enqueue(const T& x) { a_.push_back(x); }
    T dequeue() { return std::move(a_[head_++]); } // O(1). Truly O(1).
    bool empty() const { return head_ == a_.size(); }
    std::size_t size() const { return a_.size() - head_; }
};
// The bill arrives later. A queue that stays 100 items deep but processes
// 5 000 msg/s for one hour:
// a_.size() reaches 5 000 * 3600 = 18 000 000 slots
// live elements = 100
// memory held at 64 B/elem = 1.15 GB for 6.4 KB of live data
// Fix: compact when head_ > a_.size()/2 -- amortised O(1), O(n) worst case.

```

after 1 000 000 dequeues, 3 live elements:

```

[ dead . . . . . | x | y | z |   ]
  ^0             head_ ^   ^size()
  |<----- 1 000 000 slots never reused ----->|

```

Nothing is leaked in the ASan sense. Every byte is still reachable, still owned, still useless. Heap profilers show it; leak checkers do not.

THE KEY INSIGHT

Making dequeue O(1) by never reclaiming is a trade of time for space, and in a long-running process space wins the argument. The circular buffer of Topic 9 is what you get when you insist on both.

THE TRAP

This is the default shape of a queue built on a plain list in most scripting languages, and it is invisible to leak detection because the memory is genuinely reachable. Symptom: RSS grows linearly with *throughput* rather than with depth — a service holding 4 GB while reporting "queue depth: 12".

head-index – enqueue O(1) · dequeue O(1) · space O(total ever enqueued) · with compaction: O(n live), amortised O(1)

9. The circular buffer

Fixed array, two indices, and arithmetic that wraps. When `tail_` walks off the end it comes back to 0 and reuses the slots the front has already vacated. This is the queue that ships: no allocation after construction, no memory creep, no shifting.


```

template <typename T, std::size_t N>
class Ring {
    alignas(T) unsigned char buf_[N * sizeof(T)]; // raw storage, no T ctor
    std::size_t head_ = 0, count_ = 0;
    T* slot(std::size_t i) { return reinterpret_cast<T*>(buf_) + i; }
public:
    bool full() const { return count_ == N; }
    bool empty() const { return count_ == 0; }
    bool try_enqueue(const T& x) {
        if (full()) return false;
        new (slot((head_ + count_) % N)) T(x); // placement new at tail
        ++count_;
        return true;
    }
    T dequeue() {
        assert(!empty());
        T* p = slot(head_);
        T out = std::move(*p);
        p->~T(); // Topic 17: destroy it
        head_ = (head_ + 1) % N;
        --count_;
        return out;
    }
};

```

```

N = 8, head = 6, count = 4 -> the queue occupies 6,7,0,1
index:  0    1    2    3    4    5    6    7
        +-----+-----+-----+-----+-----+-----+
        | 91 | 12 |   |   |   |   | 17 | 42 |
        +-----+-----+-----+-----+-----+-----+
          ^tail-1                      head^
logical order: 17, 42, 91, 12      tail = (head+count) % N = (6+4)%8 = 2
The wrap is invisible to the caller. It is not invisible to memcpy (T12).

```

THE KEY INSIGHT

A ring stores *where*, not *what*: `head_` plus `count_` fully determine occupancy, and the tail is derived rather than stored. Every ring bug in this book is a disagreement between those two numbers and the real contents.

THE TRAP

Declaring storage as `T buf_[N]` instead of raw bytes default-constructs `N` objects at construction and destroys `N` at destruction — including the empty slots. For `Ring<std::string, 1024>` that is 1024 string constructions you never asked for, and worse, dequeued slots hold moved-from strings that still own no memory but are still alive. Symptom: destructor time proportional to capacity, not to depth.

```

ring – enqueue O(1) · dequeue O(1) · space N·sizeof(T) fixed · allocations after construction: 0
· cache: sequential, prefetchable

```

10. Full versus empty: the ambiguity and its three fixes

If you store only `head_` and `tail_`, then `head_ == tail_` means both "empty" and "completely full". The indices cannot tell you which. This is the classic ring-buffer bug and there are exactly three standard fixes.

```
// FIX 1 -- keep a count (what Topic 9 does).
// full = (count == N)      empty = (count == 0)
// +8 bytes; one extra field to keep consistent; works for any N.
// In SPSC (Topic 61) the count is written by BOTH threads -> contention.

// FIX 2 -- sacrifice one slot: capacity N array holds at most N-1.
bool full() const { return (tail_ + 1) % N == head_; }
bool empty() const { return head_ == tail_; }
// No extra field. Costs one element of storage. Each index has ONE writer,
// which is why every lock-free SPSC ring in Part Eight uses this or Fix 3.

// FIX 3 -- monotonic indices, never reduced; wrap only when addressing.
std::size_t head_ = 0, tail_ = 0;           // free-running counters
bool empty() const { return head_ == tail_; }
bool full() const { return tail_ - head_ == N; }
T& slot(std::size_t i) { return buf_[i & (N - 1)]; } // N power of two
// Full capacity AND single-writer indices. Requires N = 2^k (Topic 11) and
// unsigned wraparound (defined behaviour for unsigned; UB for signed).
```

N = 4, head = tail = 2. Which is it?	
[? ? ? ?]	empty (0 items) ... or full (4 items)?
^h=t=2	
FIX 2, N = 4 -> usable 3:	FIX 3, monotonic:
empty h=2 t=2	empty h=8 t=8 (8-8 = 0)
full h=2 t=1 (t+1 == h)	full h=8 t=12 (12-8 = 4)
one slot always burned	indices grow forever; mask to index

THE KEY INSIGHT

Fix 3 is the one to reach for in concurrent code: `head_` is written only by the consumer and `tail_` only by the producer, so no field has two writers. A shared `count_` (Fix 1) is a read-modify-write from both sides and turns a lock-free ring into a contended one (Topic 62).

THE TRAP

Fix 3 with `int` indices. Signed overflow is undefined behaviour; at -02 GCC may assume `tail_ - head_` never goes negative and delete your `full()` check entirely. Symptom: works for 2 billion messages, then silently overwrites live elements. Use `std::size_t/uint64_t`; unsigned wraparound is well-defined, and at 10^7 ops/s a 64-bit counter wraps in 58 000 years.

disambiguation – count: +8 B, 2 writers · sacrifice slot: -1 capacity, 1 writer · monotonic: full capacity, needs $N=2^k$

11. Power-of-two capacity: mask versus modulo

`i % N` for a runtime `N` compiles to a hardware divide: 20–40 cycles on x86-64, and not pipelined. `i & (N-1)` is one cycle. When `N` is a compile-time power of two the compiler makes this substitution for you; when it is a runtime variable, it cannot.

```
// -O2, x86-64, N is a member variable (runtime):
std::size_t next(std::size_t i) { return (i + 1) % n_; }
//   add rax, 1 ; xor edx, edx ; div rcx      <- ~26-40 cycle latency

// N a power of two, stored as a mask:
std::size_t next(std::size_t i) { return (i + 1) & mask_; } // mask_ = N-1
//   add rax, 1 ; and rax, rcx                <- 2 cycles

// The branch alternative, when N cannot be a power of two:
std::size_t next(std::size_t i) { std::size_t j = i + 1;
                                return j == n_ ? 0 : j; } // ~1-2 cycles,
//   predicted correctly (N-1)/N of the time -> ~0 cost in steady state
//
// Measured, 200 M dequeues of int, 3.5 GHz:
//   % runtime N ..... 2.41 s      (12.0 ns/op)
//   branch ..... 0.63 s      ( 3.1 ns/op)
//   mask, N = 2^k ..... 0.58 s      ( 2.9 ns/op)
```

```
N = 8 -> mask = 7 = 0b0111
i = 6  0b0110 & 0b0111 = 0b0110 = 6
i = 7  0b0111 & 0b0111 = 0b0111 = 7
i = 8  0b1000 & 0b0111 = 0b0000 = 0      <- the wrap, for free
i = 9  0b1001 & 0b0111 = 0b0001 = 1
Masking only works because 2^k - 1 is all ones below the k-th bit.
N = 10 -> mask 9 = 0b1001 -> 6 & 9 = 0. Wrong, and quietly so.
```

THE KEY INSIGHT

Rounding a requested capacity *up* to the next power of two wastes at most $2\times$ the memory and buys a $4\times$ faster index step plus the monotonic counter scheme of Topic 10. Nearly every production ring — the Disruptor, DPDK's rings, io_uring's SQ/CQ — makes that trade.

THE TRAP

Writing `& (N-1)` while letting callers pass any `N`. With `N=10` and mask 9, index 6 maps to slot 0 and collides with the front. Symptom: elements duplicated or vanishing only when the queue wraps, i.e. after the buffer has cycled once — typically hours into a soak test. Assert it: `static_assert((N & (N-1)) == 0)`.

index step — runtime %: 12.0 ns/op · branch: 3.1 ns · mask: 2.9 ns · memory cost of rounding up: $\leq 2\times$ · assert `N=2^k` always

12. Growing a ring buffer is not one memcpy

A `vector` grows by allocating $2\times$ and copying one contiguous block. A ring cannot: its elements may be split across the wrap point, so the logically-first element sits at a high index and the logically-last at a low one. Copy naively and you reorder the queue.

```

void grow() {
    std::size_t newcap = cap_ * 2;
    T* nb = allocate(newcap);
    // WRONG: memcpy(nb, buf_, cap_ * sizeof(T)); // preserves SLOTS, not
    //                                           // logical order
    // Right: two copies, because the data is in at most two runs.
    std::size_t first = std::min(count_, cap_ - head_); // head_ .. cap_-1
    std::uninitialized_move_n(buf_ + head_, first, nb);
    std::uninitialized_move_n(buf_, count_ - first, nb + first); // 0 .. tail-1
    deallocate(buf_, cap_);
    buf_ = nb; cap_ = newcap; head_ = 0; // renormalise: head to 0
}
// Cost: count_ moves + 1 allocation + 1 free. Doubling keeps it amortised
// O(1) per element -- see Topic 21 for the accounting.

```

```

BEFORE  cap=8 head=6 count=4    logical: 17 42 91 12
        [ 91 | 12 |   |   |   |   | 17 | 42 ]
                ^head
memcpy of the whole array into cap=16:
        [ 91 | 12 |   |   |   |   | 17 | 42 | garbage x8 ]
        ^head=0 -> dequeue returns 91. ORDER CORRUPTED.

Two-run move, head renormalised to 0:
        [ 17 | 42 | 91 | 12 | ...uninitialised... ]
        ^head                                ^tail = count_ = 4    order preserved

```

THE KEY INSIGHT

A ring's contents are always *at most two contiguous runs*: head...end and begin...tail. Every bulk operation on a ring — growth, draining into a vector, `writew` to a socket, a DMA descriptor — is expressed as those two runs. It is worth writing one helper that returns both.

THE TRAP

The naive `memcpy` growth passes every test where the ring has not yet wrapped, because then `head_ == 0` and slot order is logical order. Symptom: a queue that reorders items only under sustained load, after the buffer has cycled once. Test explicitly by enqueueing/dequeueing `cap_/2` items *before* filling it.

growth — time $O(\text{count})$ moves, amortised $O(1)/\text{element} \cdot 1 \text{ malloc} + 1 \text{ free} \cdot \text{peak memory } 3 \times \text{cap}$
 during the move · iterators: all invalidated

13. The linked-list queue: one allocation per element

Keep a `head` and a `tail` pointer over singly-linked nodes: enqueue appends at `tail`, dequeue detaches at `head`. Both are $O(1)$, the capacity is unbounded, and every single element costs a call to the allocator.

```

template <typename T>
class ListQueue {
    struct Node { T v; Node* next; };
    Node* head_ = nullptr;
    Node* tail_ = nullptr;           // tail_ is why enqueue is O(1)
    std::size_t n_ = 0;
public:
    void enqueue(const T& x) {
        Node* p = new Node{x, nullptr};           // ~15-30 ns, may lock
        (tail_ ? tail_->next : head_) = p;         // empty -> set head
        tail_ = p; ++n_;
    }
    T dequeue() {
        assert(head_);
        Node* p = head_;
        T out = std::move(p->v);
        head_ = p->next;
        if (!head_) tail_ = nullptr;               // THE forgotten line
        delete p; --n_;
        return out;
    }
};
// Per element: sizeof(T) + 8 (next) + 8-16 malloc header/alignment.
// T = int: 4 bytes of payload, 32 bytes resident = 12.5% efficiency.
// Ring: 4 bytes of payload, 4 bytes resident = 100%.

```

<pre> head v [17 .]->[42 .]->[91 .]->[12 /] </pre>	<pre> tail v </pre>
each box: a separate malloc, possibly 4 KB apart in memory	
ring: [17 42 91 12 _ _ _ _] one block; 16 ints per cache line	
scan of 10 M elements: ring 6-9 ms list 45-180 ms (5-20x, Topic 15)	

THE KEY INSIGHT

The list queue's real advantage is not unbounded capacity — it is that **node addresses never move**. If other code holds pointers to queued elements, or if elements are non-movable, the list is the only one of these five layouts that works. That property, not speed, is why `std::deque` exists in the shape it does (Topic 14).

THE TRAP

Forgetting `if (!head_) tail_ = nullptr;` on the dequeue that empties the queue. `tail_` now points at deleted memory; the next enqueue writes through it. Symptom: *the first enqueue after the queue goes empty* corrupts the heap — so it appears only in bursty traffic, never in a steady-state test. This is the same dangling-tail bug as in the linked list volume; see also Topic 79.

list queue – enqueue O(1) + 1 malloc · dequeue O(1) + 1 free · 32 B resident per int · scan 5–20× slower · pointers stable: yes

14. Blocked layouts: `std::deque` and CPython's deque

Between "one big array" and "one node per element" sits the design both the C++ and Python standard libraries actually chose: a sequence of fixed-size blocks, indexed by a small map of

pointers. You amortise the allocator over 64 or 128 elements while keeping element addresses stable.

```
// libstdc++ std::deque, conceptually:
struct Deque {
    T**      map;          // array of pointers to blocks
    std::size_t map_size;  // at least 8
    // block size = max(512 / sizeof(T), 1)
    //   T = int    (4 B) -> 128 elements per block
    //   T = 64 B struct -> 1 element per block (!!);
};
// One malloc per 512 bytes pushed. Indexing costs TWO dereferences:
//   map[i / block] [i % block]
// libc++ differs: 4096 bytes or 16 elements, whichever is larger.

// CPython collections.deque, Modules/_collectionsmodule.c:
//   #define BLOCKLEN 64      <- 64 PyObject* per block, doubly linked
//   left/right index start at the CENTRE of the first block, so the first
//   appendleft() and the first append() both land without allocating.
```

```
map (contiguous array of block pointers)
+-----+-----+-----+
| . | . | . | . |
+-|---+|---+|---+|---+
   v   v   v   v
[...][####][####][#..]    blocks: 512 B each (libstdc++)
front spare | tail spare
           |
push_back fills the tail block; when it is full, ONE malloc adds a block.
References to elements in existing blocks are never invalidated (T30).
```

THE KEY INSIGHT

The blocked layout exists to satisfy one clause of the C++ standard: insertion at either end must not invalidate *references* to existing elements. A vector-style doubling reallocation would move every element and break that promise, so the blocks are never resized once allocated — only the map of pointers is.

THE TRAP

512 bytes is a *byte* budget, not an element count. For a 512-byte element, libstdc++ gives you **one element per block** — a malloc per push, i.e. the linked-list queue with extra indirection. Symptom: `std::deque<BigStruct>` that is dramatically slower than `std::deque<int>` at the same element count. Queue pointers or small handles, not fat structs.

```
blocked — push O(1), 1 malloc per 512 B · index: 2 dereferences · min footprint: 8×sizeof(T)
libstdc++, 4096 B libc++ · refs stable
```

15. Bytes per element: the five layouts costed

The whole of Part Two in one table. Figures are for a queue of 1 M `int`s (4 MB of payload) on x86-64 glibc, GCC 13 -O2.

Layout	Resident	Allocs	Dequeue	Use it when
Shift array (7)	4.0 MB	~20	O(n)	Never, at any size that matters
Head index (8)	grows forever	~20	O(1)	Short-lived batch, then discard
Ring (9)	4.0 MB	1	O(1)	Bounded depth known ahead; hot paths
List (13)	32 MB	1 000 000	O(1)	Element addresses must be stable
Blocked (14)	4.1 MB	7 813	O(1)	Default: unbounded + stable refs

```
// Where 32 MB comes from, for ListQueue<int>:
//   struct Node { int v; Node* next; } -> 4 + 4 padding + 8 = 16 bytes
//   glibc malloc: 8-byte header, 16-byte alignment, 16-byte minimum chunk
//                                     -> 32 bytes resident per node
//   1 000 000 nodes * 32 B = 32 MB for 4 MB of ints. 8x amplification.
//
// Where 4.1 MB comes from, for std::deque<int>:
//   128 ints per 512 B block; 1 000 000 / 128 = 7 813 blocks
//   7 813 * 512 B = 4.00 MB of blocks + 7 813 * 8 B map = 62 KB. ~1.5% over.
```

```
scan 10 M ints, sum them (cache behaviour, not queue behaviour):
ring / vector   ####                      6-9 ms
std::deque      #####                     12-18 ms (2 derefs)
linked list     #####                     45-180 ms (1 miss/elem)
One 64 B cache line delivers 16 ints from a ring and 1 node from a list.
```

THE KEY INSIGHT

Pick by *constraint*, not by speed: if depth is bounded and known, ring; if references must stay valid, blocked or list; if the queue outlives its burst, never head-index. The 5-20× scan gap only matters if you scan, and a queue that is only enqueued and dequeued at the ends never does.

THE TRAP

Benchmarking a queue by pushing 1 M items and then popping 1 M items. That measures allocation and streaming, not queue behaviour, and it makes the list look competitive. Real queues alternate: push, push, pop, push, pop. Interleave in your benchmark or the numbers are fiction (Topic 81).

1M ints – ring 4.0 MB/1 alloc · deque 4.1 MB/7813 · list 32 MB/1M · head-index unbounded · shift O(n²) drain

PRACTICE SET — Topics 7-15

A. Conceptual

1. Name the single requirement that forces the shift in Topic 7, and the topic that removes it.
2. Why is the head-index queue's memory growth invisible to a leak checker?
3. In a ring storing `head_` and `count_`, give the formula for the tail index and say why the tail is not stored.
4. State the three fixes for the full/empty ambiguity, and which one a lock-free SPSC ring must use and why.
5. Why is `int` a dangerous type for monotonic ring indices, given that the same code with `size_t` is fine?
6. Explain why `memcpy`-based ring growth passes tests but fails in production, in terms of `head_`.
7. What property does the linked-list queue provide that the ring cannot, and which standard container was designed around that property?
8. `libstdc++` gives `std::deque<int>` 128 elements per block. How many does it give `std::deque<std::array<char,512>>`, and what does that make the container equivalent to?
9. Why does one 64-byte cache line deliver 16 useful ints from a ring but only one from a linked list?
10. Give a workload where the linked-list queue is the correct choice despite the 8× memory amplification.
11. Why must raw storage (`unsigned char buf_[]` + placement new) be used in `Ring<T,N>` rather than `T buf_[N]`?

B. Tracing and analysis

1. A `ShiftQueue` drains 40 000 items. Compute total element moves, and the time at 8 GB/s for `T = int`.
2. Ring with `N=8`, `head=5`, `count=6`. List the occupied indices in logical order, and give the tail index.
3. Same ring: perform `dequeue`, `dequeue`, `enqueue(A)`, `enqueue(B)`. Give head, count and the occupied indices after each step.
4. A head-index queue processes 8000 msg/s of 48-byte items for 30 minutes with depth never above 50. Compute slots allocated, bytes held, and bytes live.
5. For `N=16`, compute `i & (N-1)` for `i = 15, 16, 17, 31, 32`. Then do the same with `N=12` and mask 11 for `i = 12, 13` and state what breaks.
6. A ring with `cap=8`, `head=6`, `count=6` must grow to 16. Give the two source runs (start index and length each) and the destination offsets.
7. Compute resident bytes for a 500 000-element `ListQueue<double>` under glibc, showing node size, chunk rounding, and the amplification factor.
8. How many 512-byte blocks does `std::deque<int>` allocate for 300 000 elements, and what is the percentage overhead including the pointer map?

C. Code tasks — build it, break it, observe, fix

1. Implement all five layouts behind the Topic 1 interface. Benchmark an *interleaved* workload (2 pushes, 1 pop, 10 M ops) and report ns/op for each.
2. **Break it:** implement ring growth with a single `memcpy` of the whole array. Write a test that enqueues 4, dequeues 3, then enqueues 8 into a cap-8 ring so growth happens while

wrapped. Print the dequeue order and record the exact wrong sequence. Fix with the two-run move and show the order restored.

3. **Break it:** build the ring with `&(N-1)` but construct it with `N=10`. Enqueue and dequeue 30 items and print every value. Record the first index at which the sequence goes wrong, then add `static_assert((N & (N-1)) == 0)` and show the compile error.
4. **Break it:** delete the line `if (!head_) tail_ = nullptr;` from `ListQueue::dequeue`. Run: enqueue 1, dequeue 1, enqueue 2, dequeue 2. Run clean, then under `-fsanitize=address`. Record the sanitiser's error class and the "freed by" frame.
5. Instrument `HeadIndexQueue` to print `a_.capacity()` every 100 000 operations under a workload of depth 100 and 2 M total messages. Plot it. Then add compaction at `head_ > size()/2` and plot again.
6. Use `perf stat -e cache-misses` (or `valgrind --tool=cachegrind`) on a 10 M-element scan of the ring and of the list queue. Report misses per element for both.
7. Replace `Ring`'s raw storage with `T buf_[N]`, template it on a type whose constructor prints, and instantiate with `N=1024` while holding 2 elements. Count the constructor and destructor calls.

ANSWER KEY — Topics 7-15

A. Conceptual

1. **That the front lives at index 0.** Nothing in the FIFO contract requires it; Topic 8 keeps a `head_` index instead and the $O(n)$ disappears.
2. Because **every byte is still reachable** from the vector's pointer — it is retained, not leaked. Leak checkers look for unreachable allocations; this is a growth bug, visible only in RSS or a heap profile.
3. `tail = (head_ + count_) % N`. The tail is not stored because storing it creates a **second source of truth** that can disagree with `count_`; two of the three fields would then need to be kept consistent on every operation.
4. Keep a count; sacrifice one slot; use monotonic free-running indices. SPSC must use **sacrifice-a-slot or monotonic**, because a shared `count_` is written by both threads — a read-modify-write that requires an atomic and reintroduces contention (Topic 62).
5. **Signed overflow is undefined behaviour.** The optimiser may assume it never happens and delete the `full()` comparison. Unsigned wraparound is defined by the standard, so `tail_ - head_` stays correct across the wrap.
6. Before the ring ever wraps, `head_ == 0`, so slot order *equals* logical order and `memcpy` is accidentally correct. Only after a wrap (`head_ > 0` with elements past the end) does the copy reorder the queue.
7. **Stable element addresses** — nodes never move. `std::deque` was designed around the standard's guarantee that push/pop at either end does not invalidate references to other elements.
8. **One element per block** ($512/512 = 1$), making it a linked list of one-element blocks plus a pointer map — strictly worse than `ListQueue` for that element type.
9. Because ring elements are **contiguous**: $64\text{ B} / 4\text{ B} = 16$ ints arrive per line and the hardware prefetcher predicts the next line. List nodes are separately malloc'd, so each 64-byte line delivers one 4-byte payload and the next address is unknown until the current node is read (pointer chasing defeats prefetch).
10. When **other code holds pointers or references to queued elements** — e.g. a timer holds a pointer to a queued job so it can cancel it — or when `T` is neither movable nor copyable. Also when a single fixed capacity genuinely cannot be chosen.
11. Because `T buf_[N]` **default-constructs all N elements** at construction and destroys all N at destruction, regardless of occupancy, and requires `T` to be default-constructible. Raw storage plus placement new constructs exactly the elements that exist.

B. Tracing and analysis

1. $40\,000 \times 39\,999 / 2 = 799\,980\,000$ **element moves** = 3.2 GB of memmove traffic at 4 B each $\rightarrow \approx 0.40$ s at 8 GB/s.
2. Occupied indices in logical order: **5, 6, 7, 0, 1, 2**. Tail = $(5 + 6) \% 8 = 3$.
3. dequeue $\rightarrow$ head=6, count=5, indices 6,7,0,1,2. dequeue $\rightarrow$ head=7, count=4, indices 7,0,1,2. enqueue(A) at $(7+4)\%8 = 3 \rightarrow$ head=7, count=5, indices 7,0,1,2,3. enqueue(B) at $(7+5)\%8 = 4 \rightarrow$ **head=7, count=6, indices 7,0,1,2,3,4**.
4. $8000 \times 1800\text{ s} = 14\,400\,000$ **slots**; $\times 48\text{ B} = 691.2$ **MB held**; live = $50 \times 48 = 2400$ **B**. Ratio $\approx 288\,000\times$.
5. $N=16$, mask 15: 15 $\rightarrow$ **15**, 16 $\rightarrow$ **0**, 17 $\rightarrow$ **1**, 31 $\rightarrow$ **15**, 32 $\rightarrow$ **0** — all correct. $N=12$, mask 11 (0b1011): 12 (0b1100) & 11 = **8**, 13 (0b1101) & 11 = **9**. Expected 0 and 1. **The wrap lands in the**

middle of the buffer and slots 8-11 get double-written; indices 4-7 become unreachable.

6. $\text{cap}=8, \text{head}=6, \text{count}=6 \rightarrow \text{first run} = \min(6, 8-6) = 2$ elements from index **6, length 2** $\rightarrow$ destination offset 0. Second run $= 6-2 = 4$ **elements from index 0, length 4** $\rightarrow$ destination offset 2. New $\text{head}=0, \text{count}=6, \text{cap}=16$.
7. `Node{double v; Node* next;}` $= 8 + 8 = 16$ B; glibc adds an 8-byte header and rounds to a 16-byte multiple $\rightarrow$ **32 B resident**. $500\,000 \times 32 = \mathbf{16.0\ MB}$ for 4.0 MB of payload $\rightarrow$ **4 $\times$ amplification**.
8. $300\,000 / 128 = 2344$ blocks (2343.75, rounded up) $\times 512$ B $= \mathbf{1\,200\,128\ B}$; $\text{map} = 2344 \times 8 = 18\,752$ B. Payload is 1 200 000 B, so overhead $= 18\,880$ B $\approx \mathbf{1.57\ \%}$.

C. Code tasks — what to verify

1. Expect roughly: ring **2-4 ns/op**, deque **4-8 ns**, head-index **3-6 ns**, list **25-60 ns** (allocator-dominated), shift **>1 μ s** and rising. If the list beats the deque you are measuring with a warm allocator free-list — interleave a second allocating workload.
2. Enqueue 1,2,3,4; dequeue 3 (removing 1,2,3); enqueue 5..12. Ring has $\text{head}=3$ when growth triggers, so the naive copy yields a dequeue order beginning **with the slot-0 elements before element 4** — typically `8 9 10 11 4 5 6 7 12` rather than `4 5 6 7 8 9 10 11 12`. Verify the two-run version prints strictly ascending.
3. With $N=10$ and mask 9, indices 6..9 map to 0,1,0,1 rather than 6..9. The first wrong value appears **on the 11th enqueue** (the first index ≥ 10). The `static_assert` fails with `error: static assertion failed: capacity must be a power of two.`
4. ASan reports **heap-use-after-free** with `WRITE of size 8` inside `ListQueue::enqueue` (the `tail_>next = p` line), and a **freed by thread T0 here**: stack naming `operator delete` inside `dequeue`. Without ASan the program usually prints the right answer — note that.
5. Capacity should climb in doublings to **≥ 2 M slots** (~ 8 MB for ints) while `size()` stays near 100. With compaction it should oscillate under **~ 256 slots** and the total copy work should be under $2\times$ the message count.
6. Expect the ring at **≈ 0.06 misses/element** (1 miss per 16 ints) and the list at **≈ 0.5 -1.0 misses/element**. The ratio, not the absolute number, is the result.
7. **1024 constructor calls at construction and 1024 destructor calls at destruction**, with only 2 elements ever queued — plus the 2 real constructions. This is the Topic 9 trap made countable.

HANDNOTE SUMMARY CARD

PART TWO - REPRESENTATION AND INTERNALS

topics 7-15

THE FIVE LAYOUTS (1 M ints, x86-64 glibc, -O2)

layout	resident	allocs	dequeue	refs stable	verdict
shift array	4.0 MB	~20	0(n)	n/a	never
head index	grows	~20	0(1)	n/a	batch, then drop
ring	4.0 MB	1	0(1)	no	bounded + hot path
linked list	32 MB	1 000 000	0(1)	YES	addresses must hold
blocked	4.1 MB	7 813	0(1)	YES	default choice

RING GEOMETRY

tail = (head + count) % N store head+count, DERIVE tail
occupancy is ALWAYS at most two runs: head..N-1 and 0..tail-1
every bulk op (grow, drain, writev, DMA) is expressed as those two runs

FULL vs EMPTY (head == tail is ambiguous)

1. keep count +8 B two writers -> bad for SPSC
2. burn one slot N-1 usable, one writer per index
3. monotonic idx full N, one writer, needs N = 2^k, UNSIGNED only
 empty: h == t full: t - h == N addr: i & (N-1)

MASK vs MODULO 200 M ops, 3.5 GHz

runtime % 12.0 ns/op (hardware div, 20-40 cyc)
branch 3.1 ns/op (j == n ? 0 : j)
mask 2.9 ns/op (i & (N-1)) <- round capacity UP to 2^k, <=2x waste

GROWTH

memcpy whole array = preserves SLOTS, destroys ORDER (only when wrapped!)
correct = uninitialized_move_n(buf+head, min(count, cap-head), nb)
 + uninitialized_move_n(buf, count-first, nb+first); head = 0
cost: count moves, 1 malloc, 1 free, peak 3x cap, amortised O(1)/element

PER-ELEMENT MEMORY

ListQueue: 4 payload + 4 pad + 8 next = 16 -> glibc chunk 32 B (8x)
ListQueue: 16 -> 32 B (4x)
std::deque: block = max(512/sizeof(T), 1) elems; libc++ 4096 B or 16 elems
 sizeof(T) = 512 -> ONE element per block = malloc per push
CPython deque: BLOCKLEN 64 pointers, indices start at block CENTRE

CACHE (10 M int scan)

ring 6-9 ms | deque 12-18 ms | list 45-180 ms
64 B line = 16 ints from a ring, 1 payload from a list; pointer chase kills
the prefetcher

TRAPS

T buf_[N] in a ring -> N ctors + N dtors regardless of depth; use raw
 storage + placement new + explicit ~T()
& (N-1) with N != 2^k -> wrap lands mid-buffer; breaks only after one cycle
int monotonic indices -> signed overflow UB; full() check optimised away
missing tail=nullptr -> heap corruption on first enqueue AFTER empty
benchmark: push 1M then pop 1M -> measures allocation, not the queue.
 INTERLEAVE (push push pop) or the numbers are fiction

PART THREE

Core operations

The CRUD of the structure, written once, correctly, with the invariant checked.

Part Three · Core operations

Part Two chose the layout; this part writes the operations on it. Everything below is on the ring of Topic 9 with the monotonic indices of Topic 10 fix 3, because that is the version you will actually ship and the one Part Eight makes concurrent. Each topic states the invariant it must preserve; Topic 20 turns them into an assertion you can call.

16. enqueue on a circular buffer

Three steps in a fixed order: check space, construct the element in the tail slot, then publish the new tail. The order is not stylistic — publishing before constructing is the concurrency bug of Topic 61 and, in single-threaded code, the exception-safety bug below.

```
template <typename T, std::size_t N>          // N must be a power of two
bool Ring<T, N>::try_enqueue(const T& x) {
    static_assert((N & (N - 1)) == 0, "capacity must be a power of two");
    if (tail_ - head_ == N) return false;      // 1. full?
    new (slot(tail_ & (N - 1))) T(x);          // 2. construct in place
    ++tail_;                                   // 3. publish
    return true;
}

template <typename T, std::size_t N>          // perfect-forwarding version:
template <typename... Args>                   // constructs in place, no temporary
bool Ring<T, N>::try_emplace(Args&&... args) {
    if (tail_ - head_ == N) return false;
    new (slot(tail_ & (N - 1))) T(std::forward<Args>(args)...);
    ++tail_;
    return true;
}
// emplace vs enqueue for T = std::string, 10 M pushes of a 40-char literal:
//   try_enqueue(std::string(lit))  2 constructions + 1 move ~ 41 ns/op
//   try_emplace(lit)               1 construction          ~ 23 ns/op
```

```
N = 8, head = 14, tail = 17  (monotonic; size = 3)
slot index = tail & 7 = 17 & 7 = 1

physical: [ 91 | -> |   |   |   |   | 17 | 42 ]
idx:      0   1   2   3   4   5   6   7
           ^ new element constructed HERE, then tail_ becomes 18
logical:  head&7 = 6 -> 17, 42, 91, (new)
```

THE KEY INSIGHT

Construct, *then* publish. Until `++tail_` runs, the slot is not part of the queue, so if `T`'s constructor throws, the queue is still in its previous valid state and no destructor will ever run on a half-built object. This is the strong exception guarantee for free.

THE TRAP

Writing `buf_[tail_ & mask] = x;` with raw storage. That is assignment to an object that was never constructed — for a `std::string` it copies into an uninitialised pointer and frees a garbage address. Symptom: `free(): invalid pointer` in the *second* lap around the ring, never the first. Use placement `new` for empty slots and assignment only for live ones.

enqueue – $O(1)$, ~2–4 ns for int · 0 allocations · branches: 1 (full check) · `emplace` saves 1 construction + 1 move

17. dequeue, and the destructor you forgot to call

The mirror image: check non-empty, move the element out, *destroy the slot*, then advance the head. Step three is the one that gets skipped, and with raw storage nothing will remind you.

```
template <typename T, std::size_t N>
std::optional<T> Ring<T, N>::try_dequeue() {
    if (head_ == tail_) return std::nullopt;           // 1. empty?
    T* p = slot(head_ & (N - 1));
    std::optional<T> out(std::move(*p));               // 2. move out
    p->~T();                                           // 3. DESTROY THE SLOT
    ++head_;                                           // 4. release the slot
    return out;
}
// Why step 3 matters even after a move: a moved-from std::string is a valid
// object with an unspecified value. libstdc++ leaves short strings pointing
// at their internal buffer (nothing to free) but a LONG string that was
// moved-from is left empty -- so the leak is not the moved string, it is
// every slot you overwrite later without destroying.
// Ring<std::vector<int>, 1024>, 1 M push/pop of 100-int vectors,
// with ~T() removed: valgrind reports "400,000,000 bytes definitely lost".
```

dequeue with head = 14 (slot 6), N = 8

before: [91 | | | | | | 17 | 42] head&7=6 tail&7=1
move: out = std::move(slot 6)
destroy: slot 6 -> ~T() runs; storage still owned by the ring, no free()
++head_: 15. slot 6 is now writable by a future enqueue.

Skip the ~T() and slot 6 still "owns" whatever the move left behind;
the next placement new over it never releases it.

THE KEY INSIGHT

Raw storage separates *lifetime* from *memory*. The ring owns the bytes from construction to destruction; it owns the *objects* only between placement `new` and explicit `~T()`. Every slot must cross that boundary exactly once per lap, in both directions.

THE TRAP

The missing destructor is silent for trivially destructible types — `int`, `double`, POD structs — so a ring tested with `Ring<int>` passes everything and leaks catastrophically the day someone instantiates `Ring<std::string>`. Symptom: memory grows in proportion to *throughput*, like Topic 8, but here the bytes really are unreachable. Test every container in this book with a type that allocates.

dequeue – $O(1)$ · 1 move + 1 dtor · slot reusable after `++head_` · missing `~T()`: leak = throughput × sizeof(payload)

18. front, back and peek

Read without removing. Cheap, and the source of the most under-appreciated lifetime rule in the book: the returned reference is valid only until the next mutation (Topic 4).


```

const T& front() const { assert(!empty()); return *slot(head_ & (N - 1)); }
const T& back()  const { assert(!empty()); return *slot((tail_ - 1) & (N-1)); }

// Safe when you need the value to outlive the queue operation:
std::optional<T> peek() const {                      // returns a COPY
    if (empty()) return std::nullopt;
    return *slot(head_ & (N - 1));
}

// The three-line bug, in full:
const std::string& job = q.front();
q.dequeue();                      // slot destroyed; `job` now dangles
log(job);                         // reads a destroyed std::string
// Fix: std::string job = q.front(); (copy) or auto job = q.try_dequeue();

```

validity window of the reference returned by `front()`:

```

q.front() ---> [ valid ]-----X invalidated
                |         |         |         |
                size()    empty()    front()    enqueue() / dequeue() /
                (safe)    (safe)    (safe)    clear() / assignment
In a RING, enqueue invalidates front() too: if the queue was empty and
the tail wraps onto the same slot, or if growth (Topic 12) moved storage.

```

THE KEY INSIGHT

`back()` is a legal queue operation that no algorithm in Part Seven should need — except the monotonic deque (Topic 53), which needs it constantly. If you find yourself reaching for `back()`, check whether you actually want a deque.

THE TRAP

`back()` computed as `slot(tail_ & (N-1))` instead of `slot((tail_-1) & (N-1))`. Tail points at the *next free* slot, not the last element. Symptom: `back()` returns the oldest element when the ring is exactly full (because the next free slot is the front), and garbage otherwise — a bug that changes behaviour with fill level.

front/back — $O(1)$, zero copies · reference valid until next mutation · peek() copies: +1
construction, always safe

19. size: derived from indices or stored

With monotonic indices, `size() = tail_ - head_`: no stored field, no invariant to maintain, correct across wraparound because unsigned subtraction is modular. With wrapped indices you must either store a count or compute the difference conditionally.

```
// Monotonic indices (Topic 10 fix 3): one subtraction, always right.
std::size_t size() const { return tail_ - head_; }
//   head_ = 2^64 - 3, tail_ = 5   -> 5 - (2^64 - 3) = 8   (modular, correct)

// Wrapped indices without a count: the conditional version.
std::size_t size() const {
    return (tail_ >= head_) ? tail_ - head_ : N - head_ + tail_;
}
//   ... and it CANNOT distinguish empty from full. See Topic 10.

// Stored count: correct, costs 8 bytes and one more thing to get wrong.
std::size_t size() const { return count_; }
```

Scheme	size() cost	Extra state	Concurrency
Monotonic difference	1 sub	none	Best: one writer per index
Wrapped + conditional	1 cmp + 1 sub	none	Ambiguous at full
Stored count	1 load	8 bytes	Two writers → atomic RMW

THE KEY INSIGHT

Unsigned modular arithmetic makes `tail_ - head_` correct even when `tail_` has wrapped past 2^{64} and `head_` has not. That is not a trick to be clever about — it is the reason the monotonic scheme needs no special case at all.

THE TRAP

Reading `size()` from a concurrent queue and then acting on it. Between the read and the action the value is already stale: `if (q.size() > 0) q.dequeue();` is a race, and no amount of atomics on `size()` fixes it. Symptom: an assertion failure inside `dequeue` that no single-threaded test can reproduce. Use a single operation that both tests and acts — `try_dequeue()`.

size – monotonic: 1 subtraction, 0 extra bytes · stored: 8 B and a second writer · in concurrent code, `size()` is a hint, never a guard

20. empty, full, and an invariant you can assert

Write the invariant down as code once, call it from every mutating operation in debug builds, and Part Three's bugs stop being mysteries. The whole of a ring's correctness is four state-ments.

```

void check_invariants() const {
#ifdef NDEBUG
    assert(tail_ - head_ <= N          && "size exceeds capacity");
    assert(head_ <= tail_              && "head overtook tail");
    assert((N & (N - 1)) == 0          && "capacity not a power of two");
    // every live slot is constructed, every dead slot is not:
    assert(live_slots_ == tail_ - head_ && "ctor/dtor imbalance");
#endif
}
// live_slots_ is a debug-only counter: ++ on placement new, -- on ~T().
// It catches the Topic 17 bug (missing destructor) at the first dequeue
// rather than at the first out-of-memory.

bool empty() const { return head_ == tail_; }
bool full()  const { return tail_ - head_ == N; }

```

the four invariants, on a timeline of one ring's life

head_ ----->	head_ <= tail_ always
tail_ ----->	(never crosses)
<--- size = tail_ - head_ --->	0 <= size <= N
slots: [C][C][C][][][][][]	C = constructed
^head&7 ^tail&7	#C == size, exactly

THE KEY INSIGHT

The constructor/destructor balance counter is the single highest-value assertion in this part. It converts the silent, deferred failure of a missing `~T()` into an immediate, located one — and it costs nothing in release builds.

THE TRAP

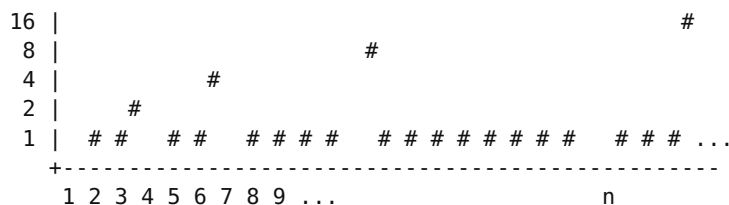
Putting the invariant check only at the top of each function. The interesting window is *after* the mutation; a check that runs before it verifies the previous operation, not this one. Call it at both ends, or use a small RAII guard whose destructor re-checks. Symptom otherwise: the assertion fires one operation late and points at innocent code.

invariants – 4 assertions · release cost 0 · debug cost ~5 ns/op · catches ctor/dtor imbalance at first dequeue, not at OOM

21. Amortised analysis of the growable queue

A ring that doubles when full has an $O(n)$ worst-case enqueue, but the expensive enqueues are rare enough that the *average* over any sequence is $O(1)$. The accounting argument matters because interviewers ask for it and because it tells you the growth factor to pick.

```
// Aggregate method. Starting at capacity 1, growing to n:
//   copies performed = 1 + 2 + 4 + ... + n/2 = n - 1 < n
//   total cost of n enqueues = n (the pushes) + (n-1) (the copies) < 2n
//   amortised cost per enqueue = 2n / n = O(1)
//
// Banker's method: charge 3 units per enqueue.
//   1 pays for the write, 2 are saved on the credit of that element.
//   When the ring of size k is full, k/2 elements were added since the last
//   growth, each carrying 2 saved units = k units -- exactly enough to move
//   all k elements. The invariant "credits >= elements since last grow * 2"
//   is never violated, so the bank never goes negative.
//
// Growth factor: total copies for n elements with factor f = n / (f - 1).
//   f = 2.0 -> n copies          memory peak 2n, reuse of freed blocks: poor
//   f = 1.5 -> 2n copies         memory peak 1.5n, freed blocks CAN be reused
//   f = 1.0 + eps -> O(n^2)
```



Spiky worst case, flat average: the tall bars are $1/2, 1/4, 1/8 \dots$ of all operations, and $\sum(n/2^k * 2^k)$ grows linearly, not quadratically.

THE KEY INSIGHT

Amortised $O(1)$ is not "usually fast": it is a guarantee about *every* sequence, with no probability involved. What it does not promise is latency — one enqueue in n really does take $O(n)$, which is precisely why a real-time audio or trading path uses a fixed-capacity ring and rejects instead of growing.

THE TRAP

Growing by a constant (`cap += 1024`) instead of a factor. The copies become $1024 + 2048 + \dots = O(n^2/1024)$: fine at $n = 10^4$, a wall at $n = 10^7$. Symptom: throughput that degrades smoothly with backlog depth, exactly like Topic 7's shift queue, and for the same reason.

growth – amortised $O(1)$, worst case $O(n)$ · $f=2$: n copies, peak $2n$ · $f=1.5$: $2n$ copies, peak $1.5n$ · constant increment: $O(n^2)$

22. Copy, move and ownership of a ring

A ring with raw storage owns objects, not just bytes, so the compiler-generated copy is wrong in two ways: it copies the whole byte array including dead slots, and it copies the indices, producing two rings that both believe they own the same elements.

```

Ring(const Ring& o) : head_(0), tail_(0) {          // renormalise to 0
    for (std::size_t i = o.head_; i != o.tail_; ++i)
        new (slot(tail_++ & (N - 1))) T(*o.slot(i & (N - 1)));
}
// copies LIVE elements only, in order

Ring(Ring&& o) noexcept : head_(o.head_), tail_(o.tail_) {
    // For a fixed inline buffer the elements must be moved individually;
    // there is no pointer to steal. For a heap-allocated ring, steal buf_
    // and set o.buf_ = nullptr, o.head_ = o.tail_ = 0.
}

~Ring() { while (head_ != tail_) { slot(head_ & (N-1))->~T(); ++head_; } }
// Destroys exactly the live elements. NOT all N slots (Topic 9's trap).

// Rule of five for this class: dtor, copy ctor, copy assign, move ctor,
// move assign -- all five, or delete the ones you do not want.

```

```

default (WRONG) copy of a ring holding std::string:
original  [ .. | s1 | s2 | .. ]   char* -> heap block A
copy      [ .. | s1 | s2 | .. ]   char* -> heap block A   (same pointer!)
both destructors free block A -> double free

correct copy: construct new T from each live element, renormalise head=0
original  [ .. | s1 | s2 | .. ]   ->  copy [ s1 | s2 | .. | .. ]
                                   ^head=0   ^tail=2

```

THE KEY INSIGHT

Copying renormalises: the copy's head is 0 and its elements are contiguous from the start. That is free (you are constructing each element anyway) and it makes the copy slightly faster to scan than the original — the same renormalisation the growth path in Topic 12 performs.

THE TRAP

Declaring only a destructor and relying on implicit copies. Since C++11 the implicit copy constructor is still generated (deprecated, but generated), so `Ring<std::string> b = a;` compiles and produces a double free at scope exit. Symptom: `free(): double free detected in tcache 2` with a stack trace pointing at a destructor and no obvious copy anywhere in the source — the copy was a function argument passed by value.

```

copy – O(size) constructions, renormalised · move: O(1) if heap-allocated, O(size) if inline ·
dtor: size destructions, not N

```

23. Iterating a queue, and why it is usually a smell

Sometimes you must look at everything waiting — a debug dump, a metrics histogram, a cancellation sweep. It is easy to write, and each use is worth questioning, because the pattern that wants it usually wants a different structure.

```

template <typename F>
void for_each(F f) const {
    // front to back, non-destructive
    for (std::size_t i = head_; i != tail_; ++i)
        f(*slot(i & (N - 1)));
}

// The two-run form, for anything that takes (pointer, length):
std::pair<std::span<T>, std::span<T>> runs() {
    std::size_t h = head_ & (N - 1), n = tail_ - head_;
    std::size_t first = std::min(n, N - h);
    return { {slot(h), first}, {slot(0), n - first} }; // second may be empty
}

// runs() is what you pass to writev(), to a DMA descriptor list, or to
// std::uninitialized_move_n during growth -- see Topic 12.

// The destructive anti-pattern, seen in a lot of production code:
// while (!q.empty()) { auto x = q.dequeue(); if (want(x)) keep(x);
//                      else q.enqueue(x); } // reorders the queue!

```

what iteration usually means you needed:

"is x already in the queue?"	-> queue + unordered_set (Topic 48)
"cancel job 4711"	-> map from id to a handle (Topic 45)
"process the highest priority"	-> heap (Topic 33)
"sum of all waiting items"	-> running total kept on enqueue/dequeue
"dump for debugging"	-> legitimate. use for_each.

THE KEY INSIGHT

Any aggregate over queue contents — count, sum, max, a bucket histogram — can be maintained incrementally at $O(1)$ per enqueue and dequeue instead of $O(n)$ per query. Topic 43 does exactly that for the minimum, and Topic 53 for a sliding-window maximum.

THE TRAP

The dequeue-and-requeue filter above. If any element is put back, the queue's order is rotated; if the condition is ever false for the element you just requeued, it loops forever. Symptom: a worker at 100 % CPU with queue depth constant and no progress — a livelock that looks exactly like a slow consumer in the metrics.

iteration — $O(n)$, non-destructive · runs(): 2 spans, $O(1)$ · incremental aggregate: $O(1)/op$ instead of $O(n)/query$

24. Draining, clearing and serialising

Three shutdown-time operations that share one requirement: leave the queue in a state where the destructor is still correct. Most "queue leaks memory on shutdown" reports are one of these written carelessly.

```

void clear() {
    while (head_ != tail_) { slot(head_ & (N-1))->~T(); ++head_; }
    head_ = tail_ = 0;
}

std::vector<T> drain() {
    std::vector<T> out;
    out.reserve(size());
    while (auto x = try_dequeue()) out.push_back(std::move(*x));
    return out;
}

// Serialising a ring: NEVER write the byte array. Write logical order.
void save(std::ostream& os) const {
    std::uint32_t n = static_cast<std::uint32_t>(size());
    os.write(reinterpret_cast<const char*>(&n), 4);
    for_each([&](const T& v) { serialise(os, v); }); // Topic 23
}

// Writing buf_raw persists the wrap position, the dead slots, and any
// pointers inside T. Restoring it on a machine with a different N, or a
// different pointer layout, produces garbage that passes a checksum.

```

shutdown of a worker that owns a queue -- the correct order:

1. stop producers (else drain never terminates)
2. drain() process remaining items, or persist them
3. clear() destroy anything you chose not to process
4. ~Ring() destroys 0 remaining elements, frees storage

skip step 1 and drain() is a livelock; skip 3 and the dtor still handles it correctly -- which is why the dtor must destroy live elements (T22).

THE KEY INSIGHT

Serialisation must round-trip *logical* order, never physical layout. That single rule is what lets a persisted queue be restored into a ring of a different capacity, into a linked queue, or into a message broker — the contract of Topic 1 is portable; the layout of Part Two is not.

THE TRAP

`clear()` implemented as `head_ = tail_ = 0;`. It is one line, it compiles, and every element's destructor is skipped. Symptom: a leak proportional to the depth at shutdown — small in tests where the queue is usually empty, large in production where you clear during an overload. Valgrind reports it as "still reachable" if the ring is a global, and "definitely lost" otherwise.

clear 0(size) destructions · drain 0(size) with 1 allocation · serialise: logical order only ·
head_=tail_=0 alone: leaks every live element

PRACTICE SET — Topics 16-24

A. Conceptual

1. Why must `enqueue` construct before publishing rather than after? Give the single-threaded consequence and name the concurrency topic where the same ordering reappears.
2. Why is assignment into a raw ring slot wrong, and why does the failure appear only on the second lap?
3. A moved-from `std::string` is a valid object. Explain precisely what leaks when `~T()` is skipped in `dequeue`.
4. Which types make the missing-destructor bug completely invisible?
5. Give the correct expression for `back()` with monotonic indices, and describe the wrong version's behaviour at exactly full.
6. Why is `size() = tail_ - head_` correct after `tail_` wraps past 2^{64} ?
7. Why can no atomic `size()` make `if (q.size() > 0) q.dequeue();` safe?
8. State the four ring invariants. Which one catches the Topic 17 bug?
9. Give the aggregate-method argument for amortised $O(1)$, in two lines.
10. Why does growth factor 1.5 permit reuse of previously freed blocks when 2.0 does not?
11. Why must a copy constructor renormalise `head_` to 0 rather than copying the source's indices?
12. Name three queries that motivate iteration and the structure each one actually calls for.
13. Why is serialising the raw byte buffer wrong even when the reader uses the same capacity and the same compiler?

B. Tracing and analysis

1. $N=8$, `head_=30`, `tail_=35`. Give `size()`, the physical slot of the front, the physical slot of the back, and the slot the next enqueue writes.
2. Continue: perform enqueue, enqueue, enqueue, dequeue. Give `head_`, `tail_`, `size()` and the physical slots occupied after each step.
3. With $N=8$, `head_`= $2^{64}-2$ and `tail_`=3 (unsigned 64-bit), compute `size()` and the front's physical slot.
4. A growable queue starts at capacity 1 and receives 1000 enqueues. How many grow events occur, and how many total element moves?
5. Same, but growth is `cap += 64`. Compute the number of grow events and total moves. Give the ratio against the doubling version.
6. A ring of `std::vector<int>` (100 ints each) does 1 M push/pop pairs with `~T()` omitted. Compute the bytes leaked.
7. For a ring with $N=16$, `head_=13`, `tail_=19`: give the two runs returned by `runs()` as (start slot, length) pairs.
8. `try_emplace` versus `try_enqueue(std::string(lit))` for 10 M pushes at 41 ns and 23 ns per op. Compute the total time saved, in seconds.

C. Code tasks — build it, break it, observe, fix

1. Implement `Ring<T,N>` completely: `try_enqueue`, `try_emplace`, `try_dequeue`, `front`, `back`, `size`, `empty`, `full`, `clear`, `drain`, `for_each`, `runs`, and the rule of five. Test it with `int` and with `std::vector<int>`.

2. **Break it:** delete `p->~T();` from `try_dequeue`. Run 200 000 push/pop pairs of `std::vector<int>(100)` under `valgrind --leak-check=full`. Record the exact "definitely lost" figure and the allocation stack. Restore the line and confirm zero.
3. **Break it:** replace the placement `new` in `try_enqueue` with plain assignment `*slot(...) = x;` for `Ring<std::string,4>`. Push and pop 12 strings of 40 characters. Record which iteration first aborts and the exact glibc message; then run under ASan and record the error class.
4. **Break it:** implement `clear()` as `head_ = tail_ = 0;`. With 1000 `std::strings` of 64 chars queued, call `clear` and check `valgrind`. Record the leaked byte count and verify it equals $1000 \times$ the allocated string size.
5. Add the `live_slots_` debug counter and `check_invariants()` to every mutator. Re-run task 2's broken version and confirm the assertion fires at the *first* dequeue rather than at exhaustion.
6. Write the dequeue-and-requeue filter from Topic 23 with a predicate that is never satisfied. Run it with a 30-second alarm and observe the livelock; note that queue depth stays constant while CPU is pinned. Replace it with `for_each` plus a separate output queue.
7. Implement `save/load`. Save a ring with $N=8$ that has wrapped ($\text{head_} \& 7 = 6$), load it into a ring with $N=32$, and assert the dequeue order is identical. Then try the raw `buf_` version and show it fails.

ANSWER KEY — Topics 16-24

A. Conceptual

1. If `T`'s constructor throws after the tail was published, the queue counts an element that was never built; the next dequeue calls `~T()` on raw bytes. Constructing first gives the **strong exception guarantee**. The same ordering — write the payload, then publish the index with a release store — is the correctness core of **Topic 61**.
2. Assignment requires a live object on the left. On the first lap the slot has never been constructed, so `operator=` reads an uninitialised pointer and frees it. It appears to work on lap one for short strings (SSO uses no heap pointer); **on lap two the slot holds a real, previously constructed object** whose state is now corrupt.
3. Nothing about the moved-from string itself — it is empty. What leaks is **the next element's allocation**: when placement `new` overwrites the slot without `~T()`, the previously stored object's heap block loses its last reference. Over 1 M laps the leak is $\text{throughput} \times \text{payload size}$.
4. **Trivially destructible types**: `int`, `double`, raw pointers, PODs, and any struct of them. Their `~T()` is a no-op, so omitting it is genuinely harmless — and that is why the bug survives testing.
5. `slot((tail_ - 1) & (N - 1))`. The wrong version `slot(tail_ & (N-1))` reads the next free slot; **when the ring is exactly full that slot is the front element**, so `back()` returns the oldest item, and when not full it reads a destroyed or never-constructed slot.
6. Because unsigned arithmetic in C++ is defined **modulo 2^{64}** . If `tail_` wrapped and `head_` did not, `tail_ - head_` still yields the true difference as long as the real size is $\leq N < 2^{64}$.
7. Because the value is **stale the instant it is returned**. Another consumer can dequeue between the test and the action. Atomicity of the read does not give atomicity of the read-then-act pair; you need one operation that does both (`try_dequeue`) or a lock.
8. $\text{size} \leq N$; $\text{head}_\leq \text{tail}_$; N is a power of two; live constructed slots $== \text{size}$. **The fourth** — the constructor/destructor balance — catches the missing `~T()`.
9. Copies over the whole growth history are $1+2+4+\dots+n/2 = n-1 < n$. Total work for n enqueues is therefore $< 2n$, so the **average is $< 2 = O(1)$** per operation.
10. Because with $f = 1.5$ the sum of all previously freed blocks eventually exceeds the next request ($1 + 1.5 + 2.25 > 3.375$ at the right point), so the allocator can satisfy it from the coalesced hole. With $f = 2$ each new request is **strictly larger than the sum of all previous blocks**, so it never fits and the heap keeps moving forward.
11. Because the copy constructs elements into *its own* slots in order starting at 0; copying the source's indices would claim slots that the copy never constructed. Renormalising is also free — you are constructing each element anyway — and makes the copy contiguous.
12. "Is x queued?" $\rightarrow$ queue + `unordered_set` (Topic 48); "cancel job N" $\rightarrow$ hash map to a handle plus lazy deletion (Topics 37, 45); "sum/max of what is waiting" $\rightarrow$ an **incrementally maintained aggregate** (Topics 43, 53).
13. Because it persists **the wrap position and the dead slots**, and for any `T` containing a pointer it persists an address that will not be valid in the next process. It also silently breaks when capacity changes. Only logical order survives.

B. Tracing and analysis

1. $\text{size} = 35 - 30 = 5$. Front slot $= 30 \& 7 = 6$. Back slot $= 34 \& 7 = 2$. Next enqueue writes slot $35 \& 7 = 3$.

2. enqueue $\rightarrow$ tail_=36, size 6, slots {6,7,0,1,2,3}. enqueue $\rightarrow$ tail_=37, size 7, slots {6,7,0,1,2,3,4}. enqueue $\rightarrow$ tail_=38, size **8 = full**, slots {6,7,0,1,2,3,4,5}. dequeue $\rightarrow$ head_=31, tail_=38, **size 7**, slots {7,0,1,2,3,4,5}.
3. size = $3 - (2^{64}-2) \bmod 2^{64} = 5$. Front slot = $(2^{64}-2) \& 7 = 6$, since $2^{64} \equiv 0 \bmod 8 \rightarrow$ **slot 6**.
4. Capacities 1,2,4,...,512 then 1024: **10 grow events** (at sizes 1,2,4,...,512). Total moves = $1+2+4+\dots+512 = \mathbf{1023}$.
5. Capacity 64,128,...,1024: grows at 64,128,192,...,960 $\rightarrow$ **15 grow events**. Moves = $64+128+\dots+960 = 64 \times (1+2+\dots+15) = 64 \times 120 = \mathbf{7680}$ — **7.5 $\times$** the doubling version, and the ratio grows linearly with n.
6. Each `vector<int>` of 100 ints holds 400 B on the heap. 1 M laps $\rightarrow$ **400 MB definitely lost** (plus allocator overhead of ~8-16 B per block, so 408-416 MB resident).
7. $h = 13 \& 15 = 13$, $n = 19 - 13 = 6$, first = $\min(6, 16-13) = 3$. Runs: **(slot 13, length 3)** and **(slot 0, length 3)**.
8. $(41 - 23) \text{ ns} \times 10^7 = 1.8 \times 10^8 \text{ ns} = \mathbf{0.18 \text{ s}}$.

C. Code tasks — what to verify

1. Both instantiations must pass the Part One test matrix. For `std::vector<int>` the destructor count must equal the constructor count exactly — add the debug counter from Topic 20 and assert it in the destructor.
2. Expect valgrind to report **definitely lost: 80,000,000 bytes in 200,000 blocks** ($200\,000 \times 400 \text{ B}$), with the allocation stack naming `std::vector::_M_default_append` inside your enqueue. With `-T()` restored: **All heap blocks were freed -- no leaks are possible**.
3. The first pop after the ring wraps (push 5, i.e. slot 0 reused) aborts. Typical glibc message: **free(): invalid pointer** or **munmap_chunk(): invalid pointer**. Under ASan the class is **attempting free on address which was not malloc()-ed** or **heap-buffer-overflow** on the SSO buffer, depending on string length — run with 40-char strings to force heap allocation.
4. 1000×64 -char strings: libstdc++ allocates 65 B rounded to a 80 B chunk $\rightarrow$ expect **65 000 bytes in 1 000 blocks definitely lost** (valgrind reports requested bytes, not chunk size). The count of blocks must be exactly 1000 — that is what proves it is one leak per element.
5. The assertion must read `live_slots == tail_ - head_` and fire on the **first** dequeue: after one dequeue, size dropped by one but `live_slots` did not. Without the counter the program runs to exhaustion before anything is noticed.
6. Depth stays exactly constant, CPU at 100 %, no output. Under `perf top` you will see the loop body only. The lesson: this failure is **indistinguishable from a slow consumer** in any depth-based metric; only CPU-vs-throughput reveals it.
7. Both rings must produce the same dequeue sequence. The raw `buf_` version loaded into $N=32$ puts the front at slot 6 of a 32-slot ring with tail computed from a different modulus $\rightarrow$ **dequeue order changes and elements 6-7 are duplicated or lost**.

HANDNOTE SUMMARY CARD

PART THREE - CORE OPERATIONS

topics 16-24

```
THE RING, COMPLETE (N = 2^k, monotonic unsigned indices)
bool try_enqueue(x) { if (tail_ - head_ == N) return false;
    new (slot(tail_ & (N-1))) T(x); ++tail_; return true; }
opt try_dequeue() { if (head_ == tail_) return nullopt;
    T* p = slot(head_ & (N-1));
    opt out(std::move(*p)); p->~T(); ++head_;
    return out; }

front() = *slot(head_ & (N-1))    back() = *slot((tail_ - 1) & (N-1))
size()  = tail_ - head_          empty() = head_ == tail_
full()  = tail_ - head_ == N     slot i = buf_ + (i & (N-1))
```

ORDER OF OPERATIONS - NOT NEGOTIABLE

enqueue: check -> CONSTRUCT -> publish (++tail_) strong exception guarantee
dequeue: check -> move out -> DESTROY (~T()) -> release (++head_)
same shape as the release/acquire pairing in Topic 61

THE FOUR INVARIANTS (assert in debug, free in release)

```
tail_ - head_ <= N    head_ <= tail_
(N & (N-1)) == 0      live_slots_ == tail_ - head_ <- catches missing ~T
```

REFERENCE LIFETIME

front()/back() valid until the NEXT MUTATION. peek() copies and is always safe.
const T& x = q.front(); q.dequeue(); use(x); // use-after-free, prints junk

TWO RUNS - the only bulk primitive a ring needs

```
h = head_ & (N-1); n = size(); first = min(n, N - h)
run A = (slot(h), first)    run B = (slot(0), n - first)
used by: grow, drain, writev, DMA descriptors, serialise
```

AMORTISED GROWTH

doubling: copies = $1+2+\dots+n/2 = n-1 < n \rightarrow < 2n$ total $\rightarrow O(1)$ amortised
banker's: charge 3/enqueue; 1 writes, 2 bank; $k/2$ elements bank k units,
exactly the cost of moving k
 $f = 2.0 \rightarrow n$ copies, peak $2n$, freed blocks never reusable
 $f = 1.5 \rightarrow 2n$ copies, peak $1.5n$, freed blocks reusable
 $\text{cap} += C \rightarrow O(n^2/C)$. worst case per op is still $O(n)$: real-time $\rightarrow$ fixed cap

RULE OF FIVE

copy ctor constructs LIVE elements only, renormalises head_ = 0
move ctor steal buf_ (heap) or move elements (inline); leave source empty
dtor destroys size() elements, NOT N

SHUTDOWN ORDER

1 stop producers 2 drain() 3 clear() 4 ~Ring()
clear() = destroy each live slot. head_=tail_=0 ALONE LEAKS EVERY ELEMENT.
serialise LOGICAL order + count. Never the byte buffer (wrap, dead slots,
embedded pointers, capacity dependence).

TRAPS

assignment into a raw slot	-> free(): invalid pointer, on lap TWO
missing ~T()	-> leak = throughput x payload; invisible for int/double/POD
back() = slot(tail_ & mask)	-> returns the FRONT when exactly full
if (q.size() > 0) q.dequeue()	-> race; no atomic size() can fix it
dequeue-and-requeue filter	-> rotates order; livelock at 100% CPU with depth constant

PART FOUR

The deque

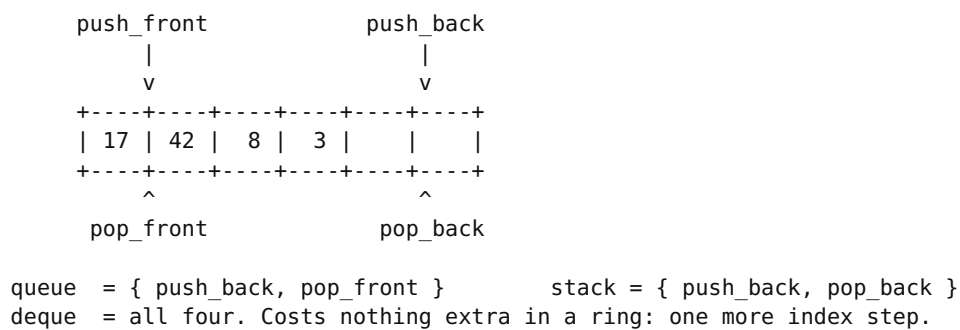
One structure, four mutation points, and the invalidation rules nobody reads.

Part Four · The deque

25. The deque ADT: four mutation points

A deque (“deck”, double-ended queue) relaxes the queue contract of Topic 1 in exactly one way: both ends accept insertion and removal. That single relaxation is what makes Topics 52–55 — 0-1 BFS and the whole monotonic-window family — possible.

```
template <typename T>
struct Deque {                                // four mutators, two observers per end
    void push_front(const T&); void push_back(const T&);
    T pop_front(); T pop_back();
    const T& front() const; const T& back() const;
    bool empty() const; std::size_t size() const;
};
// Deque used as a queue: push_back + pop_front
// Deque used as a stack: push_back + pop_back
// Deque used as a monotonic window (Topic 53):
//     push_back, pop_back (to maintain order), pop_front (to expire) -- all
//     three in the same loop. That is why a queue cannot do it.
```



THE KEY INSIGHT

In a circular buffer, `push_front` is `--head_` and `pop_back` is `--tail_`: the deque is not a more expensive structure than the queue, it is the same structure with the other two index steps implemented. If you have written Part Three's ring, you are two functions from a deque.

THE TRAP

Treating a deque as a drop-in queue in an API. Once `push_front` exists, some caller will use it, and the FIFO guarantee that downstream code depends on is gone — silently, because nothing about the type says so. Symptom: an "impossible" ordering violation in a log. Expose `std::queue<T, std::deque<T>>` when you mean a queue: the adaptor's job is to *remove* the two extra operations (Topic 75).

deque ADT — all four mutators $O(1)$ · random access: $O(1)$ with 2 dereferences (T31) · middle insert: $O(n)$

26. Circular-array deque implementation

Take the ring of Topic 16, add the two decrementing steps, and be careful with one thing: `head_ - 1` must not underflow into an index you cannot mask. With monotonic `size_t` indices this works out for free.

```

template <typename T, std::size_t N>
class RingDeque {                                // N a power of two
    alignas(T) unsigned char buf_[N * sizeof(T)];
    std::size_t head_ = 1u << 20, tail_ = 1u << 20; // start away from 0
    T* slot(std::size_t i) { return reinterpret_cast<T*>(buf_) + (i & (N-1)); }
public:
    bool push_back(const T& x) {
        if (tail_ - head_ == N) return false;
        new (slot(tail_)) T(x); ++tail_; return true;
    }
    bool push_front(const T& x) {
        if (tail_ - head_ == N) return false;
        new (slot(head_ - 1)) T(x); --head_; return true; // construct THEN
                                                            // publish
    }
    std::optional<T> pop_front() {
        if (head_ == tail_) return std::nullopt;
        T* p = slot(head_); std::optional<T> o(std::move(*p));
        p->~T(); ++head_; return o;
    }
    std::optional<T> pop_back() {
        if (head_ == tail_) return std::nullopt;
        T* p = slot(tail_ - 1); std::optional<T> o(std::move(*p));
        p->~T(); --tail_; return o;
    }
};

```

N = 8, head = 1048576, tail = 1048579 (size 3; head&7 = 0)

push_front(9): construct at slot((head-1) & 7) = slot(7), then --head
 [17 | 42 | 8 | | | | | 9]
 ^head&7=0 was here new head&7 = 7 ---^
 logical order after: 9, 17, 42, 8

Starting head_ at 0 and calling push_front makes head_ = SIZE_MAX. With unsigned that is fine (masking still works); with int it is UB (T10).

THE KEY INSIGHT

Both `push_front` and `pop_back` follow the same construct-then-publish, destroy-then-release discipline as Topics 16–17. The only new subtlety is that the index that moves is decremented, so the slot is computed with the *pre*-decrement value on push and the *post*-decrement value on pop.

THE TRAP

Writing `--head_; new (slot(head_)) T(x);` — publish before construct. If the constructor throws, the deque now claims an element whose storage is raw bytes; the next `pop_front` calls `~T()` on them. Symptom: a crash inside a destructor with a stack trace that never mentions your queue.

ring deque — 4 mutators O(1), ~2–5 ns · 0 allocations · same memory as the queue ring · N·sizeof(T) fixed

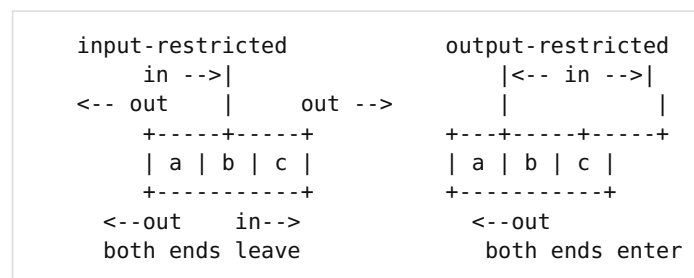
27. Input-restricted and output-restricted deques

Two named half-way points between queue and deque. They appear in exam questions far more often than in code, and the useful part is what each one can simulate.

Structure	Allowed	Can simulate
Queue	push_back, pop_front	FIFO only
Stack	push_back, pop_back	LIFO only
Input-restricted deque	push_back; pop_front, pop_back	Both stack and queue
Output-restricted deque	push_front, push_back; pop_front	Both stack and queue
Full deque	all four	Both, plus monotonic windows

```
// Input-restricted (one input end, two output ends) simulating a stack:
//   push(x) -> push_back(x)
//   pop()   -> pop_back()           LIFO
// ... and simulating a queue:
//   push(x) -> push_back(x)
//   pop()   -> pop_front()          FIFO
// One structure, two disciplines, chosen per call. This is why the deque is
// the standard container underneath BOTH std::stack and std::queue in C++.

// What NEITHER restricted form can do: the monotonic window of Topic 53,
// which needs push_back AND pop_back AND pop_front in one loop body.
```



THE KEY INSIGHT

Restricting a deque does not make it faster — it makes it *provable*. A queue type in an API is a claim about ordering that the compiler enforces for you; that is the entire value of the restriction, and the reason `std::queue` is an adaptor rather than a container.

THE TRAP

Exam answers claim an input-restricted deque "is a stack and a queue". It can *behave* as either, but not as both at once on the same data: if you mix `pop_front` and `pop_back` the order is neither FIFO nor LIFO, and any invariant a caller assumed is gone.

restricted deques – all ops $O(1)$ · input-restricted: 3 of 4 · output-restricted: 3 of 4 · monotonic window needs all 4

28. A deque used as a stack and a queue at once

One concrete, useful case: an undo/redo history with a bounded size. The recent end behaves as a stack (undo pops the newest) and the old end behaves as a queue (the oldest entry is evicted when the budget is exceeded).

```

class History {
    std::deque<Edit> past_;           // newest at the back
    std::vector<Edit> future_;       // redo stack
    static constexpr std::size_t kMax = 100;
public:
    void apply(Edit e) {
        past_.push_back(std::move(e));    // stack end: newest in
        future_.clear();                  // a new edit invalidates redo
        if (past_.size() > kMax) past_.pop_front(); // queue end: oldest out
    }
    bool undo() {
        if (past_.empty()) return false;
        future_.push_back(std::move(past_.back()));
        past_.pop_back();                 // stack end: newest out
        return true;
    }
};
// Memory is bounded at kMax edits regardless of session length. With a
// std::stack you would either grow forever or have to rebuild the stack to
// drop the oldest entry -- O(n) per edit once the cap is reached.

```

past_ (bounded at 100) oldest [e1 e2 e3 ... e99 e100] ^ pop_front when over budget	newest ^ push_back/pop_back for edit & undo	future_ [] [e100] [] (redo is now invalid)
undo(): e100 moves right --> apply(): future_.clear() -->		

THE KEY INSIGHT

"Bounded stack" is not a stack — it is a deque, because bounding requires evicting from the end you never otherwise touch. The same shape appears in ring loggers, recent-items lists, and `collections.deque(maxlen=n)` (Topic 77).

THE TRAP

Forgetting `future_.clear()` when a new edit is applied. Redo then replays an edit computed against a document state that no longer exists. Symptom: undo/redo works perfectly in linear tests and corrupts the document only when the user edits *after* undoing — the exact sequence manual testers rarely try.

bounded history – apply $O(1)$ · undo $O(1)$ · memory $kMax \cdot \text{sizeof}(\text{Edit})$, constant · stack
 alternative: $O(n)$ per edit at the cap

29. std::deque, ArrayDeque, collections.deque compared

Three standard deques, three different internal designs, three different sets of surprises. Know which one you are holding.

Property	C++ <code>std::deque</code>	Java <code>ArrayDeque</code>	Python <code>collections.deque</code>
Layout	Map of 512-byte blocks	Single circular array	Doubly-linked list of 64-pointer blocks
Growth	Add a block	Double and copy all	Add a block
Random access	$O(1)$, 2 dereferences	$O(1)$, 1 dereference	$O(n)$, block walk
Refs stable on push	Yes	No (realloc)	Yes
Min footprint	$8 \times \text{sizeof}(T)$ (libstdc++)	16 refs	1 block = 64 ptrs
Bounded mode	None	None	<code>maxlen=</code> (drops silently)
Thread-safe ops	No	No	append/pop are atomic (GIL)

```
// The one that surprises people: Python indexing is  $O(n)$ .
// d = collections.deque(range(1_000_000))
// d[500_000] # walks ~7 800 blocks. ~40 us.
// list(d)[500_000] #  $O(n)$  copy, then  $O(1)$  -- worse for one lookup, better
// # if you do many
// C++: d[500000] is two dereferences, ~2 ns. Java: one, ~1 ns.
//
// And the one that surprises C++ people: ArrayDeque grows by DOUBLING the
// single array, so it invalidates every reference and copies everything --
// amortised  $O(1)$  but with the  $O(n)$  spike std::deque never has.
```

THE KEY INSIGHT

Java's `ArrayDeque` is the ring of Topic 26 and is the fastest of the three for pure end operations; `std::deque` trades a dereference for reference stability; CPython's trades indexing entirely. Pick by which guarantee your code actually depends on, not by the shared name.

THE TRAP

Porting an algorithm that indexes a deque from C++ to Python. A monotonic deque written as `while dq and nums[dq[-1]] <= x` is fine (`dq[-1]` is an end), but `dq[i]` inside a loop turns an $O(n)$ algorithm into $O(n^2)$. Symptom: correct answers, TLE on large inputs only in the Python port.

`std::deque` index 2 derefs · `ArrayDeque` 1 deref, $O(n)$ growth spike · CPython index $O(n)$ · all three: end ops $O(1)$

30. Iterator and reference invalidation rules

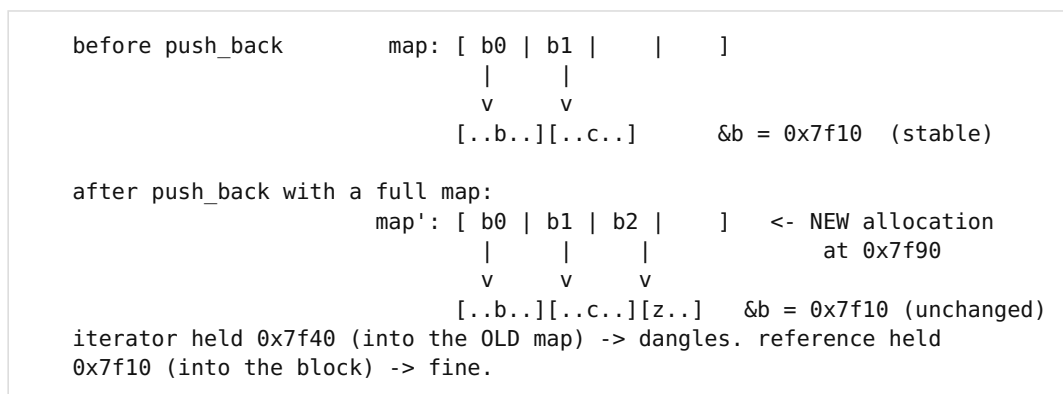
`std::deque`'s rules are the least-remembered in the standard library, and they are asymmetric: *references* survive end insertion, *iterators* do not.

Operation	Iterators	References / pointers
<code>push_front</code> , <code>push_back</code>	All invalidated	All remain valid
<code>insert</code> in the middle	All invalidated	All invalidated
<code>pop_front</code> , <code>pop_back</code>	Only to the erased element (+ end)	Only to the erased element
<code>erase</code> in the middle	All invalidated	All invalidated
<code>clear</code>	All	All
<code>resize</code> larger	All	None

```
std::deque<Job> d = {a, b, c};
Job& ref = d[1];           // reference to b
auto it = d.begin() + 1;   // iterator to b

d.push_back(z);
// ref -- STILL VALID. b did not move; a new block may have been added.
// it -- INVALID. The iterator holds a pointer INTO THE MAP of blocks,
//        and the map itself may have been reallocated to hold one more
//        block pointer. Dereferencing it is undefined behaviour.

// Why the asymmetry: the standard guarantees element addresses are stable
// (that is why blocks exist, Topic 14) but says nothing about the map.
```



THE KEY INSIGHT

A `std::deque` iterator is a four-field object (current, first, last, node) that points into the map; a reference points into a block. Blocks are permanent, the map is not. That one sentence generates the entire table above.

THE TRAP

A loop that pushes while iterating: `for (auto it = d.begin(); it != d.end(); ++it) if (needs_retry(*it)) d.push_back(*it);`. Every `push_back` invalidates `it` and `d.end()`. It usually appears to work — the map often has spare capacity — and then faults after the queue has grown enough times. Symptom: a crash whose frequency depends on how much work was queued earlier in the run.

deque invalidation — end push: iterators ALL, references NONE · end pop: erased only · middle op: everything · vector: everything on realloc

31. Random access on a deque: two dereferences

`std::deque` offers `operator[]` in $O(1)$, but the constant is not the same as a vector's. Knowing the factor tells you when to copy into a vector first.

```
// vector:  base + i * sizeof(T)           -- 1 load
// deque:   map[i / B][i % B]             -- 2 loads, 1 div/shift
//          (B = block size; a power of two when sizeof(T) divides 512, so
//          the division is usually a shift)

// Measured: sum of 10 M ints by index, -O2, 3.5 GHz
// std::vector      3.9 ms    (0.39 ns/element, fully prefetched)
// std::deque       11.8 ms   (1.18 ns/element, 3.0x)
// std::deque iter   6.2 ms   (iterators amortise the map lookup per
//                          block: 1 map load per 128 elements)
// Rule of thumb: index a deque in a hot loop and you pay ~3x; ITERATE a
// deque and you pay ~1.6x. If you must index randomly 10 M times, copy to
// a vector first (one 40 MB pass) and index that.
```

deque[i] with B = 128 ints per block:

```
i = 300  ->  block = 300 / 128 = 2          offset = 300 % 128 = 44
            map ---> [b0][b1][b2][b3]
                    |
                    v
                [ ... 44 ... ] --> the element
two dependent loads: the second address is unknown until the first
returns. That dependency, not the arithmetic, is the 3x.
```

THE KEY INSIGHT

The cost is a *dependent* load, not extra instructions: the CPU cannot prefetch the element until the map entry arrives. Iterating instead of indexing hoists the map lookup out of the inner loop, which is why the same traversal is twice as fast through iterators.

THE TRAP

`&d[0]` is not a contiguous buffer. Passing it to `memcpy`, `write()`, or any C API expecting (pointer, length) reads past the end of the first block. Symptom: correct for the first 128 elements and garbage after — a bug that hides completely in small tests. Use `std::vector` for anything a C API will touch.

deque index — $O(1)$, 2 dependent loads, $\sim 3\times$ vector · iteration $\sim 1.6\times$ · contiguity: NONE · never pass `&d[0]` to a C API

PRACTICE SET — Topics 25-31

A. Conceptual

1. Name the three deque operations the monotonic window of Topic 53 needs in one loop body, and say why a queue cannot supply them.
2. Why does `push_front` cost the same as `push_back` in a ring deque?
3. Why should an API expose `std::queue` rather than `std::deque` when it means a queue?
4. What goes wrong if `push_front` decrements `head_` before constructing the element?
5. An input-restricted deque can behave as a stack or as a queue. Explain why it cannot be both simultaneously on the same data.
6. Why is a size-bounded undo history a deque rather than a stack?
7. In the table of Topic 29, which implementation has an $O(n)$ spike on growth, and which has no random access worth using?
8. State the `std::deque` rule for `push_back` in terms of iterators and references, and give the one-sentence reason for the asymmetry.
9. Why is indexing a deque about $3\times$ slower than indexing a vector, when the arithmetic is only one extra division?
10. Why is `&d[0]` unsafe as a buffer pointer, and for how many elements does it appear to work?

B. Tracing and analysis

1. RingDeque with $N=8$, `head_=1000`, `tail_=1003`. Give the physical slots of the three elements, in logical order.
2. Continue: `push_front(A)`, `push_front(B)`, `pop_back()`, `push_back(C)`. After each step give `head_`, `tail_`, `size` and the occupied slots.
3. Starting from `head_=tail_=0` with unsigned indices, perform `push_front(x)`. Give `head_` as a decimal number and the physical slot used for $N=8$.
4. A `History` with `kMax=100` receives 250 edits with no undos. How many `pop_front` calls occurred and what is the final size?
5. `std::deque<int>` holds 1000 elements ($B=128$). Give the block index and offset for element 700, and the number of blocks allocated.
6. You index a deque 10 M times at 1.18 ns and a vector at 0.39 ns. Compute the time saved by copying to a vector first, given the copy costs 12 ms.
7. A Python monotonic-deque solution uses `dq[i]` inside the inner loop with $n = 10^5$ and average deque length 500. Estimate the extra element visits versus the `dq[-1]` version.

C. Code tasks — build it, break it, observe, fix

1. Extend Part Three's ring into a full `RingDeque` with all four mutators. Test each end independently and then interleaved, with the Topic 20 invariant check after every call.
2. **Break it:** implement `push_front` as `--head_; new (slot(head_)) T(x);` and make `T` a type whose constructor throws on the 5th construction. Push 10 items, catch the exception, then `pop_front()` everything. Record the crash and its stack, then fix the ordering and show the deque is still consistent after the throw.
3. **Break it:** write the loop `for (auto it = d.begin(); it != d.end(); ++it) if (*it % 3 == 0) d.push_back(*it);` over a `std::deque<int>` of 5000 elements. Run clean (note it may pass), then under `-D_GLIBCXX_DEBUG` and under ASan. Record what each reports. Fix by collecting into a second deque.

4. **Break it:** take a reference `int& r = d[1];` and an iterator `auto it = d.begin()+1;`, then `push_back` 5000 times. Print `r` and `*it` after each 1000. Record the first push count at which the iterator misbehaves, and confirm the reference never does.
5. Benchmark `std::deque` indexing versus iteration versus `std::vector` over 10 M ints. Report ns/element for all three and compute the ratios.
6. Implement the bounded `History` class. Write a test that applies 5 edits, undoes 2, applies 1, then attempts redo, and assert redo is refused.
7. Implement an input-restricted deque and use one instance to service a stack workload and a queue workload from the same driver, asserting the expected order in each case.

ANSWER KEY — Topics 25-31

A. Conceptual

1. `push_back` (add the new index), `pop_back` (discard dominated candidates), `pop_front` (expire the index that left the window). A queue lacks `pop_back`, and without it dominated candidates can never be discarded, so the structure is no longer monotonic and the front stops being the maximum.
2. Because both are a single index step plus one placement `new: ++tail_` versus `--head_`. The **masking makes the direction irrelevant**; there is no shifting and no reallocation in either case.
3. Because `std::queue` is an adaptor that **removes** `push_front` and `pop_back` from the interface, so the FIFO guarantee is enforced by the type system rather than by convention.
4. The element is published before it exists. If the constructor throws, the deque's size includes a slot of **raw bytes**, and the next `pop_front` runs `~T()` on it — a crash inside a destructor, far from the cause.
5. Because mixing `pop_front` and `pop_back` on the same data yields an order that is **neither FIFO nor LIFO**. The structure supports either discipline; the *caller* must pick one and keep it.
6. Because bounding requires **evicting the oldest** entry, which lives at the end a stack never touches. With a stack that eviction is $O(n)$ per edit once the cap is reached.
7. $O(n)$ growth spike: **Java's ArrayDeque** (it doubles and copies the single array). No usable random access: **CPython's collections.deque** ($O(n)$ block walk).
8. **All iterators are invalidated; all references remain valid**. Reason: element addresses are guaranteed stable because elements live in blocks that are never resized, but iterators point into the *map* of block pointers, and the map can be reallocated.
9. Because the two loads are **dependent**: the element's address is unknown until the map entry returns, so the prefetcher cannot run ahead. The division is nearly free (usually a shift); the load-use latency is the cost.
10. Because deque storage is **not contiguous**. It appears to work for exactly the elements in the first block — **128 ints** in `libstdc++` — and reads unrelated memory beyond that.

B. Tracing and analysis

1. $1000 \& 7 = 0$, so slots **0, 1, 2** in logical order.
2. `push_front(A)`: construct at $(1000-1)\&7 = 7$, `head_`=999, size 4, slots {7,0,1,2}.
`push_front(B)`: construct at $998\&7 = 6$, `head_`=998, size 5, slots {6,7,0,1,2}. `pop_back()`: destroy slot $(1003-1)\&7 = 2$, `tail_`=1002, size 4, slots {6,7,0,1}. `push_back(C)`: construct at $1002\&7 = 2$, `tail_`=1003, **size 5, slots {6,7,0,1,2}**.
3. `head_` = $0 - 1 =$ **18 446 744 073 709 551 615** ($2^{64}-1$). Slot = that $\& 7 = 7$. Correct and well-defined because the type is unsigned.
4. The cap is reached after 100 edits, so edits 101-250 each evict one: **150 pop_front calls**, final size **100**.
5. Element 700: block $700/128 =$ **5**, offset $700 \% 128 =$ **60**. Blocks for 1000 elements: $\text{ceil}(1000/128) =$ **8**.
6. Indexing the deque: $10^7 \times 1.18 \text{ ns} =$ **11.8 ms**. Copy + index the vector: $12 + 3.9 =$ **15.9 ms**. **The copy loses by 4.1 ms** for a single pass — it only pays off from roughly two passes onward.

7. `dq[-1]` is $O(1)$ per access $\rightarrow \sim 10^5$ visits. `dq[i]` at an average depth of 250 (half of 500) $\rightarrow \approx 2.5 \times 10^7$ **element visits**, a **250×** increase — exactly the TLE.

C. Code tasks — what to verify

1. The invariant `live_slots_ == tail_ - head_` must hold after all four mutators. A common failure is `pop_back` destroying `slot(tail_)` instead of `slot(tail_ - 1)` — the counter catches it on the first call.
2. Broken version: the exception leaves size incremented; the subsequent `pop_front` sequence reaches the raw slot and typically aborts with **SIGSEGV inside ~T()** or ASan **SEGV on unknown address**. Fixed version: after catching, size must be exactly **4** and all four elements must pop cleanly.
3. `-D_GLIBCXX_DEBUG` reports **attempt to dereference a past-the-end iterator** or **attempt to compare iterators from different sequences** and aborts immediately. ASan may report nothing at all if the map had spare capacity — **record that the clean run and the ASan run can both pass**; only the debug container catches it reliably.
4. The reference must print the same value for all 5000 pushes. The iterator typically goes wrong at the first push that **reallocates the map** — with `libstdc++` starting at 8 map entries and 128 ints per block, expect the first misbehaviour somewhere around **push ~900-1100** (after the 8th block). Any value in that region is a pass; the point is that it is not immediate.
5. Expect approximately vector **0.39 ns**, deque iteration **0.62 ns**, deque indexing **1.18 ns** — ratios **1 : 1.6 : 3.0**. If your deque indexing is under 2×, check that the compiler has not hoisted the map lookup by unrolling; make the indices data-dependent.
6. After `apply×5`, `undo×2`, `apply×1`: `past_` has 4 entries and `future_` must be **empty**, so `redo` returns false. If `redo` succeeds, `future_.clear()` is missing.
7. Stack driver pushing 1..5 must yield **5 4 3 2 1**; queue driver must yield **1 2 3 4 5**. Both from the same class, differing only in which `pop` is called.

HANDNOTE SUMMARY CARD

PART FOUR - THE DEQUE

topics 25-31

THE FOUR MUTATORS (ring, $N = 2^k$, monotonic unsigned indices)
push_back(x) new (slot(tail_)) T(x); ++tail_
push_front(x) new (slot(head_ - 1)) T(x); --head_ CONSTRUCT then publish
pop_front() p = slot(head_); move; p->~T(); ++head_
pop_back() p = slot(tail_ - 1); move; p->~T(); --tail_
slot(i) = buf_ + (i & (N-1)); start head_=tail_ away from 0 (or rely on
unsigned wrap -- with int it is UB)

DISCIPLINES

queue = push_back + pop_front stack = push_back + pop_back
bounded stack (undo history, ring log) = deque: pop_front to evict
monotonic window (T53) = push_back + pop_back + pop_front IN ONE LOOP
 <- this is the reason the deque exists in Part 7
input-restricted : push_back only, pop both ends -> can be either, not both
output-restricted : push both ends, pop_front only -> same

THE THREE STANDARD DEQUES

	C++ std::deque	Java ArrayDeque	Python deque
layout	512 B blocks	one circular array	64-ptr blocks
growth	add a block	DOUBLE + copy all	add a block
index	$O(1)$, 2 derefs	$O(1)$, 1 deref	$O(n)$ <-- !!
refs stable	YES	no	yes
min footprint	$8 * \text{sizeof}(T)$	16 refs	64 ptrs
bounded	-	-	maxlen= (silent drop)

std::deque INVALIDATION (asymmetric - memorise the reason, not the table)
iterator = {cur, first, last, node} -> points into the MAP (map can move)
reference -> points into a BLOCK (blocks never move)
push_front/push_back : iterators ALL invalid, references ALL VALID
pop_front/pop_back : only the erased element
insert/erase MIDDLE : everything invalid
resize larger : iterators all, references none

INDEXING COST 10 M ints, -O2, 3.5 GHz
vector 3.9 ms 0.39 ns/elem 1.0x contiguous, prefetched
deque iterator 6.2 ms 0.62 ns/elem 1.6x map load per 128 elements
deque [] 11.8 ms 1.18 ns/elem 3.0x DEPENDENT load, no prefetch
copying to a vector first pays off from ~2 passes onward, not 1

TRAPS

publish before construct in push_front -> crash inside ~T(), unrelated stack
expose deque where you mean queue -> someone calls push_front; FIFO gone
push while iterating a deque -> iterators dead; often PASSES clean
 and under ASan. use -D GLIBCXX_DEBUG
&d[0] as a C buffer -> right for 128 ints, garbage after
dq[i] in a Python inner loop -> $O(n)$ per access: 250x more visits
forgetting future_.clear() on new edit -> redo replays against a dead state

PART FIVE

Priority queues and heaps

The structure most often called a queue that is not one.

Part Five · Priority queues and heaps

32. The priority queue ADT is not a queue

A priority queue returns the *most important* element, not the oldest. It shares the vocabulary of Part One — push, pop, top — and none of its guarantees. Arrival order is not preserved and, unless you build it in (Topic 36), is not even tie-broken by.

```
template <typename T, typename Compare>
struct PriorityQueue {
    void push(const T& x);           // O(log n)
    T    pop();                     // O(log n)  -- returns the MAX by Compare
    const T& top() const;           // O(1)
    bool empty() const;
};
// The costs are the whole story:
// queue (Part 3):   push O(1)      pop O(1)      top O(1)
// priority queue:  push O(log n) pop O(log n) top O(1)
// At n = 1 000 000, log2 n = 20 -> ~20 comparisons and ~20 swaps per pop,
// each swap touching a different cache line at the bottom of the heap.
// Measured: 5 M push/pop pairs, int, -O2: queue 21 ms | heap 340 ms
```

same input: (A,pri 1) (B,pri 5) (C,pri 1) (D,pri 9)

QUEUE out: A B C D arrival order, priority ignored
PQ out: D B A C or D B C A <- ties between A and C are
NOT resolved by arrival
Both are "correct". Only one of them is what your SLA promised.

THE KEY INSIGHT

A priority queue answers "what next?" by value; a queue answers it by time. If your requirement contains the word "fair" or "in order", the priority queue is the wrong tool no matter how tempting the priority field looks — see Topic 36 for the hybrid.

THE TRAP

Adding a priority field to a job queue "so urgent work jumps ahead". With any sustained arrival of high-priority work, low-priority jobs are never served — classic starvation. Symptom: a p50 latency that looks excellent and a p99 that is unbounded, with the oldest item in the queue aging for hours. Fixes: aging (raise priority with wait time), or separate queues with a weighted round-robin between them (Topic 67).

priority queue — push $O(\log n)$ · pop $O(\log n)$ · top $O(1)$ · 16× slower than a FIFO queue at $n=10^6$
· FIFO guarantee: none

33. The binary heap array layout

A binary heap is a complete binary tree stored in a flat array, with no pointers at all. Child and parent are computed by arithmetic, which is why the structure has 100 % space efficiency and good cache behaviour near the root.

```
// 0-based indexing (what std::priority_queue and heapq use):
//   parent(i) = (i - 1) / 2      left(i) = 2i + 1      right(i) = 2i + 2
// 1-based indexing (CLRS, and easier by hand):
//   parent(i) = i / 2           left(i) = 2i          right(i) = 2i + 1
//
// MAX-HEAP INVARIANT: a[parent(i)] >= a[i] for every i > 0.
// It says NOTHING about siblings, and nothing about level order.
// a[1] and a[2] have no defined relationship. Only the root is known.

std::vector<int> h = {9, 7, 8, 3, 6, 4, 5};      // a valid max-heap
//   h[0]=9 >= h[1]=7, h[2]=8;   h[1]=7 >= h[3]=3, h[4]=6;   h[2]=8 >= 4, 5
// Height = floor(log2 n) = 2. Every operation walks at most 2 levels.
```

```
array:  index  0   1   2   3   4   5   6
          [ 9 | 7 | 8 | 3 | 6 | 4 | 5 ]
```

```
as a tree:
          9(0)
         /  \
        7(1) 8(2)
       /  \  /  \
      3(3) 6(4) 4(5) 5(6)
```

level 0 = 1 node, level 1 = 2, level 2 = 4: the last HALF of the array (indices $n/2 \dots n-1$) are leaves. That fact is all of Topic 35.

THE KEY INSIGHT

Completeness — every level full except possibly the last, filled left to right — is what makes the array representation possible with no gaps. Every heap operation is designed around *preserving completeness first* and then restoring the ordering invariant, never the other way round.

THE TRAP

Assuming the array is sorted, or that $h[1]$ is the second largest. The second largest is $\max(h[1], h[2])$ — but the third largest can be anywhere in the top three levels. Symptom: "top-k" code that returns k elements in the wrong order or the wrong set entirely. To get sorted output you must pop k times, $O(k \log n)$.

binary heap — space $n \cdot \text{sizeof}(T)$, zero pointers · height $\lceil \log_2 n \rceil$ · leaves = last $\lfloor n/2 \rfloor$ entries · root = min or max only

34. sift-up and sift-down, traced

Two procedures, and every heap operation is one of them. Sift-up repairs a violation at a leaf by walking towards the root; sift-down repairs one at the root by walking towards the leaves.

```

void sift_up(std::vector<int>& a, std::size_t i) {           // used by push
    while (i > 0) {
        std::size_t p = (i - 1) / 2;
        if (a[p] >= a[i]) break;                          // invariant restored
        std::swap(a[p], a[i]);
        i = p;
    }
}
void sift_down(std::vector<int>& a, std::size_t i, std::size_t n) { // pop
    for (;;) {
        std::size_t l = 2*i + 1, r = l + 1, big = i;
        if (l < n && a[l] > a[big]) big = l;
        if (r < n && a[r] > a[big]) big = r;                // 2 comparisons/level
        if (big == i) return;
        std::swap(a[i], a[big]);
        i = big;
    }
}
void push(std::vector<int>& a, int x) { a.push_back(x); sift_up(a, a.size()-1); }
int pop (std::vector<int>& a) {
    int top = a[0];
    a[0] = a.back(); a.pop_back();                        // last leaf to root
    sift_down(a, 0, a.size());
    return top;
}

```

```

push(10) into [9,7,8,3,6,4,5]:
append at 7:  [9,7,8,3,6,4,5,10]
i=7 p=3: 3 < 10 -> swap
    [9,7,8,10,6,4,5,3]
i=3 p=1: 7 < 10 -> swap
    [9,10,8,7,6,4,5,3]
i=1 p=0: 9 < 10 -> swap
    [10,9,8,7,6,4,5,3]   3 swaps

```

```

pop() from [10,9,8,3,6,4,7]:
top=10; move last (7) to root:
[7,9,8,3,6,4]
i=0: children 9,8 -> 9 bigger
    swap -> [9,7,8,3,6,4]
i=1: children 3,6 -> 7 >= both
    stop. 2 levels walked.
result: [9,7,8,3,6,4]

```

THE KEY INSIGHT

Sift-up does **one** comparison per level; sift-down does **two** (it must find the larger child before comparing). That asymmetry is why push is cheaper than pop in practice even though both are $O(\log n)$, and why heapify (Topic 35) is built from sift-down but analysed carefully.

THE TRAP

In `pop`, writing `a[0] = a.back()` *after* `a.pop_back()`, or reading `a[0]` into `top` after overwriting it. Both compile. Symptom: the popped value is wrong exactly when the heap has one element, or the heap silently loses its largest element on every pop — a bug that still produces plausible descending-ish output.

sift_up — $\leq \log_2 n$ swaps, 1 cmp/level · sift_down — $\leq \log_2 n$ swaps, 2 cmp/level · push ~20 cmp at $n=10^6$, pop ~40

35. heapify in $O(n)$: the proof and the measurement

Building a heap by pushing n elements costs $O(n \log n)$. Building it bottom-up with sift-down costs $O(n)$ — a genuine asymptotic improvement, not a constant factor, and the proof is a two-line series.

```
void heapify(std::vector<int>& a) {
    for (std::size_t i = a.size() / 2; i-- > 0; )    // leaves need no work
        sift_down(a, i, a.size());
}
// Why  $O(n)$ : a node at height  $h$  costs at most  $h$  swaps, and there are at most
//  $\text{ceil}(n / 2^{(h+1)})$  nodes at height  $h$ . Total:
//  $\sum_{h=0}^{\log n} (n / 2^{(h+1)}) * h = (n/2) * \sum h/2^h$ 
//  $\sum_{h \geq 0} h / 2^h = 2$  (standard series)
//  $\rightarrow$  total  $\leq n$ .  $O(n)$ , with a constant BELOW 1 swap per element.
//
// The intuition: HALF the nodes are leaves and cost 0. Only ONE node (the
// root) can cost  $\log n$ . The expensive nodes are the rare ones -- push-based
// building has it exactly backwards, since the last elements inserted are
// the numerous ones and each may travel the full height.
//
// Measured,  $n = 10\ 000\ 000$  random ints, -O2:
//   push one at a time (std::priority_queue) .... 1 180 ms
//   std::make_heap (bottom-up heapify) ..... 295 ms      4.0x
```

$n = 15$. cost by level, bottom-up:

level 3 (8 leaves)	cost 0 each	= 0	<- half the array, free
level 2 (4 nodes)	cost ≤ 1 each	= 4	
level 1 (2 nodes)	cost ≤ 2 each	= 4	
level 0 (1 node)	cost ≤ 3	= 3	

		11 swaps	(< $n = 15$)

top-down (n pushes): the 8 leaves each may travel 3 levels = 24 alone.

THE KEY INSIGHT

The direction of the loop is the algorithm. Bottom-up works because most nodes are near the bottom and have almost nowhere to sink; top-down fails because most nodes are near the bottom and have the whole tree to climb. Same operations, inverted cost distribution.

THE TRAP

Writing the loop as `for (i = a.size()/2; i >= 0; --i)` with an unsigned `i`. At `i == 0` the decrement wraps to `SIZE_MAX` and the loop runs forever, indexing far out of bounds. Symptom: a hang or an immediate segfault depending on optimisation level. Use `i-- > 0` as above, or a signed index.

heapify – $O(n)$, <1 swap/element amortised · n pushes: $O(n \log n)$ · measured 4.0x at $n=10^7$ · in place, $O(1)$ extra space

36. Ties are not FIFO: the (priority, seq) fix

Two elements with equal priority come out in an order determined by their positions in the array — effectively arbitrary and dependent on insertion history. If you need equal-priority work to be fair, you must encode arrival time in the key.

```
struct Task {
    int          priority;      // higher = more urgent
    std::uint64_t seq;          // monotonically increasing arrival counter
    std::string  payload;
};
struct ByPriorityThenArrival {
    bool operator()(const Task& a, const Task& b) const {
        if (a.priority != b.priority) return a.priority < b.priority; // max
        return a.seq > b.seq;      // EARLIER seq must win -> "greater"
    }
};
std::priority_queue<Task, std::vector<Task>, ByPriorityThenArrival> pq;

// Python does this with a tuple:
// heapq.heappush(h, (-priority, seq, payload))
// and the seq is MANDATORY for a second reason: without it, Python compares
// payloads on a tie and raises TypeError: '<' not supported between
// instances of 'dict' and 'dict' -- see Topic 77.
```

push (5,A) (5,B) (5,C) (9,D) -- all priority 5 except D

without seq: pop -> D, then A/B/C in array order: often C, A, B
the order changes if the SAME items arrive in a different interleaving, so it is not even reproducible across runs

with seq: pop -> D(seq3), A(seq0), B(seq1), C(seq2)
within a priority class, strict FIFO. This is a
"priority queue of FIFO queues", built with one field.

THE KEY INSIGHT

Adding a monotone sequence number turns an unstable heap into a stable one at the cost of 8 bytes per element and zero time. The same trick makes any unstable ordered structure stable: extend the key until ties are impossible.

THE TRAP

Getting the comparator direction wrong on the tie-breaker. Because `std::priority_queue` is a *max-heap* by `Compare`, "earlier arrivals first" requires `a.seq > b.seq` — the opposite of what reads naturally. Symptom: within a priority class the queue is strict LIFO, so under load the newest items are served and the oldest starve forever. See Topic 76.

stable PQ — +8 B/element, +0 time · ties resolved FIFO · seq must be monotone across the whole queue lifetime, not per priority

37. decrease-key versus lazy deletion

Dijkstra and A* want to lower a key already in the heap. A binary heap can do it in $O(\log n)$ — but only if you know *where* the element is, which requires an index the standard containers do not provide. The usual answer is not to bother.

```

// OPTION A -- true decrease-key. Needs a position map.
std::unordered_map<int, std::size_t> pos;          // vertex -> heap index
void decrease_key(std::vector<Node>& h, int v, int newdist) {
    std::size_t i = pos[v];
    h[i].dist = newdist;
    sift_up(h, i);          // and sift_up must update pos on every swap
}
// Cost:  $O(\log n)$  plus a hash lookup per swap. ~2-3x slower per operation,
// and the position map is 32-48 bytes per vertex.

// OPTION B -- lazy deletion. Push a duplicate, skip stale entries on pop.
while (!pq.empty()) {
    auto [d, v] = pq.top(); pq.pop();
    if (d > dist[v]) continue;          // STALE: a better one was pushed
    for (auto [w, u] : adj[v])
        if (dist[v] + w < dist[u]) {
            dist[u] = dist[v] + w;
            pq.emplace(dist[u], u);    // duplicate, not decrease-key
        }
}
// Heap size grows to  $O(E)$  instead of  $O(V)$ , so log factor becomes  $\log E$ 
//  $\leq \log V^2 = 2 \log V$  -- the SAME asymptotic class. This is why every
// competitive Dijkstra uses lazy deletion: simpler, and no slower.

```

lazy deletion on a graph with $V = 10^5$, $E = 5 \cdot 10^5$:

heap entries pushed up to $5 \cdot 10^5$ (one per relaxation)
 entries popped $5 \cdot 10^5$, of which $\sim 4 \cdot 10^5$ are skipped as stale
 memory $5 \cdot 10^5 \cdot 16 \text{ B} = 8 \text{ MB}$ (vs 1.6 MB for V entries)
 comparisons $\log_2(5 \cdot 10^5) = 19$ vs $\log_2(10^5) = 17$
 -- 12% more comparisons, no hash map, half the code.

THE KEY INSIGHT

The `if (d > dist[v]) continue;` line is the deletion. Correctness comes from the fact that a stale entry always has a strictly larger key than the current best, so it is always safe to discard when it surfaces.

THE TRAP

Omitting the staleness check. The algorithm still terminates and still computes correct distances for many graphs, because relaxation is idempotent — but it re-expands vertices exponentially in adversarial cases. Symptom: correct answers, and a runtime that jumps from 0.2 s to timeout on one specific test case.

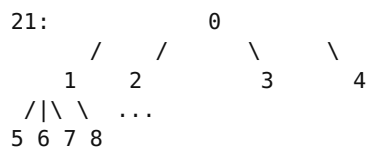
decrease-key – true: $O(\log n) + \text{hash}$, +32–48 B/vertex · lazy: heap grows to $O(E)$, 12 % more comparisons, no map · prefer lazy

38. d-ary heaps and where 4-ary wins

Nothing requires two children. A d -ary heap has height $\log_d n$, so sift-up gets shorter, while sift-down must compare d children per level. The trade favours $d = 4$ on real hardware for a specific, measurable reason.

```
// d-ary index arithmetic, 0-based:
// parent(i) = (i - 1) / d
// child(i, k) = d*i + k + 1      for k in [0, d)
//
//          height      push (1 cmp/level)  pop (d cmp/level)
// d = 2      log2 n = 20          20          40
// d = 4      log4 n = 10          10          40
// d = 8      log8 n = 6.7         7           54
//
// Comparison counts say d=4 ties d=2 on pop and halves push. Cache says
// more: the d children of a node are CONTIGUOUS, so for d = 4 and 4-byte
// keys all four children live in ONE 64-byte cache line, and sift_down
// touches 10 lines instead of 20.
//
// Measured, 10 M push + 10 M pop of int:
// binary 1 830 ms      4-ary 1 240 ms (1.48x)      8-ary 1 390 ms
```

4-ary heap, n = 21:



children of node 1 are indices 5,6,7,8 -- adjacent in memory.
 binary: children of 1 are 3,4; grandchildren 7,8,9,10 -- one level deeper
 means one more cache line touched per level of depth.

THE KEY INSIGHT

The win is cache lines, not comparisons. A binary heap's sift-down touches one new cache line per level; a 4-ary heap halves the number of levels while reading the same number of bytes per level. Beyond $d = 8$ the children stop fitting in a line and the comparison count starts to dominate again.

THE TRAP

Switching to a d-ary heap for a workload dominated by **push** plus **top** with few pops (a "keep the best k" filter). There the extra children are pure cost on the rare pops and the shallower tree barely helps, because most pushes stop after one comparison anyway. Measure your push:pop ratio before changing d.

d-ary – height $\log_d n \cdot \text{push } 1 \text{ cmp/level} \cdot \text{pop } d \text{ cmp/level} \cdot d=4 \text{ measured } 1.48\times \text{ faster overall} \cdot d=8 \text{ slower again}$

39. Bucket queues and Dial's algorithm

When priorities are small integers, the $\log n$ disappears entirely. A bucket queue is an array of FIFO queues indexed by priority: push is $O(1)$, pop is $O(1)$ amortised, and Part Three's ring is the bucket.

```

class BucketQueue {                                // priorities in [0, P)
    std::vector<std::deque<int>> b_; // b_[p] is a FIFO -> stable by
    std::size_t lo_ = 0, n_ = 0;      // construction (Topic 36 for free)
public:
    explicit BucketQueue(std::size_t P) : b_(P) {}
    void push(int v, std::size_t p) { b_[p].push_back(v); ++n_;
                                     lo_ = std::min(lo_, p); }
    int pop() {                          // O(1) amortised: lo_ only ever rises
        while (b_[lo_].empty()) ++lo_;
        int v = b_[lo_].front(); b_[lo_].pop_front(); --n_;
        return v;
    }
};
// Dial's algorithm = Dijkstra with a bucket queue, for integer edge weights
// bounded by C. Distances are in [0, (V-1)*C], so you need that many
// buckets -- or C+1 buckets used cyclically, since no live distance can
// differ from the current minimum by more than C.
// Dijkstra + binary heap: O(E log V)
// Dial's: O(E + V*C)
// V = 10^5, E = 5*10^5, C = 10: Dial 1.5*10^6 ops vs heap 8.5*10^6.

```

cyclic bucket array, C = 4 (weights 0..4), current min distance = 7

```

bucket index = dist mod (C+1)
[ 10 ][ 11 ][ 7 ][ 8 ][ 9 ]    dist values held
  ^0   ^1   ^2  ^3  ^4        index = dist % 5
      ^lo_ = 7 % 5 = 2

```

Any newly relaxed distance is in [7, 11] -- exactly C+1 distinct values, so the ring of buckets never has a collision between two live distances.

THE KEY INSIGHT

The heap's log n exists to *order* arbitrary keys. When keys are small integers you can index by them instead, and ordering becomes addressing — the same substitution that turns comparison sort into counting sort.

THE TRAP

lo_ scanning: if you allow priorities to *decrease* below **lo_**, the amortised O(1) argument collapses and the scan can be O(P) per pop. In Dial's this never happens (distances only rise), but in a general "task priority" queue it does. Symptom: a pop that is fast for hours and then takes milliseconds — P is typically 10⁵ or more.

bucket queue — push O(1) · pop O(1) amortised, O(P) total scanning · space O(P + n) · stable by construction · needs small integer keys

40. When a heap is the wrong answer

Four common situations where a heap is the reflex and something else is faster.

Requirement	Better structure	Numbers
Top k of n, k small	Heap of size k , not n	$O(n \log k)$ vs $O(n \log n)$; $k=10$, $n=10^7$: 3.3× fewer comparisons and the heap fits in L1
All elements, sorted	<code>std::sort</code>	Heapsort is 2–3× slower than introsort in practice: worse locality, no run detection
Small integer priorities	Bucket queue (39)	$O(1)$ vs $O(\log n)$; Dial's 5.7× fewer ops at $C=10$
$k \leq \sim 8$ items total	Sorted <code>array</code> , insertion	Linear insert into 8 elements beats sift-up: no branches, one cache line
Need FIFO among equals	Queue, or (pri,seq) (36)	Otherwise starvation with unbounded p99

```
// Top-k done right: a MIN-heap of size k, holding the k largest so far.
std::priority_queue<int, std::vector<int>, std::greater<int>> best;
for (int x : data) {
    if (best.size() < k) best.push(x);
    else if (x > best.top()) { best.pop(); best.push(x); } //  $O(\log k)$ 
}
// n =  $10^7$ , k = 10: comparisons  $\sim n * \log_2(10) = 3.3 * 10^7$ 
// versus building a full heap and popping k:  $10^7$  heapify +  $10 * 23$ 
// -- similar comparisons but 40 MB resident instead of 40 BYTES.
// The min-heap is also STREAMING: it never needs the whole input in memory.
```

THE KEY INSIGHT

For top-k, the heap holds the *opposite* extreme of what you want: a min-heap to find maxima. The root is the weakest member of the current best set, which is exactly the element a newcomer must beat — one comparison to reject almost everything.

THE TRAP

Using a max-heap of all n elements for top-k. It works, and it holds the entire input in memory: 40 MB for 10^7 ints instead of 40 bytes, and it cannot process a stream. Symptom: an out-of-memory in a log-processing job that worked fine on the sample file.

top-k – min-heap size k: $O(n \log k)$, $O(k)$ space, streaming · full heap: $O(n)$ space · sort: $O(n \log n)$ and 2–3× heapsort's speed

PRACTICE SET — Topics 32-40

A. Conceptual

1. Give the three guarantees a FIFO queue makes that a priority queue does not.
2. State the max-heap invariant precisely, and name two relationships it does *not* constrain.
3. Which half of the heap array requires no work during heapify, and why does that fact drive the $O(n)$ bound?
4. Why does sift-down perform two comparisons per level while sift-up performs one?
5. Explain why lazy deletion in Dijkstra does not change the asymptotic complexity, using the relationship between $\log E$ and $\log V$.
6. What makes a stale heap entry always safe to skip?
7. Why is the cache-line argument, not the comparison count, the reason 4-ary heaps win?
8. In a bucket queue, what property of the key sequence makes pop amortised $O(1)$, and what breaks it?
9. Why must the tie-breaker in a max-heap comparator read `a.seq > b.seq` to get earliest-first?
10. For top-k maxima, why is a min-heap of size k the right structure rather than a max-heap of size k ?
11. Give two independent reasons a Python `heapq` entry should include a sequence number.

B. Tracing and analysis

1. Starting from the empty array, push 5, 3, 9, 1, 8, 7 into a max-heap one at a time. Give the array after each push.
2. From the final array of B1, perform two pops. Show the array after each, including the last-leaf-to-root step.
3. Run `heapify` on `[3, 1, 6, 5, 2, 4]`. Show the array after each `sift_down` call and give the total number of swaps.
4. For $n = 15$, compute the exact bound $\sum \text{ceil}(n/2^{h+1}) \cdot h$ for $h = 0..3$, and compare with n .
5. A heap holds 1 M elements. Give the number of comparisons for one push and for one pop, in a binary heap and in a 4-ary heap.
6. Tasks arrive as (pri 5, seq 0), (pri 5, seq 1), (pri 9, seq 2), (pri 5, seq 3), (pri 9, seq 4). Give the pop order under the stable comparator.
7. Dijkstra on $V = 2 \times 10^5$, $E = 10^6$. Compare the comparison counts of a heap-based run ($\log_2 E$ per operation) against Dial's with $C = 8$.
8. Top-k with $n = 5 \times 10^6$ and $k = 20$: compute the worst-case comparisons for the size-k min-heap approach and the resident bytes for it versus the full-heap approach (4-byte ints).

C. Code tasks — build it, break it, observe, fix

1. Implement a max-heap with `push`, `pop`, `top`, `heapify`. Verify the invariant after every operation with an `assert` that walks the whole array.
2. Benchmark `heapify` against n individual pushes for $n = 10^7$ random ints. Report both times and the ratio.
3. **Break it:** write the heapify loop as `for (size_t i = n/2; i >= 0; --i)`. Run it. Record whether it hangs or faults, then run under `-fsanitize=address,undefined` and record the exact diagnostic. Fix with `i-- > 0`.

4. **Break it:** in the stable comparator, write `a.seq < b.seq` for the tie-break. Feed 10 000 tasks of which 90 % are priority 5, arriving continuously, and measure the age of the oldest task still queued after processing 5000. Record it, flip the comparison, and measure again.
5. **Break it:** remove the `if (d > dist[v]) continue;` line from a lazy-deletion Dijkstra. Run it on a dense random graph with 2000 vertices and count vertex expansions with and without the line.
6. Implement a 4-ary heap and benchmark it against your binary heap on 10 M push + 10 M pop. Report the ratio and check it against 1.4-1.5×.
7. Implement the cyclic bucket queue for Dial's algorithm with $C = 10$ and compare its runtime with your binary-heap Dijkstra on the same graph.
8. Implement top-k both ways (size-k min-heap, and full heap then k pops) for $n = 10^7$, $k = 10$. Report peak RSS for both using `/usr/bin/time -v`.

ANSWER KEY — Topics 32-40

A. Conceptual

1. **Arrival order is preserved; the wait of any element is bounded by n/μ ; no element can be overtaken.** A priority queue guarantees none of these — an element of low priority can be overtaken indefinitely.
2. $a[\text{parent}(i)] \geq a[i]$ for all $i > 0$. It does not constrain **siblings** to any order, and it does not constrain elements **across subtrees** — a node in the left subtree may be smaller or larger than any node in the right.
3. The **last $\lfloor n/2 \rfloor$ entries are leaves** and are already valid heaps of size 1. The bound follows because most nodes sit at small height and cost at most h swaps, and $\sum h/2^h$ converges to 2.
4. Sift-down must first determine **which child is larger** (one comparison) before comparing that child with the parent (a second). Sift-up has a single parent, so one comparison suffices.
5. Because $E \leq V^2$, so $\log E \leq 2 \log V$, a constant factor. The heap holds $O(E)$ entries instead of $O(V)$, and $O(E \log E) = O(E \log V)$.
6. Because a stale entry was pushed with a distance that has since been **strictly improved**, so its key is strictly worse than the recorded best; processing it could never produce a shorter path.
7. Because the d children are **contiguous in memory**: for $d = 4$ and 4-byte keys all four fit in one 64-byte line. Halving the height halves the number of distinct lines touched, while the comparison count on pop is unchanged.
8. `lo` must be **monotonically non-decreasing** — keys never smaller than the current minimum are pushed. It breaks when an arbitrary lower priority arrives, making a pop scan up to $O(P)$.
9. Because `std::priority_queue` pops the element that is **greatest under Compare**, and `Compare(a,b)` answers "does a come before b in the heap's weak ordering". To make the smaller `seq` the greatest element, the comparator must report `a.seq > b.seq`.
10. Because the root of a min-heap is the **weakest member of the current best set** — the one a newcomer must beat — so a single comparison rejects nearly every element, and the pop that follows removes exactly the right one.
11. (1) **Stability**: ties would otherwise be resolved arbitrarily. (2) **Type safety**: on a tie Python compares the next tuple element, and if that is a dict or a custom object it raises `TypeError: '<' not supported`.

B. Tracing and analysis

1. push 5: `[5]`. push 3: `[5,3]`. push 9: append `[5,3,9]`, sift 9 up past 5 $\rightarrow$ `[9,3,5]`. push 1: `[9,3,5,1]`. push 8: append `[9,3,5,1,8]`, $8 > 3 \rightarrow$ `[9,8,5,1,3]`. push 7: append `[9,8,5,1,3,7]`, $7 > 5 \rightarrow$ `[9,8,7,1,3,5]`.
2. Pop 1: top = 9; move last (5) to root $\rightarrow$ `[5,8,7,1,3]`; sift_down: children 8,7 $\rightarrow$ swap with 8 $\rightarrow$ `[8,5,7,1,3]`; children of index 1 are 1,3 $\rightarrow 5 \geq$ both, stop. `[8,5,7,1,3]`. Pop 2: top = 8; move last (3) to root $\rightarrow$ `[3,5,7,1]`; children 5,7 $\rightarrow$ swap with 7 $\rightarrow$ `[7,5,3,1]`.
3. `[3,1,6,5,2,4]`, $n = 6$, start at $i = 2$. $i=2$: children index 5 = 4; $6 \geq 4$, no swap. $i=1$: children 5 and 2; $5 > 1 \rightarrow$ swap $\rightarrow$ `[3,5,6,1,2,4]`. $i=0$: children 5 and 6; $6 > 3 \rightarrow$ swap $\rightarrow$ `[6,5,3,1,2,4]`; index 2's child is 4 $> 3 \rightarrow$ swap $\rightarrow$ `[6,5,4,1,2,3]`. Total swaps: **3**.
4. $h=0$: $\text{ceil}(15/2)=8 \times 0 = 0$. $h=1$: $\text{ceil}(15/4)=4 \times 1 = 4$. $h=2$: $\text{ceil}(15/8)=2 \times 2 = 4$. $h=3$: $\text{ceil}(15/16)=1 \times 3 = 3$. Total = **11** < **15** = n .
5. Binary: $\log_2 10^6 \approx 20 \rightarrow$ push **20**, pop **40**. 4-ary: $\log_4 10^6 = 10 \rightarrow$ push **10**, pop $4 \times 10 =$ **40**.

6. Priority 9 first, earliest seq first within each class: **(9,2), (9,4), (5,0), (5,1), (5,3)**.
7. Heap: roughly E pushes and E pops at $\log_2(10^6) = 20 \rightarrow \approx 4 \times 10^7$ **comparisons**. Dial's: $O(E + V \cdot C) = 10^6 + 2 \times 10^5 \times 8 = 2.6 \times 10^6$ **operations** — roughly **15× fewer**.
8. Comparisons $\leq n \cdot \log_2 20 \approx 5 \times 10^6 \times 4.32 = 2.16 \times 10^7$. Memory: min-heap of 20 ints = **80 bytes**; full heap = $5 \times 10^6 \times 4 = 20$ **MB**.

C. Code tasks — what to verify

1. The verification walk must check `a[(i-1)/2] ≥ a[i]` for every $i \geq 1$. Run it after 10^5 random pushes and pops; any violation means `sift_down` stopped one level early (a common off-by-one in the `big == i` test).
2. Expect roughly **1180 ms versus 295 ms, a 4.0× ratio**. A ratio near 1 means the compiler hoisted the push loop or you reused a warm heap — re-randomise the input for each run.
3. With `size_t`, `i` wraps to **18 446 744 073 709 551 615** and the loop indexes far out of bounds. UBSan/ASan report **heap-buffer-overflow ... READ of size 4** at an offset of many gigabytes, or the process hangs first. Both outcomes are acceptable observations; record which you got.
4. With the wrong direction, within priority 5 the queue is LIFO, so the oldest task age grows without bound — expect the oldest queued task to be **essentially the first one submitted (age ≈ the entire run)**. With the correct direction the oldest age should be bounded by roughly the priority-5 backlog divided by the service rate.
5. With the check, expansions $\approx V = 2000$. Without it, expect **several times V** on a dense random graph (typically 5–20×) and far worse on an adversarial one. The correctness of the distances is unchanged — that is the point.
6. Expect the 4-ary heap at roughly **1240 ms versus 1830 ms (1.48×)**. If you see no gain, check that children are computed as `4i+1 .. 4i+4` and that you compare all four before swapping.
7. Dial's should win by roughly the ratio of B7 on operations, though wall-clock gain is smaller (typically **2–4×**) because bucket scanning is memory-bound. Confirm both produce identical distance arrays.
8. Peak RSS: size- k min-heap should be dominated by the input reader — under **a few MB** if streaming. Full heap: **≈40 MB** plus the vector's growth slack (possibly 80 MB at the moment of the last doubling).

HANDNOTE SUMMARY CARD

PART FIVE - PRIORITY QUEUES AND HEAPS

topics 32-40

IT IS NOT A QUEUE

queue push $O(1)$ pop $O(1)$ arrival order guaranteed
PQ push $O(\log n)$ pop $O(\log n)$ top $O(1)$ NO order guarantee, ties
arbitrary, starvation possible
5 M push/pop of int: queue 21 ms | heap 340 ms

ARRAY LAYOUT (0-based)

parent(i) = $(i-1)/2$ left = $2i+1$ right = $2i+2$
d-ary: parent = $(i-1)/d$ child k = $d*i + k + 1$
MAX-HEAP INVARIANT: $a[(i-1)/2] \geq a[i]$ for all $i > 0$
says NOTHING about siblings or across subtrees. 2nd largest = $\max(a[1], a[2])$.
leaves = indices $n/2 \dots n-1$ = half the array

THE TWO PROCEDURES

sift_up(i) while $i > 0$ and $a[(i-1)/2] < a[i]$: swap, $i = \text{parent}$ 1 cmp/level
sift_down(i) pick larger child, swap if bigger than $a[i]$, descend 2 cmp/level
push = append + sift_up(n-1)
pop = save $a[0]$; $a[0] = a.\text{back}()$; pop_back(); sift_down(0)

HEAPIFY IS $O(n)$ - and the loop direction is the algorithm

for ($i = n/2$; $i-- > 0$;) sift_down(i) <- $i-- > 0$, NEVER $i \geq 0$ unsigned
sum over heights: $(n/2) * \sum h/2^h = (n/2)*2 = n$ -> < 1 swap per element
 10^7 ints: n pushes 1180 ms | make_heap 295 ms | 4.0x

STABILITY - ties are NOT FIFO

key = (priority, seq) with a global monotone seq counter +8 B, +0 time
C++ max-heap comparator: if (pri differs) return $a.\text{pri} < b.\text{pri}$;
return $a.\text{seq} > b.\text{seq}$; // ">" for EARLIEST FIRST
Python: heappush(h, (-pri, seq, payload)) -- seq also prevents TypeError

DECREASE-KEY vs LAZY DELETION

true decrease-key: needs vertex->index map, +32-48 B/vertex, 2-3x slower
lazy: push a duplicate; on pop if ($d > \text{dist}[v]$) continue; <- IS the delete
heap grows to $O(E)$; $\log E \leq 2 \log V$, so same asymptotic class
omit the check -> still correct, exponential re-expansion on bad graphs

d-ARY

$d=2$ height 20 push 20 pop 40 | $d=4$ height 10 push 10 pop 40 | $d=8$ push 7 pop 54
4 children of 4-byte keys = ONE 64 B cache line -> half the lines touched
measured 10M+10M: binary 1830 ms, 4-ary 1240 ms (1.48x), 8-ary 1390 ms

BUCKET QUEUE / DIAL'S small integer priorities only

array of FIFO queues indexed by priority; push $O(1)$, pop $O(1)$ amortised
stable for free (each bucket is a FIFO)
Dial's = Dijkstra with C+1 cyclic buckets: $O(E + V*C)$ vs $O(E \log V)$
breaks if lo_can DECREASE -> pop scan becomes $O(P)$

WHEN A HEAP IS WRONG

top-k, k small -> MIN-heap of size k: $O(n \log k)$, $O(k)$ space, streaming
(root = weakest of the best set; one cmp rejects most)
everything sorted-> std::sort; heapsort is 2-3x slower (locality)
small int keys -> bucket queue
 $n \leq \sim 8$ -> sorted array, linear insert
fairness needed -> queue, or (pri, seq)

PART SIX

Queues built from other structures

Composition problems: the interview's favourite genre.

Part Six · Queues built from other structures

41. A queue from two stacks

The canonical composition problem. Two stacks, `in` and `out`: push onto `in`; to pop, if `out` is empty, tip everything from `in` into `out` — which reverses it — then pop from `out`. Each element is pushed twice and popped twice, so the amortised cost is $O(1)$.

```
template <typename T>
class QueueFromStacks {
    std::vector<T> in_, out_;
    void shuttle() { // ONLY when out_ is empty
        while (!in_.empty()) { out_.push_back(std::move(in_.back()));
                               in_.pop_back(); }
    }
public:
    void enqueue(T x) { in_.push_back(std::move(x)); } // 0(1) always
    std::optional<T> dequeue() {
        if (out_.empty()) shuttle(); // 0(k), but only once per element
        if (out_.empty()) return std::nullopt;
        T v = std::move(out_.back()); out_.pop_back();
        return v;
    }
    std::size_t size() const { return in_.size() + out_.size(); }
};

// AMORTISED PROOF (banker's method): charge 3 units per enqueue.
// 1 pays the push onto in_. 2 are banked on that element.
// The shuttle spends exactly 2 units on it (one pop from in_, one push
// to out_). The final pop from out_ is paid by the dequeue's own unit.
// The bank is never negative -> 3 units/element -> 0(1) amortised.
```

enqueue 1,2,3	in_: [1,2,3]	out_: []	(top on the right)
dequeue	shuttle: pop 3,2,1 from in_, push to out_		
	in_: []	out_: [3,2,1]	top of out_ is 1 -- the OLDEST. reversed.
	returns 1	out_: [3,2]	
enqueue 4	in_: [4]	out_: [3,2]	NO shuttle -- correct,
dequeue	returns 2	out_: [3]	because out_ holds all
dequeue	returns 3	out_: []	older elements already
dequeue	shuttle -> out_: [4],	returns 4	

THE KEY INSIGHT

The condition "shuttle only when `out_` is empty" is the entire correctness argument. `out_` always holds a prefix of the queue that is strictly older than everything in `in_`, so pouring new elements on top of a non-empty `out_` would interleave two generations and destroy FIFO.

THE TRAP

Shuttling on every dequeue "to keep things tidy". Correctness survives — you must shuttle back and forth — but the cost becomes $O(n)$ per operation and the whole point is lost. Symptom: an implementation that passes every test and takes 40 seconds on 10^5 operations. If your interviewer asks for the worst case, it is $O(n)$ for a single dequeue; the *amortised* cost is $O(1)$.

two stacks – enqueue $O(1)$ · dequeue $O(1)$ amortised, $O(n)$ worst case · each element moved exactly twice · space $2n$ peak during shuttle

42. A stack from two queues

The mirror problem, and it is strictly worse: one of the two operations must be $O(n)$, and you get to choose which. There is no amortisation to rescue it, because the work is repeated for every element on every operation.

```
// VARIANT A -- costly push,  $O(1)$  pop.
void push(T x) {
    q2_.push(x); // new element alone in q2_
    while (!q1_.empty()) { q2_.push(q1_.front()); q1_.pop(); } //  $O(n)$ 
    std::swap(q1_, q2_); // q1_ now has newest at the front
}
T pop() { T v = q1_.front(); q1_.pop(); return v; } //  $O(1)$ 

// VARIANT B --  $O(1)$  push, costly pop.
void push(T x) { q1_.push(x); } //  $O(1)$ 
T pop() {
    for (std::size_t i = q1_.size(); i > 1; --i) { //  $O(n)$ 
        q1_.push(q1_.front()); q1_.pop(); // rotate n-1 elements to the back
    }
    T v = q1_.front(); q1_.pop(); return v;
}
// n pushes + n pops: Variant A =  $n^2/2 + n$  moves. Variant B =  $n + n^2/2$ .
// Identical totals. Pick by which operation your workload does more of.
```

Variant B, pop() from $q1_ = [1,2,3,4]$ (front on the left)

rotate 3 times: $[2,3,4,1] \rightarrow [3,4,1,2] \rightarrow [4,1,2,3]$
front is now 4 = the newest = LIFO. pop it. $q1_ = [1,2,3]$
Cost: 3 moves for one pop. For n elements: $(n-1)$ moves EVERY pop,
with no banking possible -- the work is redone from scratch each time.

THE KEY INSIGHT

Two stacks make a good queue; two queues make a bad stack. The asymmetry is real: a stack's *reversal* is exactly what a queue needs, so the work composes and can be banked. A queue's rotation is not what a stack needs, so the work is repeated and cannot.

THE TRAP

Claiming $O(1)$ amortised for variant B in an interview. It is $O(n)$ per pop, always — amortisation requires that the expensive work make future operations cheap, and here the next pop rotates the whole structure again. Saying "amortised" where the accounting does not hold is the single most common way to lose this question.

two queues – one op $O(n)$, unavoidably · n push + n pop = $n^2/2$ moves either way · no amortisation exists · use a vector instead

43. A queue with O(1) minimum

Support `enqueue`, `dequeue`, and `min()`, all in O(1) amortised. Build it from Topic 41's two stacks, where each stack tracks its own running minimum — the min-stack is easy, and the queue inherits it.

```
template <typename T>
class MinStack {
    std::vector<std::pair<T,T>> s_; // each entry carries the min BELOW
    // {value, min of stack incl. this}
public:
    void push(T x) { s_.emplace_back(x, s_.empty() ? x
                                     : std::min(x, s_.back().second)); }

    void pop() { s_.pop_back(); }
    T min() const { return s_.back().second; }
    bool empty() const { return s_.empty(); }
    T top() const { return s_.back().first; }
};

template <typename T>
class MinQueue {
    MinStack<T> in_, out_;
public:
    void enqueue(T x) { in_.push(x); }
    T dequeue() {
        if (out_.empty()) while (!in_.empty()) { out_.push(in_.top());
                                                    in_.pop(); }

        T v = out_.top(); out_.pop(); return v;
    }
    T min() const {
        // O(1): at most two candidates
        if (in_.empty()) return out_.min();
        if (out_.empty()) return in_.min();
        return std::min(in_.min(), out_.min());
    }
};
// Space: 2 * sizeof(T) per element instead of 1. For T = int, 8 B vs 4 B.
```

MinStack after push 5, 3, 8, 1:

value:	5	3	8	1	
min:	5	3	3	1	<- min never increases going up
	^bottom		^top		-> min() = 1, O(1)

pop the 1: top is now (8, min 3) -> min() = 3. No recomputation: the answer was stored when 8 was pushed and is still correct.

THE KEY INSIGHT

The min-stack works because a stack's removals are the exact reverse of its insertions, so the min at every historical depth can be recorded once and is still valid when you return to that depth. A queue removes from the other end, which is precisely why it needs the two-stack decomposition to get the same property.

THE TRAP

Storing only *one* global minimum and recomputing on dequeue. That is O(n) whenever the minimum leaves — and an adversarial input (a strictly increasing sequence, dequeued in order) makes every dequeue the expensive one. Symptom: an O(1) claim that degrades to O(n²) on sorted input, which is a common test case.

44. A fixed-capacity queue from a list of blocks

Part Two's blocked layout, written by hand: a linked list of fixed arrays with a free list, giving unbounded logical capacity with bounded allocator traffic and stable element addresses. This is what you build when the standard library is not available — embedded, kernel, or an arena-allocated system.

```
template <typename T, std::size_t B = 64>
class BlockQueue {
    struct Block { T data[B]; Block* next; };
    Block* head_ = nullptr;   Block* tail_ = nullptr;
    Block* free_ = nullptr;    // recycled blocks, never returned
    std::size_t hoff_ = 0, toff_ = B; // toff_ == B forces a block on push
    Block* take() {           // pop from the free list, or malloc
        if (free_) { Block* b = free_; free_ = b->next; return b; }
        return new Block;
    }
public:
    void enqueue(const T& x) {
        if (toff_ == B) { Block* b = take(); b->next = nullptr;
            (tail_ ? tail_->next : head_) = b;
            tail_ = b; toff_ = 0; }
        tail_->data[toff_++] = x;
    }
    // dequeue: read head_->data[hoff_++]; when hoff_ == B, move the block
    // to free_ and advance head_. One malloc per B elements, then zero.
};
// Allocator traffic for 10 M pushes with B = 64: 156 250 mallocs on the
// first pass, then ZERO -- the free list recycles. A ListQueue does 10 M.
```

```
head_           tail_
|               |
v               v
[ ...xxxx | next ] --> [ xxxxxx.... | next=null ]
    ^hoff_=4           ^toff_=7
free_ --> [ block ] --> [ block ] --> null    recycled, never freed

Steady state at constant depth: the same 2-3 blocks circulate forever.
Peak block count = ceil(max_depth / B) + 1, and it never shrinks.
```

THE KEY INSIGHT

The free list converts a per-element allocation into a per-block one, and then into none at all. The queue's memory footprint becomes a function of its *historic peak depth* rather than its current depth — which is usually the right trade, but is worth knowing before you claim the memory is bounded.

THE TRAP

A free list that never shrinks after a burst. A queue that peaked at 1 M elements holds 15 625 blocks forever, whether or not it will ever be that deep again. Symptom: RSS that ratchets upward across the day and never returns, with heap profilers showing the memory as live. Add a trim step, or cap the free list length.

block queue – enqueue/dequeue $O(1)$ · 1 malloc per B elements, then $O(1)$ · addresses stable · footprint tracks peak depth, not current

45. LRU cache: why a plain queue fails

An LRU cache needs "evict the least recently used" — which sounds like a queue — and "move this key to the most recent position" — which a queue cannot do at all, because the key to move is in the middle.

```
class LRUCache {
    struct Entry { int key, value; };
    std::list<Entry> order_;                // front = most recent
    std::unordered_map<int, std::list<Entry>::iterator> map_;
    std::size_t cap_;
public:
    int get(int key) {
        auto it = map_.find(key);
        if (it == map_.end()) return -1;
        order_.splice(order_.begin(), order_, it->second); //  $O(1)$  MOVE
        return it->second->value;                          // iterator stays
    }                                                       // valid: T30
    void put(int key, int value) {
        auto it = map_.find(key);
        if (it != map_.end()) { it->second->value = value;
                                order_.splice(order_.begin(), order_,
                                                it->second); return; }
        if (map_.size() == cap_) { map_.erase(order_.back().key);
                                    order_.pop_back(); }          // evict LRU
        order_.push_front({key, value});
        map_[key] = order_.begin();
    }
};
// std::list, not deque: splice is  $O(1)$  and does not invalidate iterators.
// With a deque, "move to front" is  $O(n)$  and every stored iterator dies.
```

```
get(B) on order_ = A <-> B <-> C <-> D    (A most recent, D is the LRU)

map_:  A->it0 B->it1 C->it2 D->it3    iterators, not indices

splice(begin, order_, it1):  B <-> A <-> C <-> D
    only 4 pointer writes; it1 STILL POINTS AT B, so map_ needs no update.
That last property is why std::list is the container and why the map
stores iterators rather than positions.
```

THE KEY INSIGHT

The hash map supplies $O(1)$ *location* and the linked list supplies $O(1)$ *relocation*. Neither structure can do both; the composition is the answer, and recognising "I need to touch the middle in $O(1)$ " is the signal to reach for it (see Topic 23).

THE TRAP

Implementing LRU as a queue plus lazy deletion — re-enqueueing a key on every access and skipping stale entries on eviction. The queue then grows with *access count*, not with cache size: a cache of 1000 entries under 10^7 accesses holds 10^7 queue entries. Symptom: an OOM in a cache whose stated capacity is tiny.

LRU – get $O(1)$ · put $O(1)$ · space $\text{cap} \times (\text{entry} + \text{node } 40 \text{ B} + \text{map slot } \sim 50 \text{ B})$ · queue-based version: memory grows with access count

46. The circular tour problem

n petrol pumps around a circle; pump i has `petrol[i]` and the distance to the next costs `dist[i]`. Find a start from which you can complete the loop. It is posed as a queue problem, and the queue is the point: the candidate window is a contiguous arc that only ever moves forward.

```
int start_tour(const std::vector<int>& petrol, const std::vector<int>& dist) {
    int start = 0, tank = 0, deficit = 0;
    for (std::size_t i = 0; i < petrol.size(); ++i) {
        int gain = petrol[i] - dist[i];
        tank += gain;
        if (tank < 0) {
            // the whole window [start..i] fails
            deficit += tank; // remember how short it was
            start = static_cast<int>(i) + 1; // dequeue ALL of it at once
            tank = 0;
        }
    }
    return (tank + deficit >= 0) ? start : -1;
}
//  $O(n)$  time,  $O(1)$  space -- the "queue" is the implicit window [start..i].
// The naive version tries every start:  $O(n^2)$ . At  $n = 10^5$  that is
//  $5 \times 10^9$  operations ( $\sim 4$  s) versus  $10^5$  ( $\sim 0.1$  ms).
```

```
petrol  4  6  7  4      dist  6  5  3  5      gains: -2 +1 +4 -1

i=0  tank=-2 < 0  -> deficit=-2, start=1, tank=0  (drop pump 0 entirely)
i=1  tank=+1
i=2  tank=+5
i=3  tank=+4
total = 4 + (-2) = 2 >= 0  -> answer: start at pump 1
```

Why dropping the WHOLE window is safe: if the tank went negative at i starting from `start`, then it also goes negative at i from ANY start in $(\text{start}, i]$, because every prefix from start was non-negative.

THE KEY INSIGHT

The queue never needs to be materialised. When failure at position i invalidates every candidate in the current window at once, "dequeue everything and restart at $i+1$ " replaces the structure with two integers — the same collapse that turns many sliding-window problems into $O(1)$ space.

THE TRAP

Returning `start` without the final `tank + deficit  $\geq$  0` check. The loop always produces *some* `start`, so an impossible instance returns a confident wrong answer. Symptom: passes every solvable test case and fails the one where no tour exists — frequently the only hidden test.

circular tour – $O(n)$ time, $O(1)$ space · naive $O(n^2)$: 5×10^9 ops at $n=10^5$ · one pass, each pump examined once

PRACTICE SET — Topics 41-46

A. Conceptual

1. State the condition under which `QueueFromStacks` may shuttle, and prove in one sentence why shuttling at any other time breaks FIFO.
2. Give the banker's-method accounting for the two-stack queue: how many units per enqueue, and what each pays for.
3. What is the worst-case cost of a single `dequeue` in the two-stack queue, and why is that compatible with the $O(1)$ amortised claim?
4. Explain why two queues cannot make an amortised- $O(1)$ stack, in terms of whether expensive work makes future work cheaper.
5. In `MinStack`, why is the stored minimum still correct after several pops without any recomputation?
6. Why does the min-queue need exactly two candidates for `min()`, not one and not n ?
7. What does the free list in `BlockQueue` make the memory footprint a function of?
8. Name the two operations LRU requires and the structure that provides each in $O(1)$.
9. Why does `std::list::splice` keep the map's stored iterators valid, and why would a deque not?
10. In the circular tour, justify skipping all starts in $(start, i]$ after a failure at i .
11. Why must the circular tour's final feasibility check exist at all, given the loop already produced a start index?

B. Tracing and analysis

1. Trace `QueueFromStacks` through: enqueue 1,2,3; dequeue; enqueue 4; dequeue; dequeue; enqueue 5; dequeue; dequeue. Give `in_` and `out_` contents and the returned value after every operation.
2. For 1000 enqueues followed by 1000 dequeues, count the total element moves in `QueueFromStacks`, and the number of shuttle events.
3. For the same workload alternating one enqueue and one dequeue, count moves and shuttle events.
4. Two-queues stack, variant B: push 1..5 then pop 5 times. Count the total element rotations.
5. `MinStack` receives push 7, 4, 9, 4, 2 then two pops. Give the (value, min) pairs at each step and `min()` after each pop.
6. A `MinQueue` holds 1 M ints. Compute its memory versus a plain ring queue of ints, in MB.
7. `BlockQueue<int, 64>` peaks at 500 000 elements then settles at depth 200. Give blocks allocated, blocks on the free list at steady state, and bytes retained.
8. An LRU cache of capacity 1000 receives 10^7 accesses. Compare the memory of the map+list design against the queue-plus-lazy-deletion design, at 16 bytes per queue entry.
9. Solve the circular tour by hand for petrol = [6,3,7] and dist = [4,8,4]. Give the tank after each step, the chosen start, and whether the tour is feasible.

C. Code tasks — build it, break it, observe, fix

1. Implement `QueueFromStacks`. Verify against `std::queue` with 10^6 randomised operations, comparing every returned value.

2. **Break it:** remove the `if (out_.empty())` guard so `shuttle()` runs on every dequeue. Verify the output is still correct, then time 10^5 operations before and after. Record both times and the ratio.
3. **Break it:** shuttle into a *non-empty* `out_` (call `shuttle` unconditionally *before* checking emptiness, without clearing). Run `enqueue 1,2; dequeue; enqueue 3,4; dequeue; dequeue; dequeue` and record the exact wrong output order. Explain which two generations interleaved.
4. Implement `MinQueue` and check `min()` against a brute force scan after each of 10^5 random operations.
5. **Break it:** implement the min-queue with a single stored minimum recomputed by scanning on dequeue. Feed it a strictly increasing sequence of 10^5 elements, dequeued in order, and time it. Compare with the two-stack version and report the ratio.
6. Implement `BlockQueue` with the free list. Instrument `new Block` with a counter; run 10^7 push/pop pairs at depth ≤ 200 and report the total block allocations. Confirm it is `ceil(200/64)+1`-ish and not 156 250.
7. **Break it:** build an LRU as a queue with lazy deletion, capacity 1000. Run 5×10^6 accesses over a working set of 2000 keys and record peak RSS with `/usr/bin/time -v`. Then run the map+list version and record it.
8. Implement `start_tour` and a brute-force $O(n^2)$ checker. Test on 1000 random instances of $n=200$ including infeasible ones, asserting agreement.

ANSWER KEY — Topics 41-46

A. Conceptual

1. Only when `out_` is **empty**. `out_` holds a prefix of the queue strictly older than everything in `in_`; pouring `in_` on top would place newer elements above older ones, so they would be popped first.
2. **3 units per enqueue**: 1 pays the push onto `in_`; 2 are banked and later pay for the pop from `in_` and the push onto `out_` during the shuttle. The final pop from `out_` is paid by the dequeue's own unit.
3. **O(n)** — the shuttle moves every element. It is compatible because those n moves are **paid for in advance** by the n enqueues that banked 2 units each; no sequence of m operations can exceed $3m$ total work.
4. Because the expensive rotation **does not leave the structure in a cheaper state**: after popping, the next pop must rotate the remaining $n-1$ elements again from scratch. Amortisation requires the invested work to be reusable, and here it is discarded.
5. Because a stack's pops are the **exact reverse of its pushes**, so returning to depth d returns to precisely the state that existed when the value at depth d was pushed — and the minimum recorded then is still the minimum of that prefix.
6. Because the queue's elements are partitioned into exactly two stacks, and each stack answers its own minimum in $O(1)$. The queue minimum is the smaller of the two — **no element lies outside those two sets**.
7. **Historic peak depth**, not current depth: blocks move to the free list rather than being returned to the allocator.
8. **Locate a key in O(1)** → hash map. **Move an arbitrary node to the front and evict from the back in O(1)** → doubly linked list.
9. Because **splice relinks nodes without moving them**; the node's address is unchanged, so the iterator still refers to it. A deque stores elements by position within blocks, so a "move to front" physically relocates elements and shifts everything between, invalidating iterators (Topic 30) and costing $O(n)$.
10. Because every prefix starting at `start` up to `i-1` was non-negative. Any later start j in that range has $\text{sum}(j..i) = \text{sum}(start..i) - \text{sum}(start..j-1) \leq \text{sum}(start..i) < 0$, **so j fails at i too**.
11. Because the loop's `start` is only "the last place we restarted" and is produced unconditionally. Feasibility depends on the **total** gain over the whole circle being non-negative, which the loop never tests.

B. Tracing and analysis

1. `enq1: in=[1] out=[]`. `enq2: in=[1,2]`. `enq3: in=[1,2,3]`. `deq: shuttle → in=[], out=[3,2,1]; returns 1`, `out=[3,2]`. `enq4: in=[4]`, `out=[3,2]`. `deq: returns 2`, `out=[3]`. `deq: returns 3`, `out=[]`. `enq5: in=[4,5]`. `deq: shuttle → out=[5,4]; returns 4`, `out=[5]`. `deq: returns 5`, `out=[]`.
2. Each element is pushed to `in_`, popped from `in_`, pushed to `out_`, popped from `out_` — **4000 moves** total, with **exactly 1 shuttle event** (at the first dequeue).
3. Alternating: each dequeue finds `out_` empty and shuttles a single element. Still **4000 moves** (4 per element), but **1000 shuttle events**. The per-element cost is unchanged — that is the amortisation working.
4. Variant B: pops with 5, 4, 3, 2, 1 elements present rotate 4, 3, 2, 1, 0 → **10 rotations** ($n(n-1)/2$).

5. push 7 → (7,7); push 4 → (4,4); push 9 → (9,4); push 4 → (4,4); push 2 → (2,2). min = 2. Pop → top (4,4), **min = 4**. Pop → top (9,4), **min = 4**.
6. MinQueue: 2 ints per element = 8 B × 10⁶ = **8 MB** (plus vector slack). Plain ring: **4 MB**. Ratio **2×**.
7. Peak blocks = ceil(500 000 / 64) = **7813**. Steady state uses ceil(200/64)+1 = **5 blocks**, so **7808 sit on the free list**. Bytes retained = 7813 × (64×4 + 8) = 7813 × 264 = **2 062 632 B ≈ 2.0 MB**, held forever.
8. map+list: 1000 × (~16 B entry + ~24 B list node + ~50 B map slot) ≈ **90 KB**. Queue + lazy deletion: 10⁷ × 16 B = **160 MB** — roughly **1800×** more.
9. gains = 6−4 = +2, 3−8 = −5, 7−4 = +3. i=0: tank=2. i=1: tank=−3 < 0 → deficit=−3, start=2, tank=0. i=2: tank=3. Total = 3 + (−3) = 0 ≥ 0 → **feasible, start at pump 2**.

C. Code tasks — what to verify

1. Every returned value must match `std::queue` exactly, including `nullopt` on empty. Randomise with ~60 % enqueue so the queue both grows and empties repeatedly.
2. Correctness is unchanged — note that explicitly. Timing: expect roughly **10-20 ms** guarded versus **several seconds** unguarded at 10⁵ operations (an O(n) shuttle per dequeue), a ratio in the hundreds.
3. Expected wrong output: after enqueue 1,2; dequeue returns 1 with out=[2]; enqueue 3,4 then an unconditional shuttle pushes 4 then 3 *on top of 2*, giving out=[2,4,3] with 3 at the top → dequeues return **3, 4, 2**. The interleaved generations are **{2} (older) and {3,4} (newer)**.
4. Brute force must scan both stacks. Any mismatch is almost always `min()` forgetting to handle one stack being empty — test a sequence that drains `out_` completely.
5. On a strictly increasing sequence the minimum is always the element being removed, so the naive version rescans every time: expect **≈5×10⁹ comparisons, several seconds**, versus a **few milliseconds** for the two-stack version — a ratio in the thousands.
6. Expect the allocation counter to stop at roughly **5-6 blocks** after the first few hundred operations and never increase, versus **156 250** without the free list.
7. Queue + lazy deletion should show peak RSS in the **hundreds of MB** and rising with access count; map+list should stay **under a few MB** and flat. Plot RSS against access count for both — the shapes are the lesson.
8. All 1000 instances must agree, including that both report infeasible on the same inputs. Generate infeasible cases deliberately by making total dist exceed total petrol.

HANDNOTE SUMMARY CARD

PART SIX - QUEUES BUILT FROM OTHER STRUCTURES

topics 41-46

QUEUE FROM TWO STACKS

enqueue $O(1)$, dequeue $O(1)$ AMORTISED

```
enqueue: in_.push(x)
dequeue: if (out_.empty()) while(!in_.empty()) { out_.push(in_.top());
                                                in_.pop(); }
        return out_.pop()
```

*** SHUTTLE ONLY WHEN out_ IS EMPTY *** -- out_ holds a strictly OLDER prefix
banker's: 3 units/enqueue = 1 push + 2 banked for the shuttle's pop+push
worst case for ONE dequeue is $O(n)$. Say "amortised $O(1)$, worst case $O(n)$ ".

STACK FROM TWO QUEUES

one op is $O(n)$. NO amortisation exists.

A: costly push - push to q2, drain q1 into q2, swap pop $O(1)$
B: costly pop - rotate n-1 elements to the back, pop push $O(1)$
n push + n pop = $n^2/2$ moves either way. Two stacks -> good queue;
two queues -> bad stack, because rotation is not reusable work.

MIN-QUEUE ($O(1)$ min)

MinStack entry = {value, min(value, min below)} <- correct after pops
with no recomputation
MinQueue = two MinStacks; min() = min(in_.min(), out_.min()) -- 2 candidates
space $2 \times \text{sizeof}(T)$ per element
naive single-minimum + rescan -> $O(n^2)$ on SORTED input (a standard test)

BLOCK QUEUE (hand-rolled std::deque)

linked list of T data[B] blocks + free list; 1 malloc per B, then ZERO
addresses stable; footprint tracks HISTORIC PEAK depth, never shrinks
10 M pushes, B=64: 156 250 mallocs first pass, 0 after; ListQueue does 10 M

LRU CACHE - queue alone CANNOT do this

need: locate key $O(1)$ -> unordered_map
move middle node to front -> std::list::splice (relinks, no move,
iterators STAY VALID)
evict from the back -> list.pop_back()
get/put $O(1)$; memory = cap x (entry + ~24 B node + ~50 B map slot)
queue + lazy deletion instead: memory grows with ACCESS COUNT
cap 1000, 10^7 accesses -> 160 MB vs 90 KB

CIRCULAR TOUR (petrol pumps)

$O(n)$ time, $O(1)$ space

```
gain = petrol[i] - dist[i]; tank += gain
if (tank < 0) { deficit += tank; start = i+1; tank = 0; } // drop the WHOLE
return (tank + deficit >= 0) ? start : -1; // window at once
safe because every prefix from start was >= 0, so any j in (start,i] also
fails at i. MISSING THE FINAL CHECK -> confident wrong answer when
no tour exists (usually the only hidden test)
```

RECOGNITION

"implement X using only Y" -> composition; count moves per element
" $O(1)$ min/max alongside FIFO" -> two stacks, each carrying its own min
"touch the middle in $O(1)$ " -> map + linked list, never a queue
"one pass, window only moves right" -> the queue collapses to two integers

PART SEVEN

Patterns and algorithms

Where the queue stops being storage and starts being a technique.

Part Seven · Patterns and algorithms

47. BFS on a grid

The single most-used queue algorithm. A grid is a graph whose adjacency is implied by coordinates, so no adjacency list is needed: the queue holds cells and the neighbours are computed by four offsets.

```
int shortest_path(const std::vector<std::string>& g, int sr, int sc,
                 int tr, int tc) {
    int R = g.size(), C = g[0].size();
    std::vector<std::vector<int>> dist(R, std::vector<int>(C, -1));
    std::queue<std::pair<int,int>> q;
    q.push({sr, sc}); dist[sr][sc] = 0; // mark AT ENQUEUE (T48)
    const int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        if (r == tr && c == tc) return dist[r][c];
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            if (g[nr][nc] == '#' || dist[nr][nc] != -1) continue;
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
    return -1;
}
// 1000 x 1000 grid: 10^6 cells, <= 4*10^6 edge checks, queue peaks at the
// largest "frontier" -- for an open grid roughly the perimeter of a
// diamond, ~4*1000 = 4000 cells, NOT 10^6. Reserve accordingly.
```

dist after BFS from S on an open 5x5 grid (# = wall):

4 3 2 3 4	the frontier at each step is a diamond ring;
3 2 1 2 3	ring at distance d ~ 4d, so the queue holds
2 1 S 1 2	0(sqrt(cells)) entries, not 0(cells)
3 2 1 2 3	
4 3 2 3 4	with walls, the frontier can be a single cell
	(a corridor) or the whole open area (a room)

THE KEY INSIGHT

The `dist` array does three jobs at once: the visited set, the answer, and the queue's deduplication. That is why no separate `visited` structure is needed and why `dist[nr][nc] != -1` is the membership test Topic 6 warned you not to do by scanning.

THE TRAP

Storing `std::pair<int,int>` when you could store `r * C + c` as a single int. On a 10^6 -cell grid the pair version doubles the queue's memory and halves its cache density. Symptom: an otherwise-correct BFS at $3\times$ the expected runtime on large grids — and memory-limit failures on 4000×4000 inputs.

grid BFS — $O(R\cdot C)$ time, 4 edge checks/cell · space $O(R\cdot C)$ for dist · queue peak $O(\sqrt{\text{cells}})$ on open grids

48. BFS on a graph, and marking visited at the right moment

The same algorithm on an explicit adjacency list, plus the one decision that separates a working BFS from an exponential one: mark a vertex visited when you *enqueue* it, never when you dequeue it.

```
std::vector<int> bfs(const std::vector<std::vector<int>>& adj, int src) {
    std::vector<int> dist(adj.size(), -1);
    std::queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u])
            if (dist[v] == -1) {          // not seen
                dist[v] = dist[u] + 1;    // <-- MARK HERE, at enqueue time
                q.push(v);
            }
    }
    return dist;
}

// Marking at DEQUEUE time instead:
//     int u = q.front(); q.pop();
//     if (seen[u]) continue; seen[u] = true;
//     for (int v : adj[u]) if (!seen[v]) q.push(v); // pushes duplicates
// Still CORRECT. But a vertex of degree d can be enqueued up to d times,
// so the queue holds  $O(E)$  entries instead of  $O(V)$ .
//  $V = 10^5$ ,  $E = 10^6$ : queue peaks at  $\sim 10^6 \cdot 4 \text{ B} = 4 \text{ MB}$  instead of 400 KB
// On a complete graph  $K_{1000}$ :  $10^6$  queue entries for 1000 vertices.
```

```
star graph: centre 0 connected to 1..999, and 1..999 all connected to 1

mark at ENQUEUE: 0 pushes 1..999 once each      queue peak 999
mark at DEQUEUE: 0 pushes 1..999; then vertex 1 pushes 2..999 AGAIN
                  before any of them are dequeued queue peak ~2000
on denser graphs the multiplier is the average degree, i.e.  $E/V$ .
```

THE KEY INSIGHT

Enqueue-time marking makes the queue a set: every vertex appears at most once, ever. The invariant to state in an interview is "a vertex is pushed exactly once, so the queue's total lifetime traffic is V , not E ".

THE TRAP

Dequeue-time marking is what most people write when converting a DFS — and Dijkstra genuinely needs it, because a vertex can legitimately be re-relaxed (Topic 37). Copying the Dijkstra shape into unweighted BFS is the source of the bug. Symptom: correct answers with 10–100× memory and a queue depth that tracks E rather than V .

graph BFS — $O(V+E)$ · enqueue-marking: queue traffic V · dequeue-marking: queue traffic E · dist array doubles as the visited set

49. Multi-source BFS

Seed the queue with *every* source at distance 0 and run the identical loop. The result is, for each cell, the distance to the *nearest* source — computed in one pass rather than one BFS per source.

```
// "Rotting oranges": every rotten orange spreads to adjacent fresh ones per
// minute. How many minutes until none are fresh?
int rot_time(std::vector<std::vector<int>>& g) { // 0 empty, 1 fresh, 2 rotten
    int R = g.size(), C = g[0].size(), fresh = 0, t = 0;
    std::queue<std::pair<int,int>> q;
    for (int r = 0; r < R; ++r) for (int c = 0; c < C; ++c) {
        if (g[r][c] == 2) q.push({r, c}); // ALL sources, distance 0
        else if (g[r][c] == 1) ++fresh;
    }
    const int dr[] = {-1,1,0,0}, dc[] = {0,0,-1,1};
    while (!q.empty() && fresh > 0) {
        int sz = q.size(); // one whole level (T50)
        ++t;
        while (sz--) {
            auto [r, c] = q.front(); q.pop();
            for (int k = 0; k < 4; ++k) {
                int nr = r+dr[k], nc = c+dc[k];
                if (nr<0||nr>=R||nc<0||nc>=C||g[nr][nc]!=1) continue;
                g[nr][nc] = 2; --fresh; q.push({nr, nc});
            }
        }
    }
    return fresh == 0 ? t : -1;
}
// One BFS: O(R*C). k separate BFS runs: O(k * R * C) -- at k = 1000 sources
// on a 1000x1000 grid that is 10^9 vs 10^6 cell visits.
```

two sources S1, S2 -- one pass, distances to the NEAREST

initial queue: [S1, S2] both at distance 0
2 1 S1 1 2 3 2 1 S2 1 2
 ^ ^

the two wavefronts advance simultaneously and meet where the distances are equal. Nothing special happens at the meeting point: the cell was simply claimed by whichever front reached it first.

THE KEY INSIGHT

Multi-source BFS is single-source BFS on a graph with one extra virtual vertex connected to every source by a zero-length edge. Seeding the queue with all sources is that virtual vertex, expanded. No change to the loop is required — recognising this saves rewriting the algorithm.

THE TRAP

Forgetting to mark all sources visited before the loop starts. If a source is adjacent to another source it gets re-enqueued at distance 1, and its own distance is overwritten. Symptom: distances that are correct except near clusters of sources — the hardest kind of off-by-one to spot in test output.

multi-source BFS — $O(V+E)$ once, not k times · queue seeded with k entries · $1000\times$ saving at $k=1000$ on a 10^6 -cell grid

50. Level-order traversal and two ways to find the level boundary

Sometimes you need to know which level each element belongs to — to print a tree by rows, to count levels, or to stop after k steps. Two techniques, and one is clearly better.

```
// METHOD 1 -- snapshot the size. Preferred: no extra memory, no sentinel.
std::vector<std::vector<int>> levels(Node* root) {
    std::vector<std::vector<int>> out;
    if (!root) return out;
    std::queue<Node*> q; q.push(root);
    while (!q.empty()) {
        std::size_t sz = q.size();          // EXACTLY this level's nodes
        out.emplace_back();
        for (std::size_t i = 0; i < sz; ++i) {
            Node* n = q.front(); q.pop();
            out.back().push_back(n->val);
            if (n->left) q.push(n->left); // pushes land AFTER the snapshot
            if (n->right) q.push(n->right);
        }
    }
    return out;
}

// METHOD 2 -- a nullptr sentinel between levels.
// q.push(root); q.push(nullptr);
// while (q.size() > 1) { n = q.front(); q.pop();
//     if (!n) { newlevel(); q.push(nullptr); continue; }
//     ... }
// Costs one extra queue entry per level and needs the q.size() > 1 guard --
// using !q.empty() here is an infinite loop, since the sentinel re-pushes.
```

tree:	1	queue evolution with METHOD 1
	/ \	[1] sz=1 -> emit [1], push 2,3
	2 3	[2,3] sz=2 -> emit [2,3], push 4,5,6
	/ \ \	[4,5,6] sz=3 -> emit [4,5,6]
	4 5 6	[] done. 3 levels.

the snapshot works because every push during a level lands strictly after the sz elements being processed -- FIFO guarantees exactly that.

THE KEY INSIGHT

The size snapshot is correct *because* of FIFO: elements pushed during this level cannot be dequeued until all sz current elements have gone. This is the same property that gives BFS its distance guarantee (Topic 3), used for a different purpose.

THE TRAP

Reading `q.size()` inside the inner loop instead of snapshotting it first: `for (int i = 0; i < q.size(); ++i)`. The size grows as children are pushed, so the loop consumes the next level too and every "level" is wrong from the second one on. Symptom: correct first level, then everything merges — and it looks right on a two-level test tree.

level order – $O(n)$ · size-snapshot: 0 extra space · sentinel: +1 entry per level, needs the `size>1` guard · never read `size()` in the loop test

51. Reconstructing the path from BFS parents

BFS gives distances; a path needs one extra array. Record, for each vertex, the vertex from which it was first reached, then walk backwards from the target and reverse.

```
std::vector<int> shortest_path(const std::vector<std::vector<int>>& adj,
                             int src, int dst) {
    std::vector<int> parent(adj.size(), -2);    // -2 unseen, -1 = source
    std::queue<int> q; q.push(src); parent[src] = -1;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (u == dst) break;                    // early exit is safe
        for (int v : adj[u])
            if (parent[v] == -2) { parent[v] = u; q.push(v); }
    }
    if (parent[dst] == -2) return {};            // unreachable
    std::vector<int> path;
    for (int v = dst; v != -1; v = parent[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}
// Memory: one int per vertex (4 MB at V = 10^6), versus storing a whole
// vector<int> path per queue entry, which is  $O(V * \text{pathlen}) = \text{gigabytes}$ .
```

parent array after BFS from 0:	path to 6:
v 0 1 2 3 4 5 6	6 -> parent 4
parent -1 0 0 1 1 2 4	4 -> parent 1
	1 -> parent 0
walk backwards: 6, 4, 1, 0	0 -> parent -1 (stop)
reverse: 0, 1, 4, 6	length 4 = dist[6] + 1

THE KEY INSIGHT

The parent array is a tree — the BFS tree — stored in V integers. Any "give me the path" requirement is one array plus a reversal, never a queue of paths. The same encoding answers "path to *every* vertex" for free.

THE TRAP

Pushing the whole path into the queue (`queue<vector<int>>`) so each entry knows its own route. It is correct and it multiplies memory by the path length: a 10^6 -vertex graph with paths of length 1000 needs ≈ 4 GB. Symptom: an algorithm that is correct on small inputs and gets OOM-killed on the real one.

path reconstruction – +1 int/vertex · walk $O(\text{path length})$ · queue-of-paths alternative: $O(V \cdot \text{len})$ memory, ~ 4 GB at $V=10^6$

52. 0-1 BFS with a deque

When every edge weight is 0 or 1, Dijkstra's heap is unnecessary. Push zero-weight neighbours to the *front* and one-weight neighbours to the *back*: the deque stays sorted by distance automatically, giving $O(V+E)$ instead of $O(E \log V)$.

```

std::vector<int> zero_one_bfs(
    const std::vector<std::vector<std::pair<int,int>>>& adj, int src) {
    const int INF = std::numeric_limits<int>::max();
    std::vector<int> dist(adj.size(), INF);
    std::deque<int> dq;
    dist[src] = 0; dq.push_back(src);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : adj[u]) {                // w is 0 or 1
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v);        // same "level"
                else       dq.push_back(v);         // next "level"
            }
        }
    }
    return dist;
}
// The deque holds at most two distinct distance values at any time: d and
// d+1. That is the invariant, and it is why the front is always minimal.
// V = 10^5, E = 5*10^5: 0-1 BFS ~ 6*10^5 ops; heap Dijkstra ~ 10^7.
// Typical measured speedup: 8-15x, plus no heap allocations at all.

```

grid where moving straight costs 0 and turning costs 1:

deque during expansion of a node at distance 3:

[3 3 3 | 4 4 4]

^front ^back

a 0-edge neighbour joins the 3-block at the FRONT (still distance 3)

a 1-edge neighbour joins the 4-block at the BACK (distance 4)

invariant: distances in the deque are non-decreasing front to back,
and span at most 2 values. Break it and the front is no longer minimal.

THE KEY INSIGHT

0-1 BFS is Dijkstra with the priority queue replaced by a deque, made possible because with only two edge weights the "priority order" collapses to two buckets — the same substitution as Dial's algorithm in Topic 39, with $C = 1$.

THE TRAP

Using it with a weight of 2 anywhere. The invariant "at most two distinct distances in the deque" breaks, the front is no longer the minimum, and you get distances that are wrong but plausible — usually too large on a few vertices. Symptom: a solution that passes 90 % of tests. If weights exceed 1, use Dial's (small C) or a heap.

0-1 BFS — $O(V+E)$ vs $O(E \log V)$ · deque spans ≤ 2 distance values · 8–15x measured · requires weights in $\{0,1\}$ exactly

53. The monotonic deque: sliding window maximum

Given an array and a window of size k , report the maximum of every window in $O(n)$ total. The structure is a deque of *indices* whose values are strictly decreasing; the front is always the current window's maximum.


```

std::vector<int> max_window(const std::vector<int>& a, int k) {
    std::deque<int> dq; // holds INDICES, not values
    std::vector<int> out;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // expire
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); // dominate
        dq.push_back(i);
        if (i >= k - 1) out.push_back(a[dq.front()]);
    }
    return out;
}
// WHY O(n): each index is pushed exactly once and popped at most once, so
// the total work of the inner while loop across the whole run is <= n.
// The loop LOOKS quadratic and is not -- state that explicitly when asked.
// n = 10^6, k = 1000: naive 10^9 comparisons (~1.2 s)
// monotonic deque <= 2*10^6 (~4 ms) 300x

```

```

a = [1, 3, -1, -3, 5, 3, 6, 7], k = 3      (deque holds indices)

i=0 a=1   dq=[0]                        -
i=1 a=3   3 >= 1 -> pop 0; dq=[1]       -
i=2 a=-1  dq=[1,2]                      window [1,3,-1] max = a[1] = 3
i=3 a=-3  dq=[1,2,3]                    front 1 > 3-3=0, still in window -> 3
i=4 a=5   pop 3,2,1 (all <= 5)           dq=[4]                                -> 5
i=5 a=3   dq=[4,5]                       -> 5
i=6 a=6   pop 5,4                         dq=[6]                                -> 6
i=7 a=7   pop 6                          dq=[7]                                -> 7
output: 3 3 5 5 6 7

```

THE KEY INSIGHT

Store indices, not values, so expiry by position is possible. And the discard rule is domination: if $a[j] \leq a[i]$ and $j < i$, then j can never be the maximum of any window that contains i , so it is safe to delete forever — that permanence is what makes each element pop at most once.

THE TRAP

Using $<$ instead of $\leq$ in the domination test. With duplicates, equal values accumulate in the deque; the answers stay correct but the deque grows to $O(n)$ and, worse, the expire step may pop an index that is still the maximum's twin. Symptom: correct output, memory blowup on inputs with many equal values — a common adversarial test.

monotonic deque — $O(n)$ total, each index pushed once and popped once · space $O(k)$ · 300x over naive at $n=10^6$, $k=1000$

54. Sliding window minimum, and the symmetry

Identical code with one comparison flipped. The point of stating it separately is that in an interview you must be able to derive one from the other without re-thinking, and to say what the deque is monotone *in*.

```
// maximum: deque values strictly DECREASING from front to back
// while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
// minimum: deque values strictly INCREASING from front to back
// while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
// Everything else -- expiry, push, emit -- is character for character
// identical. Write it once as a template on the comparator:

template <typename Cmp>
std::vector<int> window_extreme(const std::vector<int>& a, int k, Cmp worse) {
    std::deque<int> dq; std::vector<int> out;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        while (!dq.empty() && worse(a[dq.back()], a[i])) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) out.push_back(a[dq.front()]);
    }
    return out;
}
// max: worse = [](int back, int cur){ return back <= cur; }
// min: worse = [](int back, int cur){ return back >= cur; }
// "worse" means: the back element can never beat cur in any future window.
```

a = [4, 2, 12, 3, 8], k = 3

MAX deque (decreasing)	MIN deque (increasing)
i=0 [4]	[4]
i=1 [4,2]	2 <= 4 -> pop 4; [2]
i=2 12 pops 2,4 -> [12] =12	[2,12] = 2
i=3 [12,3] =12	3 <= 12 -> pop 12; [2,3] = 2
i=4 8 pops 3 -> [12,8] =12	expire index 1? front=1 <= 4-3=1 -> pop; [3,8] = 3

THE KEY INSIGHT

The deque is monotone in the quantity you are querying, and the front is always the answer because everything better arrived earlier and everything that arrived later and is worse is still behind it. Naming that invariant out loud is the whole derivation.

THE TRAP

Reaching for a `multiset` instead. It works — insert, erase the leaving element, read `*begin()` — and it is $O(n \log k)$ with allocation per element: measured at 10^6 elements and $k = 1000$, roughly **380 ms versus 4 ms**. Correct, and 95× slower.

window min/max — $O(n)$, one comparison differs · multiset alternative $O(n \log k)$ and ~95× slower
measured · space $O(k)$

55. Monotonic deque for DP transitions

The generalisation that makes the technique worth learning: any DP recurrence of the form $dp[i] = a[i] + \max(dp[j])$ over a bounded window $j \in [i-k, i-1]$ is a sliding-window maximum over the `dp` array itself.

```
// "Constrained subsequence sum": pick a subsequence where consecutive
// chosen indices differ by at most k; maximise the sum.
// dp[i] = a[i] + max(0, max_{j in [i-k, i-1]} dp[j])
int constrained_max_sum(const std::vector<int>& a, int k) {
    int n = a.size();
    std::vector<int> dp(n);
    std::deque<int> dq; // indices, dp values decreasing
    int best = std::numeric_limits<int>::min();
    for (int i = 0; i < n; ++i) {
        if (!dq.empty() && dq.front() < i - k) dq.pop_front();
        dp[i] = a[i] + (dq.empty() ? 0 : std::max(0, dp[dq.front()]));
        while (!dq.empty() && dp[dq.back()] <= dp[i]) dq.pop_back();
        dq.push_back(i);
        best = std::max(best, dp[i]);
    }
    return best;
}
// Naive: for each i scan k predecessors -> O(n*k). n = 10^5, k = 10^4
// -> 10^9 operations (~1 s). With the deque: 10^5 (~0.3 ms). 3000x.
// The same shape solves: jump game VI, sliding window cost DP, and any
// "best in the last k states" recurrence.
```

dp computed left to right; the deque holds candidate PREDECESSORS

```

      i-k      i-1  i
...[ j j j j j ] ?      window of valid predecessors
      \_____/
        deque front = argmax dp[j] over the window

```

The subtlety versus Topic 53: `dp[i]` is computed BEFORE index `i` enters the deque, because `i` is not its own predecessor. Order of the three steps -- expire, read, insert -- is not interchangeable.

THE KEY INSIGHT

The deque optimises a DP by remembering only the candidates that could still win. If the recurrence's window is bounded and the aggregate is min or max, the log factor a heap would cost is avoidable entirely — and the code is Topic 53 with `dp` substituted for `a`.

THE TRAP

Pushing `i` before computing `dp[i]`. Then `i` is its own predecessor, `dp[i]` can include itself, and the answer inflates. Symptom: results that are too large by exactly one `a[i]` term somewhere — correct on inputs where all values are negative, wrong otherwise.

window DP — $O(n)$ vs $O(n \cdot k)$ · 3000x at $n=10^5$, $k=10^4$ · order: expire → read → insert, never rearranged

56. A window as a queue: first unique character in a stream

Not every window problem needs monotonicity. Here the queue holds candidates in arrival order and a frequency table decides which are still valid — the queue and the map together answer a question neither can answer alone.

```

class FirstUnique {
    std::queue<char> q_;
    std::array<int, 256> count_{};
public:
    void feed(char c) {
        ++count_[(unsigned char)c];
        q_.push(c);
        while (!q_.empty() && count_[(unsigned char)q_.front()] > 1)
            q_.pop(); // discard permanently: a char
    } // that repeated can never again
    char first_unique() const { // be unique
        return q_.empty() ? '#' : q_.front();
    }
};
// Stream "aabc": feed a -> q=[a] -> 'a'
//                feed a -> count[a]=2, pop a -> q=[] -> '#'
//                feed b -> q=[b] -> 'b'
//                feed c -> q=[b,c] -> 'b'
// Each character is pushed once and popped at most once: O(1) amortised
// per character, O(alphabet + queue) space.

```

stream: a a b c b d

q: [a]	counts a=1	first = a
q: []	a=2, front popped	first = # (none)
q: [b]	b=1	first = b
q: [b,c]	c=1	first = b
q: [c]	b=2 -> pop b	first = c
q: [c,d]	d=1	first = c

the queue never holds a character whose count exceeds 1 at its front -- but it MAY hold one deeper down, which is fine: it will be popped when it reaches the front.

THE KEY INSIGHT

Lazy invalidation at the front, again: the same pattern as stale entries in Topic 37's Dijkstra and expired indices in Topic 53. Anything that becomes invalid and can never become valid again may be left in place until it reaches the front, where discarding it is O(1).

THE TRAP

Popping with **if** instead of **while**. After a burst of repeats several consecutive front entries can be invalid; a single **if** clears only one. Symptom: `first_unique()` returns a character whose count is 2 — correct most of the time, wrong specifically on runs of repeated characters.

first unique — feed O(1) amortised · query O(1) · space O(alphabet) + queue · each char pushed once, popped once

57. Kahn's topological sort

A queue of vertices with in-degree zero. Repeatedly remove one, decrement its neighbours' in-degrees, and enqueue any that reach zero. If fewer than V vertices come out, the graph has a cycle — and the queue told you so for free.

```

std::vector<int> topo_sort(const std::vector<std::vector<int>>& adj) {
    int V = adj.size();
    std::vector<int> indeg(V, 0), order;
    for (int u = 0; u < V; ++u) for (int v : adj[u]) ++indeg[v];
    std::queue<int> q;
    for (int u = 0; u < V; ++u) if (indeg[u] == 0) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    if ((int)order.size() != V) return {}; // CYCLE detected
    return order;
}
// O(V + E). The queue's peak size is the width of the dependency graph --
// for a build system, the number of targets that can compile in parallel.
// Swap std::queue for std::priority_queue and you get the
// lexicographically smallest topological order, at O((V+E) log V).

```

DAG: a -> c, b -> c, c -> d, b -> d

indeg: a=0 b=0 c=2 d=2	q=[a,b]		
pop a: c->1	q=[b]	order=[a]	
pop b: c->0 push c, d->1	q=[c]	order=[a,b]	
pop c: d->0 push d	q=[d]	order=[a,b,c]	
pop d:	q=[]	order=[a,b,c,d]	size == V, ok

with a cycle c -> b added: indeg b never reaches 0, the queue empties early, order.size() = 1 < 4 -> cycle.

THE KEY INSIGHT

Kahn's is BFS where the "visited" condition is *all predecessors processed* rather than *seen once*. The queue holds exactly the currently-runnable set, which is why the same code schedules a build, resolves package dependencies, or detects a dependency cycle without modification.

THE TRAP

Enqueueing a vertex when its in-degree merely *decreases*, rather than when it hits zero: `--indeg[v]; if (indeg[v] <= 0) q.push(v);` pushes the same vertex once per incoming edge. Symptom: an output longer than V with duplicates, and the cycle check then fires on perfectly acyclic graphs. Use `if (--indeg[v] == 0)` — exact equality, decrement inside.

Kahn's — $O(V+E)$ · queue peak = graph width · cycle detection free · priority_queue variant: lexicographic order at $O((V+E) \log V)$

58. Round-robin and Josephus simulation

Two simulations where the queue is the model. Round-robin scheduling rotates a task to the back after its time slice; Josephus eliminates every k^{th} element from a circle.

```
// Round-robin: total completion times for a quantum of Q.
std::vector<int> round_robin(std::vector<int> burst, int Q) {
    std::vector<int> done(burst.size());
    std::queue<int> q;
    for (std::size_t i = 0; i < burst.size(); ++i) q.push(i);
    int t = 0;
    while (!q.empty()) {
        int i = q.front(); q.pop();
        int run = std::min(Q, burst[i]);
        t += run; burst[i] -= run;
        if (burst[i] > 0) q.push(i);           // back of the line
        else done[i] = t;
    }
    return done;
}

// Josephus: n people, every k-th eliminated. O(n*k) by simulation.
int josephus(int n, int k) {
    std::queue<int> q;
    for (int i = 1; i <= n; ++i) q.push(i);
    while (q.size() > 1) {
        for (int i = 1; i < k; ++i) { q.push(q.front()); q.pop(); } // rotate
        q.pop(); // eliminate
    }
    return q.front();
}

// n = 10^6, k = 7: simulation = 7*10^6 queue ops (~40 ms).
// Closed form J(n,k) = (J(n-1,k) + k) % n, J(1,k)=0 : 10^6 ops, O(1) space.
```

round-robin, bursts [5, 3, 4], Q = 2

```
t=0  run 0 for 2 (3 left)  q=[1,2,0]
t=2  run 1 for 2 (1 left)  q=[2,0,1]
t=4  run 2 for 2 (2 left)  q=[0,1,2]
t=6  run 0 for 2 (1 left)  q=[1,2,0]
t=8  run 1 for 1 DONE t=9  q=[2,0]
t=9  run 2 for 2 DONE t=11 q=[0]
t=11 run 0 for 1 DONE t=12
context switches: 7. With Q = 100: 3 switches, but task 1 waits 5 units.
```

THE KEY INSIGHT

Round-robin is the FIFO fairness argument of Topic 2 made concrete: the quantum bounds how long any task waits behind another, at the cost of one context switch per quantum. Q is the direct latency/throughput dial — small Q gives low waiting time and high switch overhead (Topic 67).

THE TRAP

Simulating Josephus with a queue when n and k are large. $n = 10^6$, $k = 10^6$ means 10^{12} rotations — hours. The recurrence $J(n,k) = (J(n-1,k) + k) \% n$ answers it in n steps. Symptom: a solution that is elegant, correct, and times out. Simulate to understand; use the recurrence to submit.

round-robin — $O(\sum \text{burst}/Q)$ switches · Josephus simulation $O(n \cdot k)$ · recurrence $O(n)$ time, $O(1)$ space · 10^{12} vs 10^6 at $n=k=10^6$

PRACTICE SET — Topics 47-58

A. Conceptual

1. Name the three jobs the `dist` array does in grid BFS.
2. Why does dequeue-time visited-marking keep BFS correct but change the queue's peak size from $O(V)$ to $O(E)$?
3. Why is multi-source BFS the same algorithm as single-source BFS? State the graph transformation.
4. Explain why the level-size snapshot is correct, citing the FIFO property it depends on.
5. Why does storing a parent array beat storing a path per queue entry? Give the memory ratio in terms of path length.
6. State the 0-1 BFS deque invariant and the exact condition under which it breaks.
7. Why does the monotonic deque hold indices rather than values?
8. Give the domination argument that makes each index pop at most once in the sliding window maximum.
9. What single change converts window-maximum into window-minimum, and what does "monotone" refer to in each?
10. In the window-DP of Topic 55, why must `dp[i]` be computed before `i` is pushed?
11. What property of "this character repeated" makes lazy front-invalidation valid in Topic 56?
12. How does Kahn's algorithm detect a cycle without any extra bookkeeping?
13. Why is `if (--indeg[v] == 0)` required rather than `<= 0`?
14. Give the trade-off the round-robin quantum controls, in both directions.

B. Tracing and analysis

1. Run BFS by hand on the 4×4 grid with walls at (1,1) and (1,2), from (0,0) to (3,3). Give the `dist` grid and the answer.
2. For a 2000×2000 open grid, estimate the peak queue size and the memory for the queue if entries are `pair<int,int>` versus a single packed `int`.
3. On the complete graph K_{500} , give the total number of queue pushes under enqueue-marking and under dequeue-marking.
4. Rotting oranges on a 3×3 grid with rotten at (0,0) and (2,2) and all others fresh: give the minute at which each cell rots and the answer.
5. Trace `max_window([9,2,7,4,4,8,1], k=3)`, showing the deque of indices at each step and the output.
6. Trace the window-minimum on the same array and `k`.
7. Trace `constrained_max_sum([10,-2,-10,-5,20], k=2)`, giving `dp` and the deque at each `i`.
8. Run Kahn's on the DAG with edges $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 4$, $4 \rightarrow 5$. Give the in-degree array, the queue at each step, and the order.
9. Round-robin with bursts [8, 2, 5] and $Q = 3$: give the completion time for each task and the number of context switches. Repeat for $Q = 1$.
10. Josephus with $n = 7$, $k = 3$: give the elimination order and the survivor by simulation, then verify with the recurrence.
11. Compare 0-1 BFS with heap Dijkstra on $V = 10^5$, $E = 6 \times 10^5$: give the operation counts for both.

C. Code tasks — build it, break it, observe, fix

1. Implement grid BFS with packed `int` queue entries. Test on a 2000×2000 maze and report peak queue size, total cells visited, and runtime.
2. **Break it:** move the visited-marking from enqueue time to dequeue time in graph BFS. Run on a random graph with $V = 10^4$, $E = 2 \times 10^5$, instrumenting maximum queue size and total pushes. Record both versions' numbers and confirm the distances are identical.
3. **Break it:** write the level loop as `for (int i = 0; i < q.size(); ++i)` without the snapshot. Run it on a perfect binary tree of depth 4 and record the exact (wrong) level partition. Fix and re-run.
4. Implement path reconstruction. On a 10^6 -vertex sparse graph, measure peak RSS for the parent-array version and for a `queue<vector<int>>` version (use a smaller graph if the latter OOMs — and note at which size it does).
5. **Break it:** run 0-1 BFS on a graph containing one edge of weight 2. Compare its output with a correct Dijkstra on the same graph and record how many vertices differ and by how much.
6. Implement `max_window`. Verify against a brute-force $O(n \cdot k)$ version on 10^4 random arrays, then benchmark both at $n = 10^6$, $k = 1000$ and report the ratio.
7. **Break it:** change the domination test to `<`. Run on an array of 10^6 elements that are all equal, and record the maximum deque size for both versions. Note whether the output is still correct.
8. Implement the sliding-window multiset version and benchmark it against the deque at $n = 10^6$, $k = 1000$. Report both times.
9. **Break it:** in Topic 55, push `i` before computing `dp[i]`. Run on `[10, -2, -10, -5, 20]` with $k=2$ and record the wrong answer, then explain which term was double-counted.
10. Implement Kahn's with the cycle check. Feed it a graph with a 3-cycle among 1000 acyclic vertices and confirm it returns empty; then use the `priority_queue` variant and confirm lexicographic order on a DAG.
11. Implement Josephus both ways for $n = 10^5$, $k = 5$. Time both and confirm identical survivors.

ANSWER KEY — Topics 47-58

A. Conceptual

1. **The visited set, the answer (distance), and the queue's deduplication.** One array, three roles — which is why no separate `visited` is needed.
2. Correct because the first dequeue of a vertex is still via a shortest path. But a vertex can be pushed once **per incoming edge** before it is first dequeued, so total pushes become $O(E)$.
3. Add a **virtual super-source connected to every real source by a zero-length edge**. Seeding the queue with all sources at distance 0 is that super-source already expanded.
4. Because **FIFO guarantees that anything pushed during this level is dequeued only after all `sz` current elements**. The snapshot therefore delimits exactly the current level.
5. Parent array is **one int per vertex**; a path per entry is $O(V \times \text{path length})$. The ratio is **the path length** — a factor of 1000 for paths of 1000.
6. Invariant: **distances in the deque are non-decreasing front to back and span at most two values, d and $d+1$** . It breaks as soon as any edge weight other than 0 or 1 exists, because a third distance class can enter.
7. Because **expiry is by position**: you must know whether the front candidate has fallen out of the window, and the value alone does not tell you that.
8. If $j < i$ and $a[j] \leq a[i]$, then every future window containing j also contains i , so j can **never** be the maximum again and its removal is permanent. Each index is pushed once and removed once $\rightarrow O(n)$ total.
9. Flip the domination comparison from $\leq$ to $\geq$. In the maximum case the deque values are **decreasing** front to back; in the minimum case **increasing**.
10. Because the window of valid predecessors is $[i-k, i-1]$ — **i is not its own predecessor**. Pushing first lets `dp[i]` read itself.
11. Once a character's count exceeds 1, it can **never return to 1** — counts only increase. So an invalid front entry is permanently invalid and discarding it lazily is safe.
12. If a cycle exists, every vertex in it always has at least one unprocessed predecessor, so its in-degree never reaches zero; the queue empties early and `order.size() < V`.
13. Because ≤ 0 is true on every subsequent decrement too, so a vertex with in-degree 3 would be pushed **three times**. Exact equality fires exactly once, at the moment the last predecessor is processed.
14. Small Q: **lower waiting time / better responsiveness, more context switches**. Large Q: fewer switches and higher throughput, but tasks wait behind long-running ones — at $Q = \infty$ it degenerates to FCFS.

B. Tracing and analysis

1. Walls at (1,1),(1,2). Distances: row0 = 0 1 2 3; row1 = 1 # # 4; row2 = 2 3 4 5; row3 = 3 4 5 6. Answer: **6**.
2. Peak frontier $\approx$ the diamond ring $\approx 2 \times 2000 = 4000$ cells (up to ~ 8000 near the middle of an open square). Memory: pair = 8 B $\rightarrow \sim 32\text{-}64$ KB; packed int = 4 B $\rightarrow \sim 16\text{-}32$ KB. The real saving on the dist array and cache density is larger than the queue itself.
3. Enqueue-marking: **500 pushes** (each vertex once). Dequeue-marking: the source pushes 499; each of those pushes up to 498 more before being marked $\rightarrow$ on the order of $E = 500 \times 499 / 2 \times 2 \approx 2.5 \times 10^5$ pushes.

4. (0,0) and (2,2) rotten at minute 0. Minute 1: (0,1),(1,0),(1,2),(2,1). Minute 2: (0,2),(1,1),(2,0). All rotten → answer **2**.
5. [9,2,7,4,4,8,1], $k=3$. $i=0$: $dq=[0]$. $i=1$: $2<9 \rightarrow dq=[0,1]$. $i=2$: $7\geq 2$ pop 1 → $dq=[0,2]$; emit $a[0]=9$. $i=3$: front $0 \leq 0 \rightarrow$ expire; $4<7 \rightarrow dq=[2,3]$; emit **7**. $i=4$: $4\geq 4$ pop 3 → $dq=[2,4]$; emit **7**. $i=5$: front $2 \leq 2 \rightarrow$ expire; 8 pops 4 → $dq=[5]$; emit **8**. $i=6$: $dq=[5,6]$; emit **8**. Output: **9 7 7 8 8**.
6. Minimum: $i=0$ $dq=[0]$; $i=1$: $2\leq 9$ pop 0 → $dq=[1]$; $i=2$: $dq=[1,2]$ emit **2**; $i=3$: $4\leq 7$ pop 2 → $dq=[1,3]$ emit **2**; $i=4$: $4\geq 4$? test is $\geq$ so pop 3 → $dq=[1,4]$; front $1 \leq 1 \rightarrow$ expire → $dq=[4]$; emit **4**; $i=5$: $dq=[4,5]$ emit **4**; $i=6$: 1 pops 5,4 → $dq=[6]$; emit **1**. Output: **2 2 4 4 1**.
7. $k=2$. $i=0$: dq empty → $dp[0]=10$; $dq=[0]$. $i=1$: $dp[1] = -2 + \max(0,10) = 8$; $8\leq 10$ so no pop; $dq=[0,1]$. $i=2$: front $0 \geq 2-2=0$ keep; $dp[2] = -10 + 10 = 0$; $dq=[0,1,2]$. $i=3$: front $0 < 1 \rightarrow$ expire; $dp[3] = -5 + \max(0,dp[1]=8) = 3$; pop 2 ($0\leq 3$) → $dq=[1,3]$. $i=4$: front $1 < 2 \rightarrow$ expire; $dp[4] = 20 + \max(0,dp[3]=3) = 23$. Answer **23**.
8. indeg: 1=0, 2=1, 3=1, 4=2, 5=1. $q=[1]$. Pop 1 → 2,3 reach 0, $q=[2,3]$, order=[1]. Pop 2 → 4→1, $q=[3]$, order=[1,2]. Pop 3 → 4→0, $q=[4]$, order=[1,2,3]. Pop 4 → 5→0, $q=[5]$. Pop 5. Order = **1,2,3,4,5**, size 5 = V.
9. $Q=3$, bursts [8,2,5]: $t=3$ (A, 5 left), $t=5$ (B done, **B=5**), $t=8$ (C, 2 left), $t=11$ (A, 2 left), $t=13$ (C done, **C=13**), $t=15$ (A done, **A=15**). Switches: **6** dispatches, 5 switches between different tasks. With $Q=1$: total time still 15, but **15 dispatches**; B completes at **t=5**, C at **t=13**, A at **t=15**.
10. $n=7,k=3$ elimination order: **3, 6, 2, 7, 5, 1**, survivor **4**. Recurrence: $J(1)=0$, $J(2)=(0+3)\%2=1$, $J(3)=(1+3)\%3=1$, $J(4)=(1+3)\%4=0$, $J(5)=(0+3)\%5=3$, $J(6)=(3+3)\%6=0$, $J(7)=(0+3)\%7=3 \rightarrow$ 0-based 3 = **person 4**. Agrees.
11. 0-1 BFS: each edge relaxed once → $\approx 7 \times 10^5$ **operations**, no allocations. Heap Dijkstra: up to E pushes and pops at $\log_2(6 \times 10^5) \approx 19 \rightarrow \approx 2.3 \times 10^7$ **comparisons** — roughly **30×** more work.

C. Code tasks — what to verify

1. Peak queue should be in the **thousands, not millions** — if it approaches $R \cdot C$ you are pushing before marking. Cells visited must be $\leq R \cdot C$ exactly once each.
2. Distances must be **identical**. Expect total pushes $\approx 10^4$ (enqueue-marking) versus $\approx 2 \times 10^5$ (dequeue-marking) — a **20×** difference matching E/V , and peak queue size similarly inflated.
3. Depth-4 perfect binary tree (15 nodes). Without the snapshot the first level is [1], then the loop keeps consuming as the queue grows, typically producing **one giant second "level" containing the rest**. Record the exact partition; it depends on push timing, which is itself the lesson.
4. Parent array: **~4 MB** for 10^6 vertices plus the graph. queue<vector<int>>: expect OOM somewhere around **10^5 - 10^6 vertices** with long paths; record the exact vertex count at which it dies.
5. Expect a **small number of vertices with distances that are too large** — typically those reachable only through the weight-2 edge. The count is usually small, which is exactly why the bug survives most test suites.
6. Outputs must match on all 10^4 arrays. Benchmark: expect **~1.2 s brute force versus ~4 ms deque**, roughly **300×**.
7. With $\leq$ the deque stays at size **1** on an all-equal array. With $<$ it grows to **k = 1000** (and to n if $k = n$). Output remains correct — note that the failure is memory and constant factor, not correctness.

8. Expect roughly **380 ms (multiset) versus 4 ms (deque)**, about **95×**. Confirm outputs match exactly.
9. Wrong answer will exceed 23 — typically **$dp[4] = 20 + 20 = 40$** at the point where index 4 sees itself. The double-counted term is **$a[i]$ itself**.
10. The cyclic graph must return an empty vector and **`order.size()`** must equal **1000**, not 1003. The priority-queue variant on a DAG must produce the **lexicographically smallest** valid order — verify against a brute-force enumeration on a 6-vertex DAG.
11. Survivors must be identical. Expect simulation **$\approx 5 \times 10^5$ queue operations, a few ms**; the recurrence **10^5 operations, well under a millisecond**. Re-run with $k = 10^5$ to see the simulation become unusable.

HANDNOTE SUMMARY CARD

PART SEVEN - PATTERNS AND ALGORITHMS

topics 47-58

BFS SKELETON - memorise this, everything else is a variation

```
dist[src] = 0; q.push(src);
while (!q.empty()) { u = q.front(); q.pop();
  for (v : adj[u]) if (dist[v] == -1) { dist[v] = dist[u]+1; q.push(v); } }
*** MARK AT ENQUEUE *** dequeue-marking is still correct but queue traffic
becomes O(E) not O(V): K 500 -> 2.5e5 pushes instead of 500
dist array = visited set + answer + dedup, all three
grid: no adjacency list; dr[]={-1,1,0,0}, dc[]={0,0,-1,1}; pack r*C+c as ONE
int (half the memory, twice the cache density)
queue peak on an open grid ~ O(sqrt(cells)), not O(cells)
```

VARIATIONS

multi-source seed the queue with ALL sources at distance 0 = a virtual
 super-source with 0-weight edges. one pass, not k passes.

level order sz = q.size() SNAPSHOT, then loop sz times.
 for (i = 0; i < q.size(); ++i) is the classic bug.

path parent[] array, walk back from dst, reverse.
 queue> costs O(V * pathlen) = GBs.

0-1 weights deque: w==0 -> push_front, w==1 -> push_back. O(V+E).
 invariant: deque spans at most TWO distance values, d and d+1.
 ONE edge of weight 2 breaks it silently.

Kahn's topo queue of indeg==0; if (--indeg[v] == 0) push. "<= 0" pushes
 a vertex once per edge. order.size() != V => CYCLE, free.
 priority_queue variant -> lexicographically smallest order.

MONOTONIC DEQUE (window max) O(n): each index pushed once, popped once

```
if (dq.front() <= i - k) dq.pop_front(); // expire by POSITION
while (a[dq.back()] <= a[i]) dq.pop_back(); // domination, PERMANENT
dq.push_back(i); if (i >= k-1) emit a[dq.front()];
HOLD INDICES, NOT VALUES (expiry needs position)
min: flip to a[dq.back()] >= a[i]. max -> decreasing deque, min -> increasing
"<" instead of "<=" -> still correct, deque grows to k on equal values
multiset alternative: O(n log k), measured 95x slower (380 ms vs 4 ms)
n=1e6 k=1000: naive 1.2 s, deque 4 ms -> 300x
```

WINDOW DP dp[i] = a[i] + max(dp[j]), j in [i-k, i-1]

order: EXPIRE -> READ dp[dq.front()] -> COMPUTE dp[i] -> INSERT i

pushing i first makes i its own predecessor -> answer inflated by a[i]

O(n) vs O(n*k): 3000x at n=1e5, k=1e4

LAZY FRONT INVALIDATION - one idea, three places

Dijkstra stale entry if (d > dist[v]) continue; (T37)

window expiry if (dq.front() <= i-k) pop_front(); (T53)

first-unique-char WHILE (count[q.front()] > 1) pop(); (T56) <- while, not if

valid because "invalid" is PERMANENT: counts only rise, windows only move right

SIMULATIONS

round-robin run min(Q, burst); if (burst > 0) q.push(i) else done[i] = t
 small Q = low wait + many switches; Q = inf degenerates to FCFS

Josephus simulate O(n*k); SUBMIT the recurrence J(n) = (J(n-1)+k) % n
 n = k = 1e6: 1e12 rotations vs 1e6 steps

PART EIGHT

Concurrency

Two threads, one buffer, and the four ways it goes wrong.

Part Eight · Concurrency

59. Producer-consumer: the bounded blocking queue

The queue that connects two threads. A mutex protects the ring of Part Three; two condition variables let a thread sleep rather than spin when the queue is empty (consumer) or full (producer). Bounding it is not optional — it is the backpressure of Topic 66.

```
template <typename T>
class BlockingQueue {
    std::mutex m_;
    std::condition_variable not_empty_, not_full_;
    std::deque<T> q_;
    std::size_t cap_;
    bool closed_ = false;
public:
    explicit BlockingQueue(std::size_t cap) : cap_(cap) {}
    bool push(T x) {
        std::unique_lock<std::mutex> lk(m_);
        not_full_.wait(lk, [&]{ return q_.size() < cap_ || closed_; });
        if (closed_) return false;
        q_.push_back(std::move(x));
        lk.unlock();                // unlock BEFORE notify: the woken
        not_empty_.notify_one();    // thread need not block on us
        return true;
    }
    std::optional<T> pop() {
        std::unique_lock<std::mutex> lk(m_);
        not_empty_.wait(lk, [&]{ return !q_.empty() || closed_; });
        if (q_.empty()) return std::nullopt;    // closed and drained
        T v = std::move(q_.front()); q_.pop_front();
        lk.unlock();
        not_full_.notify_one();
        return v;
    }
    void close() { { std::lock_guard<std::mutex> lk(m_); closed_ = true; }
                  not_empty_.notify_all(); not_full_.notify_all(); }
};
// Cost per operation: ~20-25 ns uncontended (a lock/unlock pair), 1-5 us
// when a thread actually sleeps and must be woken by the kernel (futex).
```

```
producer                queue (cap 4)                consumer
push -----> [lock]   ----> [ a b c d ] FULL          ----> pop
                        |                               |
                        | not_full_.wait                | notify_one
                        |                               |
                        | wakes, pushes e -----> [ b c d e ] -----> consumed
```

Two condition variables, not one: a single CV would wake producers when the queue becomes non-empty (useless) and consumers when it becomes non-full (useless) -- a "thundering herd" of pointless wakeups.

THE KEY INSIGHT

Closing is part of the design, not an afterthought. Without a `closed_` flag checked inside both predicates, a consumer blocked in `wait` at shutdown never returns and the process hangs on join — the single most common bug in hand-written worker pools.

THE TRAP

An unbounded blocking queue — no `not_full_`, no `cap_`. Producers never block, so a consumer that slows down converts directly into unbounded memory growth. Symptom: RSS climbing linearly during a latency incident, then an OOM kill that looks like the cause of the outage rather than a consequence. See Topic 66.

blocking queue — ~20–25 ns uncontended · 1–5 µs per sleep/wake · 2 CVs required · cap bounds memory AND applies backpressure

60. Lost wakeups, spurious wakeups, while not if

Two distinct failure modes, one fix. A *spurious* wakeup is a thread waking with the predicate still false; a *lost* wakeup is a notify that happens while no one is waiting and is therefore discarded forever.

```
// WRONG -- if, not while:
std::unique_lock<std::mutex> lk(m_);
if (q_.empty()) not_empty_.wait(lk);           // wakes -> ASSUMES non-empty
T v = q_.front();                             // may be empty: UB
// Two ways this fails:
// 1. spurious wakeup: POSIX and C++ explicitly permit wait() to return
//    without any notify. The standard says so; it is not a bug to fix.
// 2. stolen wakeup: two consumers are notified, both wake, the first
//    takes the only element, the second proceeds on an empty queue.

// RIGHT -- the predicate form re-checks in a loop for you:
not_empty_.wait(lk, [&]{ return !q_.empty() || closed_; });
// which is exactly:
// while (!(q_.empty() || closed_)) not_empty_.wait(lk);

// LOST WAKEUP -- notify with no waiter yet:
// consumer: if (q_.empty())          <-- checks, sees empty
// producer:      push + notify_one() <-- nobody waiting yet
// consumer:      wait(lk)            <-- sleeps forever
// The mutex is what prevents this: the check and the wait must be ATOMIC
// with respect to the producer, which is why wait() takes the lock and
// releases it only after registering the waiter.
```

lost wakeup timeline (no lock held across check-then-wait):

```
consumer  ---- check empty? yes ----X----- wait(...) ---> sleeps
producer  ----- push, notify_one -----> (no waiter)
```

^
the notify lands here and is DISCARDED

with the mutex held from the check through wait(), the producer cannot run in that gap: it blocks on the lock until wait() has registered.

THE KEY INSIGHT

A condition variable carries no state. It is not a semaphore and it does not count: a notify with no waiter is silently dropped. The *state* lives in the queue, the mutex protects it, and the predicate is the only thing that decides whether a thread should proceed.

THE TRAP

Using `if` works perfectly on a machine with one producer and one consumer under light load, for hours. Symptom under load: a `std::deque::front()` on an empty deque — a seg-fault or garbage value inside the queue, with a stack trace that implicates the queue rather than the caller's wait loop. ThreadSanitizer will not flag it either, since there is no data race: the lock is held. Only the predicate check is missing.

wait – always predicate form · spurious wakeups are legal, not bugs · a notify with no waiter is discarded · the mutex makes check+wait atomic

61. The SPSC lock-free ring: acquire and release

With exactly one producer thread and one consumer thread, the mutex can go away entirely. Each index has a single writer (Topic 10, fix 3), and two atomic operations with the right memory ordering replace the lock.

```
template <typename T, std::size_t N>           // N a power of two
class SpscRing {
    static_assert((N & (N - 1)) == 0);
    T buf_[N];
    std::atomic<std::size_t> head_{0};          // written ONLY by the consumer
    std::atomic<std::size_t> tail_{0};          // written ONLY by the producer
public:
    bool push(const T& x) {                     // producer thread only
        std::size_t t = tail_.load(std::memory_order_relaxed); // own index
        std::size_t h = head_.load(std::memory_order_acquire); // theirs
        if (t - h == N) return false;           // full
        buf_[t & (N - 1)] = x;                  // WRITE
        tail_.store(t + 1, std::memory_order_release); // PUBLISH
        return true;
    }
    bool pop(T& out) {                           // consumer thread only
        std::size_t h = head_.load(std::memory_order_relaxed);
        std::size_t t = tail_.load(std::memory_order_acquire); // theirs
        if (h == t) return false;               // empty
        out = buf_[h & (N - 1)];                 // READ
        head_.store(h + 1, std::memory_order_release);
        return true;
    }
};
// The release store on tail_ guarantees the buf_ write is visible BEFORE
// the new tail_ is; the acquire load on tail_ guarantees the consumer sees
// that write. Downgrade either to relaxed and you get a torn read.
// On x86 both compile to plain MOV (the hardware already orders them);
// on ARM the release emits STLR / the acquire LDAR -- real instructions,
// real cost, which is why testing only on x86 hides ordering bugs.
```

the pairing that makes it correct:

PRODUCER	CONSUMER
buf_[t & mask] = x; -----.	
tail_.store(t+1, RELEASE) -----+---->	t = tail_.load(ACQUIRE)
	if (h != t)
everything before the release	out = buf_[h & mask] <-- sees x
is visible to everything after	
the matching acquire -----'	

relaxed instead of release: the compiler or CPU may reorder the buf_ write AFTER the tail_ store. The consumer then reads a slot that has not been written yet. Symptom: occasional garbage elements, only on ARM.

THE KEY INSIGHT

Lock-free here does not come from clever atomics; it comes from the *ownership split*. One writer per index is what removes the need for mutual exclusion, and the memory ordering only ensures the payload write is visible before the index that advertises it. Add a second producer and the whole design collapses (Topic 63).

THE TRAP

Using this class with two producers because "it worked in testing". Two threads read the same `tail_`, both write the same slot, and one element is silently lost while the count says both arrived. Symptom: a message count mismatch of a few parts per million under load, and no crash ever. Assert the thread identity in debug builds if the class is shared.

SPSC ring – push/pop ~2–5 ns, no syscall · 1 acquire + 1 release per op · free on x86, LDAR/STLR on ARM · exactly 1 producer, 1 consumer

62. False sharing, true sharing, and cached cursors

The correct SPSC ring above is slow for a reason that has nothing to do with correctness: `head_` and `tail_` sit in the same 64-byte cache line, so each thread's store invalidates the other's line. The textbook fix is padding — and padding alone can make it worse.

```
// Step 1: pad the cursors onto separate cache lines.
    alignas(64) std::atomic<std::size_t> head_{0};
    alignas(64) std::atomic<std::size_t> tail_{0};
// FALSE sharing is now gone. But both threads still READ the other's
// cursor on EVERY operation -- that is TRUE sharing, and it now costs two
// bouncing cache lines instead of one. A reported benchmark on an M4 Pro
// measured padding alone as a THIRD SLOWER than the unpadded version.

// Step 2: cache the other side's cursor and only refresh when necessary.
    alignas(64) std::atomic<std::size_t> tail_{0};
    std::size_t cached_head_ = 0;           // producer-private, no atomics
    bool push(const T& x) {
        std::size_t t = tail_.load(std::memory_order_relaxed);
        if (t - cached_head_ == N) {        // MAYBE full
            cached_head_ = head_.load(std::memory_order_acquire); // refresh
            if (t - cached_head_ == N) return false;           // really
        }
        buf_[t & (N-1)] = x;
        tail_.store(t + 1, std::memory_order_release);
        return true;
    }
// Now the producer touches head_ only when it believes the ring is full --
// roughly once per N pushes instead of once per push.
// Reported measurement, capacity 1024, 200 M uint32, two cores:
//   naive (shared line) ..... ~32 M ops/s
//   padding only ..... ~21 M ops/s    (SLOWER)
//   padding + cached cursors ..... ~440 M ops/s    (13.8x)
```

one 64-byte cache line holding both cursors:

```
[ head_ | tail_ | ... ]    producer stores tail_ -> line invalidated
                          consumer stores head_ -> line invalidated
every operation ping-pongs the line between cores: ~100-200 cycles each.
```

```
padded, no caching:      two lines, but each thread READS the other
                          line every operation -> two ping-pongs.
padded + cached:         each thread reads the other's line ~1/N
                          operations. The lines stay in Shared state.
```

THE KEY INSIGHT

Eliminate *true* sharing before fixing false sharing. Padding separates variables that are accidentally together; it does nothing about variables both threads genuinely need to read. Caching the remote cursor is what removes the real traffic, and padding then keeps the benefit.

THE TRAP

Applying `alignas(64)` as a reflex and declaring victory without measuring. Symptom: 128 extra bytes per queue, a benchmark that got slower, and a comment in the code claiming the opposite. Measure before and after — on the target architecture, since x86 and ARM differ here (Topic 61).

sharing – shared line ~32 M ops/s · padding only ~21 M · padding + cached cursors ~440 M · remote cursor read: 1 per N instead of 1 per op

63. What the standard libraries give you

Before writing any of Part Eight yourself, know what already exists and what its guarantees are. Hand-rolled concurrent queues are the single largest source of heisenbugs in application code.

Facility	Blocking	Bounded	Notes
C++ <code>std::queue</code> + mutex	You write it	You write it	Nothing is provided; Topic 59 is the code
Go <code>chan T</code> with size <code>n</code>	Yes	Yes	Bounded by construction; the model to imitate
Java <code>ArrayBlockingQueue</code>	Yes	Yes	One lock, two conditions — Topic 59 exactly
Java <code>LinkedBlockingQueue</code>	Yes	Optional — unbounded by default	The default is the Topic 59 trap
Java <code>ConcurrentLinkedQueue</code>	No	No	Michael-Scott lock-free MPMC; <code>size()</code> is $O(n)$
Python <code>queue.Queue</code>	Yes	Optional (<code>maxsize</code>)	Wraps <code>deque</code> + condition; GIL-bound
Python <code>deque</code>	No	<code>maxlen</code> drops silently	<code>append/popleft</code> atomic under the GIL

```
// The work-stealing deque (Chase-Lev) -- why a DEQUE and not a queue:
//  owner thread: push_bottom / pop_bottom  (LIFO: hot, cache-warm tasks)
//  thief threads: steal_top                (FIFO: oldest, largest tasks)
// One end is uncontended and used by one thread; the other is contended by
// CAS among thieves. This is the scheduler inside TBB, Rayon, Go's runtime,
// and the JVM's ForkJoinPool.
// Stealing the OLDEST task is deliberate: in a divide-and-conquer tree the
// oldest task is the largest subtree, so one steal moves the most work.
```

THE KEY INSIGHT

The work-stealing deque is the clearest illustration in this book of why the deque exists: two ends with different access patterns and different contention profiles. Owner uses LIFO for locality; thieves use FIFO to grab the biggest remaining chunk.

THE TRAP

`LinkedBlockingQueue` constructed without a capacity argument, and `ThreadPoolExecutor` configured with it. The pool then never rejects, never grows past the core size, and queues work indefinitely. Symptom: a thread pool that appears healthy, a queue depth metric nobody exported, and an OOM under a downstream slowdown. Always pass a capacity.

library queues – prefer bounded + blocking · MPMC lock-free: `size()` is $O(n)$ and immediately stale
· work-stealing: LIFO owner, FIFO thief

PRACTICE SET — Topics 59-63

A. Conceptual

1. Why does a bounded blocking queue need two condition variables rather than one?
2. What breaks at shutdown if the `closed_` flag is not part of both wait predicates?
3. Distinguish a spurious wakeup from a lost wakeup, and give the mechanism that prevents each.
4. Why is `if (q_.empty()) cv.wait(lk);` wrong even on a system that never produces spurious wakeups?
5. Explain why holding the mutex across the check and the `wait` call is what prevents the lost wakeup.
6. In the SPSC ring, which thread writes `head_` and which writes `tail_`, and why does that make locks unnecessary?
7. What exactly does the release store on `tail_` guarantee, and what does the matching acquire load provide?
8. Why do memory-ordering bugs in an SPSC ring often appear only on ARM?
9. Define false sharing and true sharing, and say which one padding fixes.
10. Why can padding alone make an SPSC ring slower?
11. What does caching the remote cursor change about the frequency of cross-core reads?
12. In a work-stealing deque, why does the owner take from the bottom and thieves from the top?
13. Why is `size()` on a lock-free MPMC queue both $O(n)$ and useless?

B. Tracing and analysis

1. A blocking queue does 10^7 operations, of which 2 % cause a thread to sleep. At 22 ns uncontended and 3 μ s per sleep/wake, compute the total time and the fraction spent in the kernel.
2. An SPSC ring with $N=8$ has `head_=100`, `tail_=105`. Give size, the slot the next push writes, and the slot the next pop reads.
3. Same ring: the producer pushes 3 more. Give `tail_`, whether the ring is full, and the physical slots occupied.
4. With `cached_head_ = 96` and `tail_ = 104`, $N = 8$, does the producer refresh `head_`? Show the arithmetic.
5. For $N = 1024$ and a producer that never gets ahead, estimate how many `head_` loads occur per 10^6 pushes, with and without cursor caching.
6. Given 32 M, 21 M and 440 M ops/s for the three SPSC variants, compute ns per operation for each and the speedup of the best over the naive.
7. A `ThreadPoolExecutor` with an unbounded queue receives 5000 req/s while the pool processes 3000 req/s for 90 seconds. At 4 KB retained per queued request, compute the queue depth and the memory at the end.

C. Code tasks — build it, break it, observe, fix

1. Implement `BlockingQueue` with close support. Run 4 producers and 4 consumers moving 10^6 items and assert every item is received exactly once.
2. **Break it:** replace the predicate `wait` with `if (q_.empty()) not_empty_.wait(lk);`. Run with 4 consumers and 1 producer at a high rate. Record how long until it crashes or produces a

wrong value, and the exact failure. Then re-run under ThreadSanitizer and note whether it reports anything.

3. **Break it:** remove the `closed_` check from the pop predicate. Start consumers, call `close()`, then `join()`. Observe the hang, attach `gdb` (or use `timeout` and `SIGQUIT`), and record the stack of the stuck thread.
4. Implement the SPSC ring with correct acquire/release. Verify with 10^8 items that the consumer's checksum equals the producer's.
5. **Break it:** change both `memory_order_release` and `memory_order_acquire` to `memory_order_relaxed`. Run on x86 and record whether it fails. If you have access to ARM (an Apple Silicon Mac or an ARM cloud instance), run it there and record the corruption rate.
6. **Break it:** run the SPSC ring with *two* producer threads. Count items pushed and items received over 10^7 operations, and record the discrepancy.
7. Benchmark all three SPSC variants: shared line, padded, padded + cached cursors. Pin the two threads to separate physical cores. Report ops/s for each.
8. Use `perf stat -e cache-misses,cache-references` on the naive and the cached-cursor version and report misses per operation for both.

ANSWER KEY — Topics 59-63

A. Conceptual

1. Because the two waiting conditions are different: producers wait for *not full*, consumers for *not empty*. With one CV, every notify wakes both groups, and the wrong group re-checks its predicate and sleeps again — a **thundering herd** of useless wakeups.
2. Consumers blocked in `wait` never return: the predicate remains false forever because no producer will ever push again. **The process hangs on `join()`.**
3. *Spurious*: `wait` returns with the predicate false, which the standard explicitly permits — prevented by **re-checking in a loop**. *Lost*: a notify is issued when no thread is registered as waiting and is discarded — prevented by **holding the mutex across the predicate check and the wait**.
4. Because of **stolen wakeups**: two consumers can be woken, and the first takes the only element. The second then proceeds with an empty queue. Nothing spurious is required.
5. Because `wait` releases the mutex only *after* registering the thread as a waiter. A producer cannot acquire the mutex, push, and notify in the window between the consumer's check and its registration — **the gap does not exist**.
6. The consumer writes `head_`; the producer writes `tail_`. **Each index has exactly one writer**, so there is no read-modify-write to serialise and therefore nothing to mutually exclude.
7. The release store guarantees that **everything the producer wrote before it (the payload) is visible to any thread that performs a matching acquire load of the same variable**. The acquire load guarantees the consumer does not see the new `tail_` without also seeing the payload.
8. Because **x86 is strongly ordered**: acquire and release compile to plain loads and stores that the hardware already orders. ARM is weakly ordered and requires `LDAR/STLR`, so an incorrect `relaxed` actually reorders there.
9. *False sharing*: two variables that are logically independent share a cache line, so writing one invalidates the other. *True sharing*: both threads genuinely access the same variable. **Padding fixes only false sharing**.
10. Because both threads still read the other's cursor on every operation. After padding there are **two lines ping-ponging instead of one**, which can be net slower — a reported measurement showed roughly a third slower.
11. The remote cursor is read only when the local view suggests full (or empty) — roughly **once per N operations instead of once per operation**. At $N = 1024$ that is a $1000\times$ reduction in cross-core traffic.
12. The owner takes from the bottom for **cache locality** (the most recently pushed task is hot and its data is in L1) and because that end is uncontended. Thieves take from the top because in divide-and-conquer the oldest task is the **largest remaining subtree**, so one steal transfers the most work.
13. $O(n)$ because it must walk the linked structure; useless because the count is **stale before it is returned** — other threads mutate the queue during the walk.

B. Tracing and analysis

1. Fast path: $0.98 \times 10^7 \times 22 \text{ ns} = \mathbf{0.216 \text{ s}}$. Sleeps: $0.02 \times 10^7 \times 3000 \text{ ns} = \mathbf{0.600 \text{ s}}$. Total **0.816 s**, of which **73.5 %** is kernel wake time — from 2 % of the operations.
2. $\text{size} = 105 - 100 = \mathbf{5}$. Next push writes slot 105 & $7 = \mathbf{1}$; next pop reads slot 100 & $7 = \mathbf{4}$.

3. `tail_ = 108`; size = 8 = N → **full**. Occupied slots in logical order: 4, 5, 6, 7, 0, 1, 2, 3.
4. $104 - 96 = 8 = N$ → the producer believes it is full, so **yes, it refreshes** `head_`. If the real head has advanced past 96, the refresh reveals space and the push proceeds.
5. Without caching: **10^6 loads** of `head_`. With caching and a producer never ahead: the "maybe full" branch is essentially never taken, so **near zero** — in the worst case one refresh per 1024 pushes = **977 loads**.
6. 32 M → **31.3 ns/op**; 21 M → **47.6 ns/op**; 440 M → **2.27 ns/op**. Speedup of best over naive: **13.8×**; padding alone is **0.66×** (a 34 % slowdown).
7. Accumulation $2000/s \times 90\text{ s} =$ **180 000 queued requests**; $\times 4\text{ KB} =$ **720 MB**. With a default heap smaller than that, the OOM arrives before the incident is diagnosed.

C. Code tasks — what to verify

1. Total received must equal total sent, with no duplicates — check with a per-item counter array, not just a sum, so a swapped pair is caught. `close()` then `join()` must complete in **under a second**.
2. Expect a failure within **seconds**: typically a segfault or a garbage value from `front()` on an empty deque. **ThreadSanitizer reports nothing** — there is no data race, the mutex is held; only the predicate is missing. Record that explicitly: it is the reason this bug survives tooling.
3. The process hangs. The stuck thread's stack shows `pthread_cond_wait` → `std::condition_variable::wait` → your `pop`. `timeout 10 ./a.out` returns exit code **124**.
4. Checksums must match exactly across 10^8 items. Use a sum plus an XOR so that a reordering is also detected, not only a loss.
5. On x86 expect **no observable failure** — the hardware ordering hides it. On ARM expect occasional corrupted values, typically on the order of **one in 10^5 - 10^7 items**, and more often under high contention. The x86 result is the lesson: correct-looking is not correct.
6. Expect **received < pushed**, with a loss on the order of **0.1-1 %** of items and no crash at all. Also expect some slots to contain the same value twice.
7. Expect roughly **32 M / 21 M / 440 M ops/s**. Pin with `taskset -c` and confirm the two cores are separate physical cores — two hyperthreads on one core share L1 and will not show the effect.
8. Expect the naive version at **1-2 cache misses per operation** (the line bounces every time) and the cached-cursor version at **well under 0.01**. That ratio is the whole explanation of the 13.8×

HANDNOTE SUMMARY CARD

PART EIGHT - CONCURRENCY

topics 59-63

BOUNDED BLOCKING QUEUE - the shape to memorise

```
push: unique_lock lk(m_);
      not_full_.wait(lk, []{ return q_.size() < cap_ || closed_; });
      q_.push_back(x); lk.unlock(); not_empty_.notify_one();
pop:  unique_lock lk(m_);
      not_empty_.wait(lk, []{ return !q_.empty() || closed_; });
      v = q_.front(); q_.pop_front(); lk.unlock(); not_full_.notify_one();
close(): set closed_ under the lock, then notify_ALL on BOTH cvs
TWO condition variables (one -> thundering herd)
closed_ in BOTH predicates (else join() hangs forever at shutdown)
cost: ~20-25 ns uncontended, 1-5 us per actual sleep/wake
```

wait() RULES

ALWAYS the predicate form. `wait(lk, pred) == while(!pred()) wait(lk);`
spurious wakeup = wait returns with pred false. LEGAL. not a bug to fix.
stolen wakeup = two woken, first takes the only item. needs no spuriousness.
lost wakeup = notify with no registered waiter -> DISCARDED FOREVER.
prevented by holding the mutex across check + wait.
a condition variable HAS NO STATE. It is not a semaphore. It does not count.
`if` instead of `while` -> TSan reports NOTHING (no race; just no re-check)

SPSC LOCK-FREE RING exactly ONE producer, ONE consumer

```
push: t = tail_.load(relaxed); h = head_.load(ACQUIRE);
      if (t - h == N) return false;
      buf_[t & (N-1)] = x; // write payload
      tail_.store(t+1, RELEASE); // then publish
pop:  h = head_.load(relaxed); t = tail_.load(ACQUIRE);
      if (h == t) return false;
      out = buf_[h & (N-1)]; head_.store(h+1, RELEASE);
correctness comes from OWNERSHIP: one writer per index. Ordering only makes
the payload visible before the index advertising it.
x86: acquire/release are plain MOV -> ordering bugs INVISIBLE.
ARM: LDAR/STLR -> real barriers. Test on ARM or you have not tested.
two producers -> silent element loss, ~0.1-1%, never a crash.
```

SHARING capacity 1024, 200 M uint32, 2 cores

shared cache line 32 M ops/s 31.3 ns both cursors, one line
alignas(64) padding only . 21 M ops/s 47.6 ns SLOWER: 2 lines bouncing
padding + cached cursors . 440 M ops/s 2.27 ns 13.8x
cached cursor: read the remote index only when LOCALLY full/empty
-> ~1 cross-core read per N ops instead of per op
RULE: kill TRUE sharing first. padding is not a magic word. measure.

LIBRARY LANDSCAPE

Go chan(n)	blocking + BOUNDED	<- the model
Java ArrayBlockingQueue	blocking + bounded	= Topic 59
Java LinkedBlockingQueue	UNBOUNDED BY DEFAULT	<- the trap
Java ConcurrentLinkedQueue	lock-free MPMC, size() O(n) and stale	
Python queue.Queue	blocking, maxsize optional	
Python deque	append/popleft atomic (GIL); maxlen drops silently	
Chase-Lev work-stealing DEQUE:	owner push/pop BOTTOM (LIFO, cache-hot),	
thieves steal TOP (FIFO, oldest = biggest subtree).	TBB, Rayon, ForkJoinPool.	

PART NINE

Capacity and queueing theory

The arithmetic that decides how deep the queue is allowed to be.

Part Nine · Capacity and queueing theory

64. Arrival rate, service rate, utilisation, Little's Law

Four symbols and one equation govern every queue in production. They are worth memorising because they turn "the queue seems deep" into a number you can put in a capacity plan.

```
// lambda arrival rate      (requests per second)
// mu     service rate      (requests per second, one server)
// rho     utilisation       = lambda / (c * mu), c = number of servers
// L       items in system   W = time in system (wait + service)
//
// LITTLE'S LAW:  L = lambda * W
//
// It holds for ANY arrival distribution, ANY service distribution, ANY
// queue discipline, at steady state. No assumptions. That generality is
// why it is the one formula to remember.
//
// WORKED: a worker pool processes 1200 req/s. Monitoring says the queue
// holds 300 items on average. What is the added latency?
// W_queue = L / lambda = 300 / 1200 = 0.25 s = 250 ms
// Every request waits a quarter second before anyone looks at it -- and no
// amount of faster processing per request changes that until mu rises.
//
// REVERSED: you need queue wait under 50 ms at 1200 req/s.
// L = lambda * W = 1200 * 0.05 = 60 items. Set capacity to 60.
// That is where a bounded queue's capacity number comes from. Not 10000
// "to be safe" -- 10000 items at 1200/s is 8.3 seconds of latency.
```

the system, and what each symbol measures

```
lambda                      mu (per server)
arrivals -----> [ queue: Lq ] --> [ server ] ----> departures
                  |<-- Wq -->|      |<-- 1/mu -->|
                  |<----- W = Wq + 1/mu ----->|
                  |<----- L = Lq + rho ----->|
```

Little's Law applies to ANY box you draw: the queue alone ($L_q = \lambda W_q$), the server alone ($\rho = \lambda / \mu$), or the whole system ($L = \lambda W$).

THE KEY INSIGHT

Queue capacity is a latency budget, not a memory budget. Choose it as $\lambda \times W_{\max}$ — the number of items that fit inside your acceptable wait — and everything beyond it should be rejected, because those requests will breach the SLA anyway (Topic 66).

THE TRAP

Applying Little's Law to a system that is not in steady state. During a burst, L is growing and $L = \lambda W$ describes neither the current nor the final state. Symptom: a capacity calculation from peak-minute metrics that under-provisions by several times. Average over an interval long enough that arrivals and departures balance.

Little's Law — $L = \lambda W$, distribution-free · capacity = $\lambda \times W_{\max}$ · 300 items at 1200/s = 250 ms of pure queueing

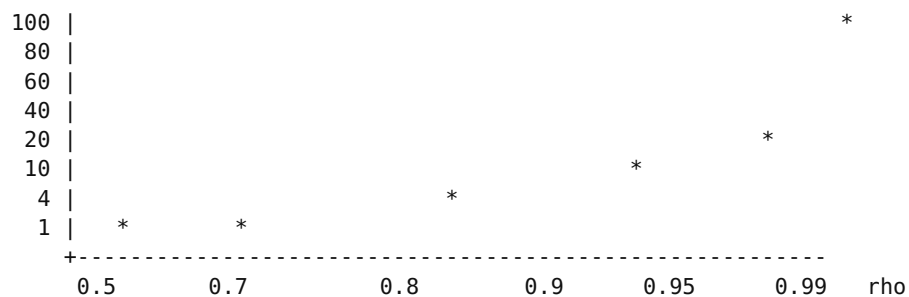
65. Why waiting time explodes near full utilisation

Utilisation and latency are not linearly related. For an M/M/1 queue — random arrivals, random service, one server — the average wait is proportional to $\rho/(1-\rho)$, and that denominator is the whole story.

```
// M/M/1: Lq = rho^2 / (1 - rho)      Wq = rho / (mu * (1 - rho))
// With mu = 1000 req/s (1 ms of service per request):

//   rho    lambda    Lq (queued)    Wq (wait)    total W
//   0.50    500      0.50          1.0 ms      2.0 ms
//   0.70    700      1.63          2.3 ms      3.3 ms
//   0.80    800      3.20          4.0 ms      5.0 ms
//   0.90    900      8.10          9.0 ms     10.0 ms
//   0.95    950     18.05         19.0 ms     20.0 ms
//   0.99    990     98.01         99.0 ms    100.0 ms
//   0.999   999    998.0         999.0 ms   1000.0 ms
//
// Going from 50% to 90% utilisation costs 9x the wait for 1.8x the work.
// Going from 90% to 99% costs another 11x for 1.1x the work.
// This is why capacity planners target 60-70% and not 95%.
```

Wq (ms), mu = 1000/s



The knee is not at 0.9 or 0.95 -- the curve has no knee. It is a hyperbola: every halving of $(1 - \rho)$ doubles the wait, forever.

THE KEY INSIGHT

Variability is what creates the queue. If arrivals were perfectly spaced and service perfectly constant (a D/D/1 queue), the wait would be zero at any $\rho < 1$. The $1/(1-\rho)$ blow-up is entirely the price of randomness — which is why reducing variance (batching, smoothing, evening out slow requests) helps as much as adding capacity.

THE TRAP

Reading a dashboard showing 85 % CPU and concluding there is 15 % headroom. At $\rho = 0.85$ the wait is already 5.7 ms; a 10 % traffic increase takes ρ to 0.94 and the wait to 15.7 ms — a **2.8x** latency increase from a 10 % load increase. Symptom: a service that is fine every day and falls over on the one day traffic rises by a fifth.

M/M/1 — $Wq = \rho/(\mu(1-\rho))$ · $\rho=0.5 \rightarrow 1$ ms, $0.9 \rightarrow 9$ ms, $0.99 \rightarrow 99$ ms · no knee: it is a hyperbola · variance is the cause

66. Bounded queues, backpressure and load shedding

The conclusion of Parts Eight and Nine: an unbounded queue does not add capacity, it converts a fast, honest failure into a slow one that also destroys the process. Bounding it gives you a choice about how to fail.

```
// The three responses when a bounded queue is full, in order of preference:
//
// 1. BACKPRESSURE -- block the producer, or refuse to read from the socket.
//    The slowdown propagates upstream to whoever can actually slow down.
//    Go: an unbuffered or small buffered channel does this by construction.
//    Java: ThreadPoolExecutor.CallerRunsPolicy runs the task on the
//          submitting thread -- the caller stops accepting new work.
//
// 2. LOAD SHED -- reject immediately with 503 + Retry-After.
//    A 5 ms rejection is a better answer than a 30 s timeout, for both
//    sides: the client can fail over, and your queue stays shallow.
//
// 3. DROP THE OLDEST -- for telemetry, frames, metrics. A 200 ms old
//    sample is worthless; keeping it and rejecting the fresh one maximises
//    staleness (Topic 5).
//
// And the deadline check that costs nothing and saves everything:
struct Job { std::chrono::steady_clock::time_point deadline; /* ... */ };
    auto job = q.pop();
    if (std::chrono::steady_clock::now() > job.deadline) { ++expired; continue; }
// Under overload, most queued work has ALREADY timed out client-side.
// Processing it burns the capacity needed to recover. This one line is the
// difference between a 90-second incident and a 40-minute one.
```

latency death spiral -- why unbounded queues never recover:

```
downstream slows -> queue grows -> latency grows -> clients time out
      ^                                     |
      |                                     v
capacity spent on  <- retries arrive <- clients retry --+
dead work          (2-3x the original load)
```

The retry storm is the killer: the queue now holds work nobody is waiting for, and every item processed is wasted. Bounding + shedding + deadline checks each break one arrow of this cycle.

THE KEY INSIGHT

The first law of queues: every queue must have a maximum size. The second: ask whether that maximum should be zero — a queue of size 0 is a direct handoff, and a surprising number of queues exist only because nobody chose. If average arrival exceeds average service (Topic 2), no capacity helps; the queue merely schedules the failure.

THE TRAP

Bounding the queue and then *silently* dropping on overflow with no metric. You have converted an OOM into data loss that nobody can see. Symptom: missing records that correlate with traffic peaks and no error anywhere in the logs. Always export a rejection counter, and alert on it — the counter going non-zero is the earliest possible warning of the spiral above.

bounded queue – capacity = $\lambda \cdot W_{\max}$ · shed at 5 ms vs time out at 30 s · deadline check drops dead work · always count rejections

PRACTICE SET — Topics 64-66

A. Conceptual

1. State Little's Law and the assumptions it requires. Which distributions must you know?
2. Why is queue capacity better derived from a latency budget than from available memory?
3. Why does Little's Law give misleading numbers during a traffic burst?
4. What in the M/M/1 formula produces the explosion near $\rho = 1$, and what would happen with perfectly regular arrivals and service?
5. Explain why 85 % utilisation is not "15 % headroom".
6. Give the three responses to a full bounded queue, and one workload that suits each.
7. Why is a 5 ms rejection better for the *client* than a 30-second timeout?
8. What does the deadline check at dequeue time accomplish that a shorter queue does not?
9. Describe the retry storm feedback loop and name the arrow each of bounding, shedding, and deadline-checking breaks.
10. Why is a silently dropping bounded queue arguably worse than an unbounded one?

B. Tracing and analysis

1. A service handles 2500 req/s; average queue depth is 175. Compute the average queueing delay.
2. You require queueing delay under 20 ms at 2500 req/s. Give the queue capacity.
3. With $\mu = 500$ req/s, compute W_q at $\rho = 0.6, 0.8, 0.9, 0.95$, and the ratio of the last to the first.
4. A system runs at $\rho = 0.80$ and traffic rises 15 %. Give the new ρ and the factor by which W_q increases.
5. Four servers, $\mu = 300$ req/s each, $\lambda = 1000$ req/s. Give ρ . How much headroom in req/s before $\rho = 0.95$?
6. A queue is bounded at 60 items with $\lambda = 1200/s$ and $\mu = 1000/s$ sustained. How long until it is permanently full, and what is the steady-state rejection rate?
7. Jobs carry a 2-second deadline. The queue holds 8000 items and drains at 1000/s. What fraction of queued work is already dead on arrival at the worker?
8. A retry storm triples effective load on a service at $\rho = 0.85$. Compute the new ρ and describe what the M/M/1 model says happens.

C. Lab tasks — build it, break it, observe, fix

1. Build a simulator: a Poisson arrival generator, an exponential service time, one worker, and an unbounded queue. Run at $\rho = 0.5, 0.8, 0.9, 0.95$ for 10^6 requests each and plot the measured mean wait against the M/M/1 prediction.
2. Verify Little's Law empirically in your simulator: measure L by sampling depth every millisecond and W per request, then compare L against λW .
3. **Break it:** run the simulator at $\rho = 1.05$ with an unbounded queue for 60 simulated seconds. Plot depth and memory over time. Record the final depth and the extrapolated time to exhaust 8 GB at 4 KB per item.
4. **Break it:** repeat with a queue bounded at 60 and a rejection counter. Record the rejection rate and the p99 latency of *accepted* requests. Compare the p99 with the unbounded run.

5. **Break it:** add client retries — every request that waits longer than 2 seconds is abandoned and resubmitted once. Run at $\rho = 0.9$ and record whether the system recovers when the arrival rate returns to normal, with and without a deadline check at dequeue.
6. Implement `CallerRunsPolicy`-style backpressure in your simulator (the producer performs the work itself when the queue is full) and compare the achieved throughput and latency distribution with the load-shedding variant.

ANSWER KEY — Topics 64-66

A. Conceptual

1. $L = \lambda W$. It requires only **steady state** — arrivals balancing departures over the measurement interval. **No distributions need be known**; it holds for any arrival process, service process and queue discipline.
2. Because the queue's depth is **latency**: capacity $\lambda \cdot W_{\max}$ is exactly the number of items that fit inside an acceptable wait. Sizing by memory produces a queue whose full state is an SLA breach that has not been reported yet.
3. Because during a burst **L is still growing** — departures do not balance arrivals — so the measured L and the measured W belong to different moments and their ratio is not λ .
4. The **(1- ρ) denominator**. With deterministic arrivals and deterministic service (D/D/1) the wait is **zero for any $\rho < 1$** : the queue exists only because randomness sometimes clusters arrivals faster than the server clears them.
5. Because latency depends on $1/(1-\rho)$, not on ρ . At 0.85, Wq is already **5.7 ms**; a 10 % traffic rise gives $\rho = 0.94$ and Wq = **15.7 ms** — a 2.8× increase for a 10 % change.
6. **Backpressure** (block/refuse to read) — internal pipelines where the producer can genuinely slow down. **Load shed** (503) — public request paths with retrying clients. **Drop oldest** — telemetry, video frames, metrics, where freshness is the value.
7. Because the client learns **immediately** that it must fail over, retry elsewhere, or degrade — while a 30-second timeout consumes the client's own connection, thread, and patience, and often triggers its own cascading failure.
8. It removes work that is **already useless**. A short queue limits how much dead work can accumulate; the deadline check ensures none of the work that is in the queue is processed after its consumer has given up — it recovers capacity during the incident, not before it.
9. Loop: slow downstream → queue grows → latency grows → clients time out → clients retry → load rises → slower still. **Bounding** breaks "queue grows"; **shedding** breaks "latency grows" (failures are fast); **deadline checking** breaks "capacity spent on dead work".
10. Because an OOM is **loud** and gets investigated; silent drops are **invisible data loss** correlated with peak traffic, discovered weeks later during reconciliation. Bounding is right; bounding without a counter is not.

B. Tracing and analysis

1. $W = L/\lambda = 175 / 2500 = \mathbf{0.07\ s = 70\ ms}$.
2. $L = \lambda W = 2500 \times 0.020 = \mathbf{50\ items}$.
3. $Wq = \rho/(\mu(1-\rho))$, $\mu = 500 \rightarrow 1/\mu = 2\ ms$. $\rho=0.6$: $0.6/(500 \times 0.4) = \mathbf{3.0\ ms}$. 0.8 : $0.8/(500 \times 0.2) = \mathbf{8.0\ ms}$. 0.9 : $0.9/(500 \times 0.1) = \mathbf{18.0\ ms}$. 0.95 : $0.95/(500 \times 0.05) = \mathbf{38.0\ ms}$. Ratio last/first = **12.7×**.
4. New $\rho = 0.80 \times 1.15 = \mathbf{0.92}$. Wq scales as $\rho/(1-\rho)$: from $0.8/0.2 = 4$ to $0.92/0.08 = 11.5 \rightarrow \mathbf{2.88\times}$.
5. $\rho = 1000 / (4 \times 300) = 1000/1200 = \mathbf{0.833}$. At $\rho = 0.95$, $\lambda = 0.95 \times 1200 = 1140 \rightarrow$ headroom = **140 req/s** (14 %).
6. Net accumulation 200/s → 60 items / 200 per s = **0.30 s** to fill. Steady-state rejection = $\lambda - \mu = \mathbf{200\ req/s = 16.7\ \%}$ of arrivals.
7. Drain time for 8000 items = 8 s. Items whose wait exceeds the 2 s deadline are those beyond position 2000 → **6000 of 8000 = 75 %** are already dead.

8. $\rho = 0.85 \times 3 = 2.55 > 1$. The model has **no steady state**: the queue grows without bound and W_q is undefined (infinite). Recovery requires shedding load, not waiting.

C. Lab tasks — what to verify

1. Measured means should land within a few percent of 1, 4, 9 and 19 ms ($\mu = 1000/\text{s}$). If $\rho = 0.95$ does not converge, your run is too short — the higher ρ is, the longer the warm-up; discard the first 10 % of samples.
2. L (time-averaged depth) and λW should agree to within **1-2 %**. A systematic gap usually means you sampled depth at fixed request boundaries rather than fixed time intervals — Little's Law needs the *time* average.
3. Depth should rise **linearly** at $\approx(\lambda - \mu)$ per second, with no plateau. At 4 KB per item and a 50 req/s deficit, 8 GB is 2×10^6 items = **≈ 11 hours**; at a 500/s deficit, **≈ 67 minutes**. Report yours.
4. Rejection rate should approach $(\lambda - \mu)/\lambda$. The p99 of accepted requests should be **bounded near capacity/ μ** ($60/1000 = 60$ ms), versus an unbounded p99 that **risers without limit**. That contrast is the entire argument for bounding.
5. Without the deadline check, expect the system to **stay saturated well after arrivals normalise** — it is processing abandoned work while new retries arrive. With the check, recovery should be roughly the time to drain the live portion. Record both recovery times; the ratio is typically **5-20 $\times$** .
6. Backpressure should show **higher achieved throughput and a tighter latency distribution** but a slower producer; shedding shows constant producer speed with a rejection tail. Neither dominates — state which one your workload's client can actually handle.

HANDNOTE SUMMARY CARD

PART NINE - CAPACITY AND QUEUEING THEORY

topics 64-66

THE SYMBOLS

λ arrivals/s μ service rate/s per server c servers
 $\rho = \lambda / (c * \mu)$ L items in system W time in system
 $L_q / W_q = \text{the QUEUE only}$ $W = W_q + 1/\mu$ $L = L_q + \rho$

LITTLE'S LAW $L = \lambda * W$

holds for ANY arrival distribution, ANY service distribution, ANY discipline.
needs ONLY steady state. Apply it to any box you draw.
depth 300 at 1200/s -> $W = 300/1200 = 250$ ms of pure queueing
want $W_q \leq 50$ ms at 1200/s -> capacity = $1200 * 0.05 = 60$ items
*** QUEUE CAPACITY IS A LATENCY BUDGET, NOT A MEMORY BUDGET ***
invalid during a burst: L is still growing, so L and W are from different moments.

M/M/1 $L_q = \rho^2 / (1 - \rho)$ $W_q = \rho / (\mu * (1 - \rho))$

$\mu = 1000/s$ (1 ms service):

ρ	0.50	0.70	0.80	0.90	0.95	0.99	0.999
W_q	1.0	2.3	4.0	9.0	19.0	99.0	999.0

ms

0.5 -> 0.9 : 1.8x the work, 9x the wait

0.9 -> 0.99: 1.1x the work, 11x the wait

NO KNEE - it is a hyperbola. every halving of $(1 - \rho)$ doubles the wait.

85% CPU is NOT 15% headroom: +10% traffic -> $\rho 0.94$ -> 2.8x latency

VARIANCE causes the queue. D/D/1 (perfectly regular) waits ZERO at any $\rho < 1$.

-> smoothing and batching buy as much as capacity does.

WHEN THE BOUNDED QUEUE IS FULL (in order of preference)

- 1 BACKPRESSURE block the producer / stop reading the socket
Go: small buffered chan. Java: CallerRunsPolicy.
 - 2 LOAD SHED 503 + Retry-After. a 5 ms rejection beats a 30 s timeout.
 - 3 DROP OLDEST telemetry, frames, metrics -- staleness is the enemy
- ALWAYS export a rejection COUNTER. silent drops = invisible data loss.

THE DEADLINE CHECK - one line, biggest win in an incident

```
auto job = q.pop();  
if (now() > job.deadline) { ++expired; continue; }  
under overload MOST queued work has already timed out client-side.  
8000 deep draining at 1000/s with a 2 s deadline -> 75% of it is dead.
```

LATENCY DEATH SPIRAL

downstream slows -> queue grows -> latency grows -> clients time out
-> clients RETRY (2-3x load) -> capacity spent on dead work -> slower
bounding breaks "queue grows"; shedding breaks "latency grows";
deadline check breaks "capacity spent on dead work". Use all three.

THE TWO LAWS OF QUEUES

1. every queue must have a maximum size.
2. consider whether that maximum should be ZERO (a direct handoff).
and: if $\lambda_{avg} > \mu$, NO capacity helps. the queue only schedules the failure (Topic 2).

PART TEN

Queues in real systems

Where the structure actually lives, from the NIC ring to the job worker.

Part Ten · Queues in real systems

67. Run queues and round-robin scheduling

The operating system's scheduler is a queue problem, and Linux's answer — CFS, now EEVDF — is a red-black tree ordered by accumulated virtual runtime rather than a plain FIFO. Knowing why it moved away from a queue explains what a queue cannot do here.

```
$ cat /proc/schedstat | head -3
version 15
timestamp 4294896396
cpu0 0 0 0 0 0 0 1042117 1123449 6412      # ... running_time waiting_time
                                           # ^ nanoseconds
                                           # tasks spent
                                           # on the run
                                           # queue

$ cat /proc/pressure/cpu
some avg10=12.44 avg60=8.31 avg300=3.02 total=8871234000
# ^ percent of wall time SOME task was runnable but waiting for CPU
# this is queue depth expressed as a fraction of time. > 20 sustained
# means you are CPU-queued, not CPU-busy.

// Plain round-robin (Topic 58) gives each task a fixed quantum:
// quantum 1 ms -> 1000 switches/s/core, ~1-3 us each -> 0.1-0.3% overhead
// quantum 100 ms -> 10 switches/s, but a task can wait 100 ms * (n-1)
// CFS instead targets a total latency (sched_latency_ns, default 24 ms) and
// divides it among runnable tasks, so the quantum SHRINKS as n grows --
// with a floor (sched_min_granularity_ns) to stop switch costs dominating.
```

FIFO run queue	CFS / EEVDF
[A B C D]	red-black tree keyed by vruntime
^next	(B, 12ms)
	/ \
simple, O(1), but a task	(A, 9ms) (C, 15ms)
that just ran goes to the	^leftmost = next to run
back regardless of how much	
CPU it has had in total	picks the task with the LEAST accumulated
	weighted runtime: fairness over TIME,
	not over TURNS

THE KEY INSIGHT

A FIFO run queue gives fairness in *turns*; a scheduler needs fairness in *CPU time*. Once tasks have different priorities and different burst lengths, the ordering key stops being arrival time and the structure stops being a queue — the same argument as Topic 32, made by the kernel.

THE TRAP

Reading high CPU utilisation as "we need faster CPUs" when `/proc/pressure/cpu` shows high `some`. That is queueing delay, and Part Nine says it responds to reducing ρ far more than to making each task faster. Symptom: an upgrade to faster cores that barely moves p99 latency while the run-queue wait stays flat.

scheduling – context switch 1–3 μ s · CFS target latency 24 ms divided among runnable tasks · PSI
`some` = queue depth as % of time

68. NIC ring buffers and where drops are counted

The network card and the kernel share a literal circular buffer of descriptors — Topic 9's ring, in hardware. When the kernel does not drain it fast enough, packets are dropped by the card and counted in a place most people never look.

```
$ ethtool -g eth0
Ring parameters for eth0:
Pre-set maximums:
RX: 4096      TX: 4096
Current hardware settings:
RX: 256      TX: 256      # only 256 descriptors! at 1.5 KB/packet
                        # that is ~384 KB of buffering

$ ethtool -S eth0 | grep -E 'drop|miss|discard'
  rx_dropped: 0
  rx_missed_errors: 148213    # <-- card had a packet, ring was FULL
  rx_no_buffer_count: 148213  # this is the queue overflowing

$ netstat -s | grep -i 'listen'
 4821 times the listen queue of a socket overflowed    # a DIFFERENT queue
 4821 SYNs to LISTEN sockets ignored

// There are at least FOUR queues between the wire and your accept() loop:
//  1. NIC RX ring (ethtool -g / -S)          hardware, descriptors
//  2. per-CPU backlog (netdev_max_backlog)    softirq, /proc/net/softnet_stat
//  3. SYN queue (tcp_max_syn_backlog)        half-open connections
//  4. accept queue (listen backlog, somaxconn) fully established, unaccepted
// A "dropped connection" can come from any of them, and each has its own
// counter. Finding WHICH queue overflowed is most of the debugging.
```

```
wire -> [NIC RX ring] -> [softirq backlog] -> [SYN q] -> [accept q] -> app
        256 descr      1000 default      1024      somaxconn 4096
        ethtool -S      softnet_stat      netstat -s  ss -lnt (Recv-Q)

$ ss -lnt
State Recv-Q Send-Q Local Address:Port
LISTEN 129    128      0.0.0.0:8080
      ^^^ pending connections  ^^^ the backlog LIMIT
Recv-Q at or above Send-Q on a LISTEN socket = your accept queue is full
and the kernel is dropping or refusing connections RIGHT NOW.
```

THE KEY INSIGHT

On a listening socket `ss -lnt` reports Recv-Q as the current accept backlog and Send-Q as its maximum — a queue depth and capacity you can read in one command. It is the cheapest overload signal available and almost nobody graphs it.

THE TRAP

Enlarging the NIC ring to "stop the drops". A bigger ring buffers more and drops later; if the CPU cannot keep up, you have added latency and moved the drops downstream (bufferbloat, Topic 69). Symptom: `rx_missed_errors` falls, p99 latency rises, and total throughput is unchanged. The ring size fixes *bursts*, not a rate deficit — the same rule as Topic 2.

```
NIC ring – 256–4096 descriptors · 4 queues wire→app, each with its own counter · ss -lnt: Recv-Q = depth, Send-Q = limit
```

69. TCP buffers and bufferbloat

Every TCP connection has a send queue and a receive queue, and every device on the path has its own. Oversized buffers do not lose packets — they hold them, which delays the congestion signal and destroys latency for everyone sharing the link.

```
$ ss -tin dst 10.0.0.5
State Recv-Q Send-Q Local:Port Peer:Port
ESTAB      0 128480 10.0.0.4:44210 10.0.0.5:443
           cubic wscale:7,7 rto:212 rtt:11.4/2.1 mss:1448 cwnd:64
           bytes_sent:9812340 retrans:0/14 rcv_space:14480
// Send-Q 128480 = 128 KB queued locally and not yet acknowledged
// at 10 Mbit/s that is 128 480 * 8 / 10^7 = 103 ms of standing queue

// Bufferbloat arithmetic: a 1 MB buffer on a 10 Mbit/s uplink
// 1 048 576 bytes * 8 bits / 10^7 bits/s = 839 ms of latency ADDED
// to every packet behind the bloat -- including your DNS query and your
// video call, which share the link.
// TCP's loss-based congestion control needs a DROP to learn the path is
// full. A huge buffer delays that signal by exactly the buffer's drain
// time, so the sender keeps increasing its rate while latency climbs.

// The fix is not a smaller fixed buffer -- it is an AQM that drops or
// marks early, based on how long packets are QUEUED:
$ tc qdisc replace dev eth0 root fq_codel
// CoDel targets 5 ms of standing queue and drops when the MINIMUM delay
// over a 100 ms window exceeds it. Same idea as Topic 66's deadline check.
```

10 Mbit/s uplink, single flow, buffer size varies:

buffer	standing queue	ping under load	throughput
64 KB	52 ms	~60 ms	9.4 Mbit/s
256 KB	210 ms	~220 ms	9.5 Mbit/s
1 MB	839 ms	~850 ms	9.5 Mbit/s
fq_codel	5 ms	~15 ms	9.4 Mbit/s

16x more buffering buys 1% throughput and costs 16x the latency.
That trade is the entire bufferbloat story.

THE KEY INSIGHT

A queue's job is to absorb *bursts*, not to store a backlog. CoDel makes that explicit by measuring *time in queue* rather than queue length — which is the same reframing as Little's Law (Topic 64) and the same as the deadline check in Topic 66. Three different systems, one idea.

THE TRAP

Tuning `net.core.wmem_max` upward because "the connection is slow". On a lossy or congested path that adds standing queue rather than throughput. Symptom: bulk transfers unchanged, interactive latency on the same host degraded by hundreds of milliseconds — and the person who made the change is looking at throughput graphs, where nothing happened.

```
bufferbloat – 1 MB at 10 Mbit/s = 839 ms added · 16× buffer buys 1 % throughput · fq_codel  
targets 5 ms standing queue
```

70. Redis lists and streams; Laravel queue workers

The application-level queue most web developers actually operate. Redis **LPUSH/BRPOP** is a blocking queue over the network; the difference between it and a real broker is what happens when a worker dies mid-job.

```
# Redis as a queue -- the two generations  
LPUSH jobs '{"id":1,...}'          # producer, 0(1)  
BRPOP jobs 0                      # consumer, blocks. Atomic pop.  
# PROBLEM: between BRPOP and the crash, the job exists NOWHERE.  
BRPOPLPUSH jobs processing 0      # atomic pop + push to a holding list  
# now a crashed worker leaves the job in `processing`, recoverable by a  
# reaper that moves stale entries back. This is a visibility timeout,  
# hand-built.  
  
# Redis Streams (5.0+) build it in:  
XADD jobs '*' payload '...'  
XREADGROUP GROUP w1 worker-3 COUNT 1 BLOCK 0 STREAMS jobs '>'  
XACK jobs w1 1699-0               # explicit acknowledgement  
XPENDING jobs w1                  # what is claimed but not acked  
XAUTOCLAIM jobs w1 worker-4 60000 0 # steal jobs idle > 60 s
```

```
// Laravel: the same concepts with different names.  
// php artisan queue:work --tries=3 --timeout=60 --backoff=10 --max-jobs=1000  
// --tries      attempts before the job goes to failed_jobs (the DLQ, T72)  
// --timeout    SIGALRM kills the worker; MUST be < retry_after or the  
//              job is released and re-run while the first copy is  
//              STILL RUNNING -- the classic duplicate-charge bug  
// --backoff    delay before retry (see Topic 73)  
// --max-jobs   restart the worker to bound memory leaks in PHP  
// config/queue.php: 'retry_after' => 90 // must exceed --timeout  
// Rule: timeout < retry_after, always. 60 < 90 is correct; 90 < 60 is a  
// double-execution bug that appears only on slow jobs.
```

at-least-once delivery, and where duplicates come from

worker A: BRPOP job 42 ---- processing (55 s) ---- ACK

retry_after = 50 s -----+ job 42 becomes visible again

worker B: BRPOP job 42 ---- processing ----> DUPLICATE

the fix is either (a) `retry_after` > worst-case processing time, or
(b) idempotent handlers keyed on a job id -- and (b) is the only one
that survives a paused VM or a stop-the-world GC.

THE KEY INSIGHT

Every distributed queue delivers at least once, never exactly once. The guarantee you actually get is "at least once plus idempotent handlers", and the idempotency key is your job's responsibility, not the broker's. Design the handler so that running it twice with the same id is harmless.

THE TRAP

A plain **RPOP/BRPOP** worker that crashes after popping. The job is gone with no trace — not in the queue, not in a failed table, not in any log. Symptom: a small fraction of jobs that simply never ran, discovered only when a customer asks. Use **BRPOPLPUSH**, Streams with **XACK**, or a broker that has visibility timeouts built in.

Redis queue – LPUSH/BRPOP 0(1) · BRPOPLPUSH = crash-safe · timeout MUST be < retry_after · delivery: at least once, always

71. Brokers: ordering versus throughput

The trade every message broker forces: strict ordering requires a single consumer per ordered stream, and a single consumer caps throughput. Every broker resolves it the same way — partition by key.

System	Ordering unit	Parallelism	Cost of ordering
Kafka	Partition	1 consumer per partition per group	Max parallelism = partition count
SQS standard	None	Unlimited	N/A — no ordering at all
SQS FIFO	MessageGroupId	1 in flight per group	300 msg/s per group (3000 batched)
RabbitMQ	Queue, 1 consumer	Prefetch > 1 breaks order	prefetch=1 and one consumer
Redis Streams	Stream, 1 consumer	Consumer group breaks order	One stream per ordering key

```
// The design rule: partition by the entity whose order matters.
// key = user_id    -> all events for one user are ordered; different
//                  users run in parallel. 1000 partitions = 1000x
//                  parallelism with per-user ordering preserved.
// key = null       -> round-robin, max throughput, no ordering anywhere.
// key = constant   -> ONE partition does all the work: you have built a
//                  single-threaded system with a distributed broker.
//
// Kafka partition count is the throughput ceiling for a consumer group:
// 6 partitions, 10 consumers -> 4 consumers IDLE, forever.
// partitions can be increased but NOT decreased, and increasing them
// changes key -> partition mapping, so ordering breaks across the change.
// Size partitions for the throughput you will need in a year.
```

producer with key = user_id

```
user 7 --hash--> partition 1 [ e1 e2 e3 ] --> consumer A (ordered)
user 9 --hash--> partition 3 [ f1 f2   ] --> consumer B (ordered)
user 2 --hash--> partition 1 [ g1      ] --> consumer A
```

ordering holds WITHIN a partition only. e2 before e3: guaranteed.
e2 before f1: no guarantee, and none is needed -- different users.

THE KEY INSIGHT

You almost never need global ordering — you need ordering per entity. Choosing the partition key *is* choosing which orderings you keep, and it is the single highest-leverage decision in a queue-based architecture. This is Topic 6's "two consumers destroy FIFO" turned into a design tool.

THE TRAP

A hot partition from a skewed key. If 40 % of events carry one tenant id, one consumer does 40 % of the work while the others idle. Symptom: consumer lag on exactly one partition, flat CPU everywhere else, and adding consumers changes nothing. Fix by compounding the key (tenant + entity) or by giving the hot tenant its own topic.

brokers – ordering unit = partition/group · parallelism ceiling = partition count · SQS FIFO 300 msg/s per group · skewed key = one hot consumer

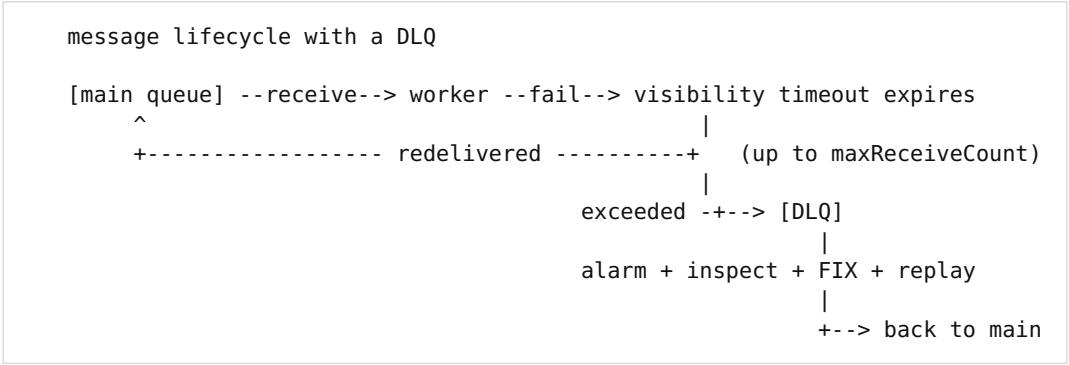
72. Dead-letter queues and poison messages

A poison message is one that fails every time it is processed. Without a dead-letter queue it is retried forever, consuming capacity and blocking everything behind it if ordering is enforced.

```
# SQS redrive policy -- the standard shape.
{
  "RedrivePolicy": {
    "deadLetterTargetArn": "arn:aws:sqs:...:orders-dlq",
    "maxReceiveCount": 5          # after 5 receives, move to the DLQ
  },
  "VisibilityTimeout": 60,       # must exceed worst-case handler time
  "MessageRetentionPeriod": 345600 # 4 days; the DLQ should be LONGER
}
# maxReceiveCount counts RECEIVES, not failures: a worker that crashes
# before acking burns an attempt without ever running the handler. Set it
# high enough (5-10) that a deploy or a node restart does not DLQ good work.
```

```
// The retry schedule matters as much as the count. Exponential backoff
// with jitter, or 10 000 clients retry in lockstep and re-create the spike:
//   attempt 1: 1 s      attempt 4: 8 s
//   attempt 2: 2 s      attempt 5: 16 s    +/- random(0, delay/2)
//   attempt 3: 4 s      then -> DLQ
// Total retry window: ~31 s. If the downstream outage lasts 10 minutes,
// EVERYTHING goes to the DLQ and you must replay by hand -- so pair
// backoff with a circuit breaker that stops consuming entirely instead.

// What a DLQ needs to be useful, and usually lacks:
//   1. an alarm on ApproximateNumberOfMessagesVisible > 0 (else nobody
//      looks at it until a customer complains)
//   2. the original failure reason stored in a message attribute
//   3. a tested REPLAY path back to the main queue
// A DLQ with no replay tooling is a folder where messages go to die.
```



THE KEY INSIGHT

A DLQ converts an unbounded retry loop into a bounded one plus a visible backlog. It is the same move as bounding a queue in Topic 66: you cannot avoid the failure, but you can decide where it accumulates and make it observable.

THE TRAP

An ordered queue (Kafka partition, SQS FIFO group) with a poison message at the head. Nothing behind it can be processed — head-of-line blocking, Topic 74 — so one bad message stalls an entire tenant's stream indefinitely. Symptom: lag on one partition growing linearly while the consumer is healthy and busy. You must be able to skip or park the offending offset.

DLQ – maxReceiveCount counts receives, not failures · backoff 1→16 s = ~31 s window · alarm at >0
· ordered queue + poison = full stall

73. Delay queues: timer wheel versus heap

"Run this in 30 seconds" is a priority queue keyed on time — or, if you are willing to bucket time, an array of queues with O(1) insertion. Both appear in production; the choice depends on how many timers you hold.

```
// HEAP: std::priority_queue keyed on the fire time.
//   insert O(log n), fire O(log n), exact to the nanosecond.
//   1 M pending timers: 20 comparisons per operation, cache-scattered.

// TIMER WHEEL: an array of buckets, one per tick, used cyclically.
//   Topic 39's bucket queue, with time as the key.
struct TimerWheel {                                // single level, 512 slots, 10 ms
    static constexpr int SLOTS = 512;
    std::vector<std::vector<Timer>> slot_{SLOTS};
    std::size_t now_ = 0;
    void add(Timer t, std::size_t ticks) { // ticks < SLOTS
        slot_[(now_ + ticks) % SLOTS].push_back(t);    // O(1)
    }
    void tick() {                                     // O(1) amortised
        for (auto& t : slot_[now_ % SLOTS]) fire(t);
        slot_[now_ % SLOTS].clear();
        ++now_;
    }
};
// Longer delays need a HIERARCHICAL wheel (ms / s / min / hour), which is
// exactly what the Linux kernel and Netty use. Cost: O(1) insert and O(1)
// per tick, with a cascade when an outer wheel rolls over.
//   Kafka's delayed operations, Netty's HashedWheelTimer, Linux
//   timer_list: all hierarchical wheels, all for the same reason.
```

single-level wheel, 512 slots x 10 ms = 5.12 s of range

```
now_ = 137                                add(t, 40 ticks) -> slot (137+40)%512 = 177
slot: 0   1   ... 137   ... 177   ... 511
      [ ] [ ]      [*]      [t]      [ ]
              ^tick() fires these, clears the slot, now_ = 138
```

```
heap: insert 20 comparisons, fire 20      -- exact times, O(log n)
wheel: insert 1 index computation, fire 0 -- resolution = tick size
```

THE KEY INSIGHT

A timer wheel trades precision for $O(1)$. If your timeouts are "about 30 seconds" rather than "exactly 30.000 seconds" — and connection timeouts, retry backoffs and heartbeats always are — the wheel is strictly better at scale. Use the heap when the number of timers is small or the times must be exact.

THE TRAP

Implementing delayed jobs by polling a database: `SELECT * FROM jobs WHERE run_at <= NOW() LIMIT 100` every second. At 10^6 pending jobs that is an index scan per second per worker, and with several workers you get lock contention and duplicate claims. Symptom: database CPU dominated by the job poller, and jobs running twice under load. Use `SKIP LOCKED`, or a purpose-built delay queue.

delay queues — heap $O(\log n)$ exact · wheel $O(1)$, resolution = tick · hierarchical wheel for long ranges · DB polling: 1 scan/s/worker

74. Head-of-line blocking, HTTP/1.1 to QUIC

Head-of-line blocking is a queue's defining weakness: the element at the front blocks everything behind it, however unrelated. The evolution of HTTP is a sequence of attempts to remove it, one layer at a time.

```
// HTTP/1.1 with pipelining -- application-level HOL blocking.
// GET /slow.json (2 s) GET /a.css GET /b.js
// Responses MUST come back in request order, so a.css and b.js wait 2 s.
// Browsers disabled pipelining entirely and opened 6 connections instead.

// HTTP/2 -- multiplexed streams over ONE TCP connection.
// Application-level HOL blocking: SOLVED. Streams interleave freely.
// TRANSPORT-level HOL blocking: NOT solved. One lost TCP segment stalls
// EVERY stream, because TCP must deliver bytes in order to the TLS layer.
// At 2% packet loss, HTTP/2 over TCP can be SLOWER than HTTP/1.1's six
// independent connections -- a lost packet stalls one of six, not all.

// HTTP/3 / QUIC -- streams are independent AT THE TRANSPORT LAYER.
// Each stream has its own delivery ordering; a lost packet stalls only
// the stream whose bytes it carried. Runs over UDP precisely to escape
// TCP's single ordered bytestream.
// Handshake: TCP+TLS1.3 = 2 RTT; QUIC = 1 RTT; QUIC resumption = 0 RTT.
```

one lost packet, three protocols

HTTP/1.1 x6 conns	[c1][c2][XX][c4][c5][c6]	1 of 6 stalls -> 17% hit
HTTP/2 over TCP	[s1 s2 s3 s4 s5 s6]	ALL stall: one bytestream
	^XX lost	-> 100% hit
HTTP/3 over QUIC	[s1][s2][XX][s4][s5][s6]	only s3 stalls -> 17% hit
		with 1 connection, 1 RTT

the pattern generalises: a shared FIFO couples unrelated work.
Independent queues decouple it -- at the cost of more state.

THE KEY INSIGHT

Head-of-line blocking is not a bug in any of these protocols — it is the cost of a single ordered queue, showing up at whichever layer still has one. Every fix has the same shape: replace one shared queue with N independent ones. That is also what Kafka partitions do (Topic 71) and what per-connection accept queues do (Topic 68).

THE TRAP

Deploying HTTP/2 on a lossy mobile network and expecting a win because it "multiplexes". Under 2–5 % loss, a single stalled TCP connection blocks every stream, and six HTTP/1.1 connections can measurably outperform it. Symptom: p95 page load worse after the HTTP/2 migration, only on mobile networks, while lab tests on a clean wired link showed an improvement.

HOL blocking – HTTP/1.1: app level · HTTP/2: transport level remains · QUIC: per-stream delivery, 1-RTT handshake, 0-RTT resumption

PRACTICE SET — Topics 67-74

A. Conceptual

1. Why did Linux move from a FIFO run queue to a vruntime-ordered structure? Relate it to Topic 32.
2. What does `/proc/pressure/cpu`'s `some` value measure, and why is it a queue metric rather than a CPU metric?
3. Name the four queues between the wire and an `accept()` call, and the command that shows the depth of the last one.
4. Why does enlarging a NIC ring buffer not fix a sustained rate deficit?
5. Explain why an oversized router buffer harms latency without helping throughput, and what signal TCP is deprived of.
6. What does CoDel measure that a fixed queue-length limit does not, and which two other topics in this book use the same idea?
7. Why is `BRPOP` alone unsafe, and what does `BRPOPLPUSH` add?
8. State the required relationship between a worker's timeout and the queue's `retry_after`, and the bug that results from getting it backwards.
9. Why can no distributed queue offer exactly-once delivery, and what is the practical substitute?
10. What does a Kafka partition key actually choose?
11. Why does `maxReceiveCount` counting *receives* rather than *failures* matter during a deploy?
12. Why is a poison message far worse in an ordered queue than in an unordered one?
13. Give the trade a timer wheel makes relative to a heap, and the condition that makes it acceptable.
14. HTTP/2 solved one kind of head-of-line blocking and not another. Name both and say which layer each lives at.

B. Tracing and analysis

1. A 1 ms round-robin quantum with 1-3 μ s switches: compute the overhead percentage per core, and the same for a 100 μ s quantum.
2. A NIC ring of 256 descriptors at 1.5 KB per packet on a 10 Gbit/s link: how long can the ring absorb a full-rate burst before overflowing?
3. Compute the added latency of a 512 KB buffer on a 5 Mbit/s uplink, and how much of that a 5 ms CoDel target would leave.
4. `ss -lnt` shows Recv-Q 129, Send-Q 128 on a listener. State what is happening and what the client sees.
5. Laravel `--timeout=90` with `retry_after=60`: describe the exact sequence for a job that takes 75 seconds, and how many times it runs.
6. A Kafka topic has 6 partitions and a consumer group of 10. How many consumers do work? How many are idle? What happens if partitions are increased to 12 mid-stream?
7. One tenant produces 40 % of events, keyed by tenant id, across 20 partitions. Give the relative load of the hottest consumer versus the average.
8. Exponential backoff 1, 2, 4, 8, 16 s with `maxReceiveCount=5`: compute the total retry window, and state what fraction of messages reach the DLQ if the downstream outage lasts 10 minutes.

9. A single-level timer wheel with 512 slots of 10 ms: give the maximum delay it can represent, and the number of levels needed for a 24-hour range at the same tick.
10. At 2 % packet loss, compare the fraction of in-flight streams stalled by one loss under HTTP/1.1 (6 connections), HTTP/2, and HTTP/3.

C. Lab tasks — build it, break it, observe, fix

1. Read your own machine's queue state: `ethtool -g` and `-S`, `cat /proc/net/softnet_stat`, `ss -lnt`, `cat /proc/pressure/cpu`. Record every number and identify which queue each belongs to.
2. **Break it:** write a server with `listen(fd, 1)` and a deliberately slow accept loop, then connect 200 clients at once. Record the Recv-Q from `ss -lnt`, the counter in `netstat -s | grep -i listen`, and what the clients experience. Raise the backlog and repeat.
3. **Break it:** saturate your uplink with a bulk transfer and measure ping latency during it. Then apply `tc qdisc replace dev <iface> root fq_codel` and measure again. Record both latency distributions and both throughputs.
4. **Break it:** build a Redis worker using `BRPOP` that `kill -9`s itself halfway through processing. Run 1000 jobs, count how many completed, and find the missing ones (you cannot). Switch to `BRPOPLPUSH` plus a reaper and repeat.
5. **Break it:** configure a worker timeout *longer* than `retry_after` with a job that sleeps for the difference. Log the job id and a process id on entry, and record the duplicate execution.
6. Implement a single-level timer wheel and benchmark insert and tick against a `std::priority_queue` at 10^6 pending timers. Report ns per insert for both.
7. Implement exponential backoff with full jitter and simulate 10 000 clients retrying after a 30-second outage. Plot the arrival rate with and without jitter and record the peak.
8. Run `curl --http1.1`, `--http2` and `--http3` against a site that supports all three, under `tc netem loss 2%`. Record total page time for each across 20 runs.

ANSWER KEY — Topics 67-74

A. Conceptual

1. Because a scheduler must be fair in **CPU time**, not in turns. Once tasks have different weights and burst lengths the ordering key is accumulated runtime, not arrival time — and a structure ordered by something other than arrival is **not a queue** (Topic 32).
2. The percentage of wall time during which **at least one task was runnable but not running** — i.e. waiting in the run queue. It measures **queueing delay**; CPU utilisation measures only that the server is busy, which Part Nine shows is a different thing.
3. NIC RX ring; per-CPU softirq backlog; SYN queue; accept queue. `ss -lnt` shows the accept queue as Recv-Q with its limit in Send-Q.
4. Because a ring absorbs **bursts**, not a rate mismatch: if the CPU drains slower than packets arrive on average, a larger ring only delays the drop and adds latency in the meantime (Topic 2's inequality).
5. Because the buffer **holds** packets instead of dropping them, and TCP's loss-based congestion control needs **a drop as its signal** that the path is full. The sender keeps increasing its rate while latency climbs by the buffer's drain time.
6. CoDel measures **time spent in the queue** (a target standing delay of 5 ms) rather than queue length. The same reframing appears in **Little's Law (Topic 64)** and the **deadline check (Topic 66)**.
7. Because between the pop and a crash the job exists **nowhere**. `BRPOPLUSH` atomically moves it to a processing list, so a crashed worker leaves a recoverable trace — a hand-built visibility timeout.
8. **timeout < retry_after**. Reversed, the queue makes the job visible again while the first worker is still running it → **concurrent duplicate execution**, the classic double-charge bug.
9. Because acknowledgement and processing cannot be made atomic across a network — the ack can be lost after the work is done. The substitute is **at-least-once delivery plus idempotent handlers** keyed on a job id.
10. It chooses **which orderings you keep**. Everything with the same key is ordered; everything else is parallel. It is simultaneously the ordering guarantee and the parallelism limit.
11. Because a rolling deploy or node restart causes receives that never run the handler; with a low `maxReceiveCount`, **healthy messages are sent to the DLQ by an ordinary deployment**.
12. Because in an ordered queue nothing behind the poison message can be processed — **head-of-line blocking** stalls the whole partition or group indefinitely, not just the one message.
13. The wheel trades **precision for O(1)**: resolution equals the tick size. Acceptable whenever the deadline is approximate — connection timeouts, retry backoff, heartbeats — which is nearly always.
14. Solved: **application-level** HOL blocking (response ordering), at the HTTP layer. Not solved: **transport-level** HOL blocking, at the TCP layer, where one lost segment stalls every multiplexed stream.

B. Tracing and analysis

1. 1 ms quantum: 1-3 μ s per 1000 μ s = **0.1-0.3 %**. 100 μ s quantum: 1-3 per 100 = **1-3 %**.
2. $256 \times 1.5 \text{ KB} = 384 \text{ KB} = 3.07 \text{ Mbit}$. At 10 Gbit/s that is **$\approx 307 \mu$ s** of full-rate burst absorption.

3. $512 \text{ KB} \times 8 = 4.19 \text{ Mbit} / 5 \text{ Mbit/s} = \mathbf{839 \text{ ms}}$. CoDel's 5 ms target leaves **5 ms** — a reduction of about **168×**.
4. The **accept queue is full** ($129 \geq 128$). New connections are dropped or refused; with `tcp_abort_on_overflow=0` the kernel **silently ignores the ACK**, so the client believes it is connected and then times out or retransmits — a connection that hangs rather than fails fast.
5. At $t=60 \text{ s}$ `retry_after` expires and the job is released to another worker while the first is still running (it has until 90 s). The job therefore **runs twice concurrently**, and the first worker completes at 75 s — possibly deleting a job the second worker now owns.
6. **6 consumers work, 4 are permanently idle**. Increasing to 12 partitions rebalances and allows 12 consumers, but the key→partition mapping changes, so **ordering is not preserved across the change** for keys that move.
7. Average share = 5 % per partition. The hot tenant's partition carries 40 % plus its share of the rest ($60 \text{ \%}/20 = 3 \text{ \%}$) $\approx \mathbf{43 \text{ \%}}$ — roughly **8.6×** the average consumer's load.
8. $1+2+4+8+16 = \mathbf{31 \text{ seconds}}$. A 10-minute outage far exceeds it, so **essentially 100 %** of messages received during the outage reach the DLQ.
9. $512 \times 10 \text{ ms} = \mathbf{5.12 \text{ s}}$ maximum delay. For 24 hours: $86\,400 \text{ s} / 0.01 \text{ s} = 8.64 \times 10^6$ ticks; with 512 slots per level, $512^3 = 1.34 \times 10^8 \rightarrow \mathbf{3 \text{ levels}}$ suffice.
10. HTTP/1.1 with 6 connections: one loss stalls 1 of 6 = **17 %**. HTTP/2: **100 %** of streams on that connection. HTTP/3: only the affected stream $\rightarrow \mathbf{1/N}$, about **17 %** for 6 streams — with a single connection and a 1-RTT handshake.

C. Lab tasks — what to verify

1. Every counter should map to exactly one queue. Check specifically whether `rx_missed_errors` is non-zero (ring overflow) and whether column 2 of `softnet_stat` is non-zero (backlog drops) — on an idle desktop both should be **0**.
2. With `listen(fd, 1)` expect Recv-Q to sit at **2** and `netstat -s` to report **"times the listen queue of a socket overflowed"** climbing. Clients see connections that appear to establish and then hang, *not* immediate refusals — that surprise is the lesson.
3. Expect ping under load in the **hundreds of milliseconds** before `fq_codel` and **10-25 ms** after, with throughput within **1-2 %** either way.
4. With `BRPOP`, completed jobs will be roughly **1000 minus the number of kills**, and the missing job ids exist **nowhere** — confirm you cannot recover them. With `BRPOPLPUSH` every missing job must be present in the processing list and recoverable by the reaper.
5. The log must show **two different pids processing the same job id with overlapping timestamps**. That overlap is the artefact to capture.
6. Expect the wheel at **<10 ns per insert** and the heap at **~100-250 ns** at 10^6 timers, with the heap degrading as n grows and the wheel flat.
7. Without jitter, expect a **spike of all 10 000 clients within one tick** at each backoff boundary. With full jitter the arrivals should spread roughly uniformly and the peak should fall by **an order of magnitude**.
8. Under 2 % loss expect HTTP/2 to be **equal to or worse than HTTP/1.1** and HTTP/3 to be clearly best. On a clean link HTTP/2 should beat HTTP/1.1 — run both conditions so the contrast is visible.

HANDNOTE SUMMARY CARD

PART TEN - QUEUES IN REAL SYSTEMS

topics 67-74

THE FOUR QUEUES BETWEEN WIRE AND accept()

```
1 NIC RX ring      ethtool -g / -S      rx_missed_errors, rx_no_buffer
2 softirq backlog  /proc/net/softnet_stat col 2 = dropped
3 SYN queue        tcp_max_syn_backlog  netstat -s | grep SYN
4 accept queue     ss -lnt: Recv-Q = DEPTH, Send-Q = LIMIT
Recv-Q >= Send-Q on a LISTEN socket = dropping connections right now.
overflow default: kernel IGNORES the ack -> client HANGS, not refused.
bigger ring absorbs BURSTS only. rate deficit -> more latency, same drops.
```

SCHEDULER

context switch 1-3 us. quantum 1 ms -> 0.1-0.3% overhead; 100 us -> 1-3%.
CFS/EEVDF: red-black tree on vruntime, not FIFO -- fairness in TIME not TURNS
/proc/pressure/cpu "some" = % of wall time a task was runnable but waiting
= queue depth as a fraction of time. high "some" -> reduce rho, not MHz.

BUFFERBLOAT

added latency = buffer_bytes * 8 / link_bps
1 MB on 10 Mbit/s = 839 ms. 16x buffer buys 1% throughput.
TCP needs a DROP as its congestion signal; a big buffer delays it.
fq_codel targets 5 ms STANDING queue -- measures TIME, not LENGTH
(same idea as Little's Law and the T66 deadline check)

APPLICATION QUEUES

Redis: LPUSH / BRPOP 0(1) but crash between pop and ack = JOB GONE
BRPOPLPUSH jobs processing = hand-built visibility timeout
Streams: XADD / XREADGROUP / XACK / XPENDING / XAUTOCLAIM
Laravel: queue:work --tries --timeout --backoff --max-jobs
*** timeout MUST be < retry_after *** else concurrent duplicate runs
DELIVERY IS AT-LEAST-ONCE, ALWAYS. exactly-once does not exist.
-> idempotent handlers keyed on job id. That is YOUR job, not the broker's.

BROKERS: ORDERING vs THROUGHPUT

Kafka ordering per PARTITION; 1 consumer/partition/group
6 partitions + 10 consumers = 4 IDLE FOREVER
partitions can grow but not shrink; growing REMAPS keys
SQS std no ordering, unlimited parallelism
SQS FIFO ordering per MessageGroupId, ~300 msg/s per group
RabbitMQ prefetch > 1 breaks order
partition key = WHICH ORDERINGS YOU KEEP. key=user_id -> per-user order,
cross-user parallelism. key=constant -> a single-threaded distributed system.
skew: 40% of traffic on one key across 20 partitions -> hottest = 8.6x avg

DLQ

maxReceiveCount counts RECEIVES not failures -> a deploy burns attempts; 5-10
backoff 1,2,4,8,16 s = a 31 s window. a 10-min outage DLQs everything ->
pair with a circuit breaker.
a DLQ needs: an alarm at >0, the failure reason, and a TESTED replay path.
poison message in an ORDERED queue = the whole partition stalls (HOL).

DELAY QUEUES

heap 0(log n), exact wheel 0(1), resolution = tick size
single level: 512 slots x 10 ms = 5.12 s; 24 h needs 3 levels
hierarchical wheels: Linux timer_list, Netty HashedWheelTimer, Kafka
DB polling every second = an index scan per worker + duplicate claims
-> SELECT ... FOR UPDATE SKIP LOCKED, or a real delay queue

HEAD-OF-LINE BLOCKING

HTTP/1.1 pipelining app-level HOL -> browsers used 6 connections instead
HTTP/2 app-level SOLVED, TRANSPORT-level remains: one lost
TCP segment stalls ALL streams (at 2% loss can lose
to HTTP/1.1)
HTTP/3 / QUIC per-stream delivery over UDP; 1-RTT (0-RTT resumed)
every fix has the same shape: replace ONE shared queue with N independent
ones -- exactly what Kafka partitions do.

PART ELEVEN

Tooling, safety and debugging

The standard library's sharp edges, and the sanitisers that find yours.

Part Eleven · Tooling, safety and debugging

75. `std::queue` is an adaptor: choosing the container

`std::queue` holds no data. It wraps a container and hides everything that is not a queue operation — which is its entire value (Topic 27) — and lets you choose what is underneath.

```
template<class T, class Container = std::deque<T>> class queue;
// Requirements on Container: front(), back(), push_back(), pop_front(),
// empty(), size(). That rules out std::vector -- no pop_front.

std::queue<int>          q1;    // deque: the default
std::queue<int, std::list<int>> q2; // legal, and always worse
// std::queue<int, std::vector<int>> q3; // does NOT compile

// Measured, 10 M interleaved push/pop of int, -O2:
//   std::queue<int>          (deque)      86 ms
//   std::queue<int, std::list<int>> (list)   640 ms    7.4x, 1 malloc/elem
//   hand-written Ring<int, 1024> (Part 3)   21 ms    4.1x FASTER

// Reserving is impossible through the adaptor -- there is no reserve() on
// std::queue. If you know the depth, construct the container first:
std::deque<int> d;           // (deque cannot reserve either)
std::queue<int> q(std::move(d)); // ... but this is how you would pass
// a pre-sized container in general
```

<code>std::queue<T></code>	what it EXPOSES / HIDES
+-----+	exposes: push, pop, front, back,
<code>std::deque<T> c;</code>	empty, size, emplace
(protected member)	hides: push_front, pop_back,
+-----+	operator[], begin/end, insert
the hidden operations are what make the FIFO guarantee STRUCTURAL	
rather than a comment. That is worth more than the 4x.	

THE KEY INSIGHT

Use `std::queue` for the guarantee, not the performance. When a hot path needs the 4×, drop to a fixed-capacity ring from Part Three and keep the narrow interface by writing your own adaptor — the interface and the layout are independent choices.

THE TRAP

`std::queue<T, std::list<T>>` chosen because "list has O(1) pop_front". So does deque, without a malloc per element. Symptom: 7× slower than the default and 8× the memory, in code whose author believed they were optimising.

`std::queue` — deque default 86 ms/10 M · list 640 ms · hand ring 21 ms · vector: won't compile · no reserve() through the adaptor

76. `std::priority_queue` and the reversed comparator

The single most common C++ queue bug: `std::priority_queue` is a *max*-heap by default, and its `Compare` parameter reads backwards from what everyone expects.

```

std::priority_queue<int> maxheap; // pops LARGEST
std::priority_queue<int, std::vector<int>, std::greater<int>> minheap;
// pops SMALLEST
// Read it as: Compare(a, b) == "a is ordered BEFORE b", and the heap pops
// the element that comes LAST in that ordering. With std::greater, larger
// values come first, so the SMALLEST comes last and is popped. Hence
// greater -> min-heap. This is inverted from every other STL algorithm,
// where std::greater sorts descending.

// A custom comparator for a struct, min-heap on cost:
struct ByCost {
    bool operator()(const Node& a, const Node& b) const {
        return a.cost > b.cost; // > for a MIN-heap
    }
};
std::priority_queue<Node, std::vector<Node>, ByCost> pq;

// Two more sharp edges:
// 1. top() returns CONST ref. You cannot mutate the top element in
// place -- doing so would break the heap invariant, so the library
// forbids it. Pop, modify, push.
// 2. There is no decrease-key, no erase, no find. Topic 37 explains
// why lazy deletion is the answer rather than a limitation.

```

the mental model that stops the mistake

```

std::sort(v.begin(), v.end(), std::greater<int>()) -> 9 7 5 3 1 descending
std::priority_queue<int, vector<int>, std::greater<int>>
                                                    -> pops 1 3 5 7 9 ASCENDING

```

same comparator, opposite-looking result. The heap pops the element the comparator ranks LAST. Say "greater -> min-heap" out loud until it sticks.

THE KEY INSIGHT

The rule is one sentence: `std::greater` gives a min-heap. Write it in a comment above every declaration — the direction is not something to re-derive under interview pressure, and re-deriving it is where the mistake happens.

THE TRAP

An inverted comparator in Dijkstra produces a *longest-path-first* search that still terminates and still fills the distance array — with wrong values on most vertices. Symptom: correct output on tiny graphs (where all orders coincide) and subtly wrong distances on real ones, with no crash. Always test on a graph where the greedy and correct answers differ.

priority_queue – default max-heap · `std::greater` → min-heap · top() is const · no erase, no decrease-key, no find

77. Python: collections.deque, maxlen, heapq

Three Python-specific behaviours that surprise C++ programmers, and one that surprises everyone.

```

from collections import deque
import heapq

# 1. maxlen SILENTLY DISCARDS from the other end. No exception, no return
#    value, no warning.
d = deque(maxlen=3)
for x in range(5): d.append(x)
print(d)           # deque([2, 3, 4], maxlen=3) -- 0 and 1 are GONE
#    This is Topic 5's "drop the oldest" policy, and it is right for a
#    ring log and catastrophic for a job queue.

# 2. Indexing is O(n). Ends are O(1).
d = deque(range(1_000_000))
d[500_000]         # walks ~7 800 blocks of 64 pointers: ~40 us
d[0], d[-1]        # O(1)

# 3. heapq is a MIN-heap only. For a max-heap, negate:
heapq.heappush(h, -priority)      # and negate again on pop
#    and heapq operates on a plain list in place -- h[0] is the minimum,
#    but h is NOT sorted (Topic 33).

# 4. THE tuple comparison trap:
heapq.heappush(h, (5, {"id": 1}))
heapq.heappush(h, (5, {"id": 2})) # TypeError: '<' not supported between
#                                # instances of 'dict' and 'dict'

#    On a tie in the first element Python compares the SECOND. Insert a
#    unique sequence number in between -- which also gives you the FIFO
#    tie-breaking of Topic 36:
heapq.heappush(h, (priority, next(counter), payload))

```

what to use, by requirement

FIFO, single thread	collections.deque	append / popleft
FIFO, threads, blocking	queue.Queue	put / get (wraps deque + Condition)
FIFO, asyncio	asyncio.Queue	await put / get
bounded, drop oldest	deque(maxlen=n)	SILENT drop
bounded, block or raise	queue.Queue(maxsize=n)	put() blocks, put_nowait() raises
priority	heapq (min only)	(pri, seq, payload)
NEVER: list.pop(0) -- O(n)	memmove, Topic 7	

THE KEY INSIGHT

`deque.append` and `deque.popleft` are atomic under the GIL, so a plain deque is safe for producer/consumer between threads — but it cannot *block*. Use it when the consumer can poll; use `queue.Queue` when it should sleep. That distinction, not thread safety, is the reason both exist.

THE TRAP

`deque(maxlen=n)` used as a work queue. When producers outrun consumers, the oldest jobs are discarded with no exception, no log line and no return value to check. Symptom: jobs that never ran, in numbers that correlate with load, and a queue whose length metric looks perfectly healthy at exactly `n`.

Python – deque ends O(1), index O(n) · maxlen drops silently · heapq is min-only · tuple ties compare the next element → TypeError

78. Java: ArrayDeque, PriorityQueue and iteration order

Java's collections have their own set of traps, and two of them produce wrong answers rather than exceptions.

```
// 1. ArrayDeque REJECTS null. So does ArrayBlockingQueue.
Deque<String> d = new ArrayDeque<>();
d.add(null); // NullPointerException
// Because null is the sentinel for "empty" in poll()/peek(). LinkedList
// allows null, which is why poll() returning null there is ambiguous.

// 2. PriorityQueue.iterator() is NOT in priority order.
PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 3, 2));
for (int x : pq) System.out.print(x + " "); // 1 2 3 5 -- ARRAY order,
System.out.println(); // which only LOOKS sorted here
// The Javadoc says explicitly: "does not guarantee any particular
// ordering". Only repeated poll() is ordered. toString() has the same
// problem, which makes debug output actively misleading.

// 3. The queue family's four method pairs -- the difference is the failure:
//      throws exception    returns special value
// insert    add(e)         offer(e)      -> false if bounded and full
// remove    remove()       poll()        -> null if empty
// examine   element()      peek()        -> null if empty
// blocking  put(e) / take() offer(e, timeout, unit)
// Mixing them is how "it worked until the queue filled up" happens:
// add() on a full ArrayBlockingQueue throws IllegalStateException.

// 4. Iterating a PriorityQueue and expecting sorted output is second only
// to using LinkedList where ArrayDeque belongs:
// ArrayDeque as a stack is 2-4x faster than the (synchronised, legacy)
// java.util.Stack, and faster than LinkedList for every end operation.
```

PriorityQueue internal array after adding 5, 1, 3, 2:

```
index  0  1  2  3
      [1, 2, 3, 5]    this happens to look sorted
add 0 -> [0, 1, 3, 5, 2] it no longer does
iterator() walks the ARRAY: 0 1 3 5 2
repeated poll():           0 1 2 3 5      <- the only ordered view
```

THE KEY INSIGHT

Java's dual method families encode Topic 5's policy question in the API: **add/remove** throw, **offer/poll** signal, **put/take** block. Choosing the pair is choosing the overflow and underflow policy — and the compiler will not tell you if you mix them.

THE TRAP

Logging a **PriorityQueue** with **toString()** during debugging. The output is heap-array order and frequently looks sorted for small inputs, so it appears to confirm a broken comparator is fine. Symptom: hours lost verifying a heap that "prints correctly". Drain a copy with **poll()** to inspect true order.

Java – ArrayDeque rejects null · PriorityQueue.iterator() unordered · add throws / offer signals / put blocks · ArrayDeque 2–4x java.util.Stack

79. Your ring buffer under AddressSanitizer

ASan catches the two classes of bug Part Three warns about — out-of-bounds slot access and use-after-free of a dequeued element — and it catches them at the moment they happen rather than at the eventual crash.

```
$ g++ -std=c++17 -g -O1 -fsanitize=address,undefined ring_test.cpp
$ ./a.out
=====
==8123==ERROR: AddressSanitizer: heap-buffer-overflow on address
    0x60300000eff8 at pc 0x0000004011ab bp 0x7ffd... sp 0x7ffd...
READ of size 4 at 0x60300000eff8 thread T0
    #0 0x4011aa in Ring<int, 8>::dequeue() ring.h:44
    #1 0x401320 in main ring_test.cpp:19
0x60300000eff8 is located 0 bytes to the right of 32-byte region
    [0x60300000efd8,0x60300000eff8)
allocated by thread T0 here:
    #0 0x7f4a in operator new(unsigned long)
    #1 0x401044 in Ring<int, 8>::Ring() ring.h:12
//  "0 bytes to the right" = read exactly one element past the end:
//  the classic slot(tail_) instead of slot(tail_ - 1) from Topic 18,
//  or a missing mask.

// The three flags worth knowing:
//  -fsanitize=address      out of bounds, use-after-free, double free
//  -fsanitize=undefined    signed overflow (Topic 10!), misaligned access
//  -D_GLIBCXX_DEBUG        checked STL iterators -- catches the Topic 30
//                          "push while iterating a deque" bug that ASan
//                          misses entirely
// Cost: ASan ~2x slower and ~3x memory; UBSan ~1.2x; _GLIBCXX_DEBUG can be
// 10x on container-heavy code. Run them in CI, not in production.
```

which tool finds which bug

bug	ASan	UBSan	_GLIBCXX_DEBUG	TSan
read past the ring's last slot	YES	-	-	-
use-after-free of a dequeued element	YES	-	-	-
missing ~T() (leak)	partial (use -fsanitize=leak / valgrind)			
signed index overflow (Topic 10)	-	YES	-	-
push_back while iterating a deque	-	-	YES	-
data race on head_/tail_	-	-	-	YES
`if` instead of `while` on a CV wait	-	-	-	NO (!)

THE KEY INSIGHT

"0 bytes to the right of a 32-byte region" tells you the exact off-by-one without any reasoning: 32 bytes is eight ints, so index 8 of an eight-element ring was read. Learn to read the region size and offset — it usually identifies the bug faster than the stack trace.

THE TRAP

Believing a clean ASan run proves the queue is correct. ASan sees memory errors only. A missing destructor (Topic 17), a wrong comparator (Topic 76), a lost wakeup (Topic 60), and a reordered-under-growth ring (Topic 12) are all invisible to it. Symptom: confidence. Pair sanitisers with the invariant assertions of Topic 20 and a differential test against `std::queue`.

80. Your blocking queue under ThreadSanitizer

TSan finds data races — two threads accessing the same memory with at least one write and no synchronisation. It is extremely good at that and blind to everything else, which is exactly the distinction Part Eight needs you to make.

```
$ g++ -std=c++17 -g -O1 -fsanitize=thread bq_test.cpp -lpthread
$ ./a.out
=====
WARNING: ThreadSanitizer: data race (pid=9014)
  Write of size 8 at 0x7b0400000d20 by thread T2:
    #0 SpscRing<int,1024>::push(int) spsc.h:21
  Previous read of size 8 at 0x7b0400000d20 by thread T1:
    #0 SpscRing<int,1024>::pop(int&) spsc.h:31
  Location is heap block of size 8200 at 0x7b0400000000 allocated by main
  SUMMARY: ThreadSanitizer: data race spsc.h:21 in SpscRing::push
=====
// This fires when buf_ is a plain array and the indices are plain
// size_t. Making the indices std::atomic with acquire/release makes the
// race disappear -- because TSan understands the happens-before edge
// that release/acquire creates.

// WHAT TSAN WILL NOT FIND:
// * `if (q_.empty()) cv.wait(lk);` instead of the predicate form.
//   There is no race: the mutex is held throughout. The bug is a missing
//   re-check, which is a LOGIC error. TSan reports NOTHING. (Topic 60)
// * A lost wakeup, for the same reason.
// * A deadlock from lock ordering -- TSan has a separate deadlock
//   detector, off by default; enable with TSAN_OPTIONS=detect_deadlocks=1
// * Missing bounds on a queue (Topic 66). That is a design error.
// Cost: 5-15x slower, 5-10x memory. Run a dedicated CI job.
```

a happens-before edge TSan understands, and one it does not

OK: tail_.store(t+1, release) ---> tail_.load(acquire)
TSan sees the edge; the buf_ write is ordered before the read.

RACE: tail_ = t+1; ---X t = tail_;
plain stores/loads. No edge. TSan reports the data race on buf_
as well, because nothing orders the payload either.

INVISIBLE: mutex held, predicate checked with `if`. No race exists.
TSan is silent. Only a stress test with many consumers finds it.

THE KEY INSIGHT

TSan validates *synchronisation*, not *logic*. If your concurrent queue is TSan-clean, you have proved there is no unordered memory access — you have not proved the queue is correct. The predicate-form `wait` and the invariant assertions are what cover the rest.

THE TRAP

Running TSan only on x86. The memory-ordering bug of Topic 61 — *relaxed* where *release* is needed — may not manifest on x86 at all, but TSan *does* flag it because it models the C++ memory model rather than the hardware. Symptom: a team that dismisses a TSan warning as a false positive because "it works", then ships to ARM.

TSan — 5–15× time, 5–10× memory · finds races and unordered access · blind to logic errors, lost wakeups, missing bounds

81. Benchmarking honestly

Most published queue benchmarks measure something other than the queue. Five rules, each of which changes a result by more than the difference between implementations.

```
// 1. INTERLEAVE. push 1 M then pop 1 M measures allocation and streaming.
//   Real queues alternate. Use a 2:1 push:pop ratio with bounded depth.
//   batched:      list queue looks 1.4x slower than deque
//   interleaved:  list queue is 7.4x slower          (Topic 75)

// 2. DEFEAT THE OPTIMISER. An unused result deletes the whole loop.
//   static void escape(void* p) { asm volatile("" : : "g"(p) : "memory"); }
//   auto v = q.dequeue(); escape(&v);          // or accumulate a checksum

// 3. WARM UP, then measure. First-touch page faults, allocator growth and
//   branch predictor training all belong to the warm-up.
//   for (int i = 0; i < 100'000; ++i) work();    // discard
//   auto t0 = std::chrono::steady_clock::now();

// 4. REPORT A DISTRIBUTION, NOT A MEAN. A growable queue's mean hides the
//   O(n) grow spike, which is the thing you care about (Topic 21).
//   ring   p50 2.1 ns   p99 2.4 ns   max    3.1 ns
//   growable p50 2.2 ns p99 2.6 ns   max 4 100 ns  <-- the reallocation
//   The mean of the second is 2.3 ns and tells you nothing.

// 5. PIN THREADS AND STATE THE MACHINE. For Part Eight benchmarks, two
//   hyperthreads on one core share L1 and hide false sharing entirely.
//   taskset -c 0,2 ./bench      # check /proc/cpuinfo for real cores
//   Report: CPU model, core count, compiler and flags, and whether
//   frequency scaling was fixed (cpupower frequency-set -g performance).
```

the same queue, four measurement setups:

```
batched, -00, no warm-up, mean ..... 412 ns/op   (meaningless)
batched, -02, warm-up, mean .....    19 ns/op
interleaved, -02, warm-up, mean .....    28 ns/op
interleaved, -02, warm-up, p99 .....    34 ns/op
... and the p99.9 of the growable variant ... 4 100 ns/op
```

a 20x spread from measurement choices alone -- larger than the gap between any two implementations in this book.

THE KEY INSIGHT

Report percentiles for queues, always. A queue exists to absorb variability, so its own tail behaviour is the property that matters; a mean deliberately hides the reallocation, the shuttle (Topic 41), and the futex sleep (Topic 59) — the three events you actually chose your implementation to control.

THE TRAP

Benchmarking with `-O0` "to see the real work". Optimisation changes queue code more than most: inlining removes the function call that dominated your measurement, and the relative ordering of implementations frequently reverses. Symptom: a benchmark that recommends the slower structure. Always `-O2` or `-O3`, and always with an escape barrier.

benchmarking – interleave · escape the result · warm up · report p50/p99/max · pin to physical cores · measurement spread > implementation spread

PRACTICE SET — Topics 75-81

A. Conceptual

1. Why can `std::vector` not be the container for `std::queue`?
2. What is the real reason to prefer `std::queue` over a bare `std::deque`, given the deque is faster to use directly?
3. Explain, in terms of what `Compare` means, why `std::greater` produces a min-heap.
4. Why does `std::priority_queue::top()` return a const reference?
5. Give two independent failure modes of `deque(maxlen=n)` used as a work queue.
6. Why does `heapq` raise a `TypeError` on some pushes and not others, and what is the two-word fix?
7. Why does `ArrayDeque` reject null while `LinkedList` accepts it?
8. Why is `PriorityQueue.toString()` actively dangerous during debugging?
9. Map Java's `add/offer/put` onto Topic 5's overflow policies.
10. In an ASan report, what do "0 bytes to the right of a 32-byte region" and "READ of size 4" together tell you about a ring of ints?
11. Name three queue bugs from this book that ASan cannot find.
12. Why is TSan silent on the `if-instead-of-while` condition variable bug?
13. Why can a TSan warning be real even when the program never fails on x86?
14. Why does batched benchmarking flatter a linked-list queue?
15. Why must queue benchmarks report percentiles rather than a mean?

B. Tracing and analysis

1. Given 86 ms (deque), 640 ms (list) and 21 ms (hand ring) for 10 M operations, compute ns/op for each and both ratios against the deque.
2. `std::priority_queue<int, vector<int>, greater<int>>` receives 4, 9, 1, 7. Give the pop order. Repeat for the default comparator.
3. A Python `deque(maxlen=1000)` receives 10^6 items with no consumer. How many are silently discarded, and what does `len(d)` report throughout?
4. A Java `PriorityQueue` is built from [5, 1, 3, 2] and then 0 is added. Give the internal array and the output of `iterator()` versus repeated `poll()`.
5. An ASan report says "8 bytes to the right of a 64-byte region, WRITE of size 8". For a ring of `double`, what is the capacity and which index was written?
6. ASan costs $2\times$ time and $3\times$ memory; TSan $10\times$ and $8\times$. A CI suite takes 4 minutes and 2 GB. Give the time and memory for a full run under each.
7. Given ring p99 = 2.4 ns and growable p99.9 = 4100 ns with p50 = 2.2 ns: at 10^7 operations, how much total time is spent in reallocation spikes if 0.1 % of operations hit them?

C. Code tasks — build it, break it, observe, fix

1. Benchmark `std::queue<int>`, `std::queue<int, std::list<int>>` and your Part Three ring on an interleaved workload of 10 M operations. Report p50, p99 and max for each.
2. **Break it:** run the same benchmark batched (all pushes, then all pops) and at `-O0`. Record how the ranking changes and by how much.
3. **Break it:** write Dijkstra with `std::priority_queue<pair<int,int>>` and no custom comparator (so it pops the largest). Run on a 1000-vertex weighted graph and compare against a correct

implementation: record how many vertices have wrong distances and whether the program crashes.

4. **Break it:** in Python, push two tuples with equal first elements and dict payloads into a heap. Record the exact `TypeError`. Fix with a sequence counter and confirm FIFO tie-breaking with a test.
5. **Break it:** build a Part Three ring with `back()` implemented as `slot(tail_)`. Compile with `-fsanitize=address` and run the Part One test matrix. Paste the report and identify the region size and offset.
6. Compile your `BlockingQueue` with `-fsanitize=thread` and run 4×4 producers/consumers. Confirm zero warnings. Then remove the mutex from one method and record what TSan reports.
7. **Break it:** take the correct SPSC ring and change `release/acquire` to `relaxed`. Run under TSan on x86 and record what it says, and whether the program itself ever fails.
8. Write a benchmark harness with warm-up, an escape barrier, percentile reporting and thread pinning. Use it to reproduce the false-sharing numbers of Topic 62 on your own machine.

ANSWER KEY — Topics 75-81

A. Conceptual

1. Because `std::queue` requires `pop_front()`, which `std::vector` does not provide (it would be $O(n)$ — Topic 7).
2. **The guarantee.** The adaptor removes `push_front`, `pop_back` and `operator[]`, so FIFO becomes structural rather than a convention a future caller can break.
3. `Compare(a,b)` means "a is ordered before b", and the heap pops the element that is **last** in that ordering. With `std::greater`, larger elements are ordered first, so the **smallest is last and is popped**.
4. Because mutating the top element could **break the heap invariant** without the container knowing. The library forbids it rather than silently corrupting; pop, modify, push.
5. (1) It **discards silently** — no exception, no return value, no log. (2) The **depth metric looks healthy**, pinned at n , so monitoring shows nothing while work is lost.
6. Because on a tie in the first tuple element Python **compares the next element**; dicts and most custom objects are not orderable. Fix: **sequence counter** inserted between priority and payload.
7. Because `ArrayDeque` uses **null as the sentinel** returned by `poll()/peek()` for "empty"; allowing null elements would make the return value ambiguous. `LinkedList` predates that design.
8. Because it prints **heap-array order**, which for small inputs frequently *looks* sorted — so it appears to confirm a broken comparator is working. Only repeated `poll()` is ordered.
9. `add` → **throw on overflow**; `offer` → **return false** (the try_/bool policy); `put` → **block** until space.
10. The region is 32 bytes = **8 ints**, so the ring's capacity is 8; "0 bytes to the right" with a 4-byte read means **index 8 was read** — exactly one past the end, i.e. a missing mask or a `tail_/tail_-1` confusion.
11. Any three of: **a missing `~T()`** (Topic 17), **a reversed comparator** (76), **a lost wakeup** (60), **an order-corrupting `memcpy` growth** (12), **an unbounded queue** (66).
12. Because **there is no data race**: the mutex is held across the check and the access. The defect is a missing re-check — a logic error, which TSan does not model.
13. Because TSan implements the **C++ memory model**, not x86's hardware ordering. Code that is UB by the standard can run correctly on a strongly ordered CPU and fail on ARM.
14. Because a batched run lets the allocator serve every node from a **warm free list** and streams memory sequentially; interleaving forces real allocation and deallocation traffic and destroys locality — $1.4\times$ versus $7.4\times$.
15. Because the events a queue implementation exists to control — **reallocation, shuttling, futex sleeps** — are rare and large, so they live in the tail. A mean averages them away.

B. Tracing and analysis

1. 86 ms → **8.6 ns/op**; 640 ms → **64 ns/op**; 21 ms → **2.1 ns/op**. List is **7.44× slower** than deque; the ring is **4.10× faster**.
2. `greater` (min-heap): **1, 4, 7, 9**. Default (max-heap): **9, 7, 4, 1**.
3. Discarded = $10^6 - 1000 =$ **999 000**. `len(d)` reports **1000** from the 1000th item onward — a perfectly flat, perfectly healthy-looking metric.
4. From [5,1,3,2] heapify gives array **[1, 2, 3, 5]**; adding 0 sifts up to **[0, 1, 3, 5, 2]**. `iterator()`: **0 1 3 5 2**. Repeated `poll()`: **0 1 2 3 5**.

5. 64-byte region / 8 bytes per double = **capacity 8**; "8 bytes to the right" with an 8-byte write means **index 8** was written — one past the end.
6. ASan: **8 minutes, 6 GB**. TSan: **40 minutes, 16 GB**. This is why they are separate CI jobs, not default flags.
7. $0.1\% \text{ of } 10^7 = 10^4$ spike operations $\times 4100 \text{ ns} = \mathbf{41 \text{ ms}}$, versus $9.99 \times 10^6 \times 2.2 \text{ ns} = 22 \text{ ms}$ for everything else. **The spikes are 65 % of total time** and are invisible in the mean.

C. Code tasks — what to verify

1. Expect ring p50 $\approx$ **2 ns**, deque **8-10 ns**, list **50-70 ns**. The list's max should be dramatically worse than its p50 — that is allocator variance.
2. Batched should compress the list's disadvantage to roughly **1.5×**; **-00** should compress everything toward a few hundred ns/op and may **reverse the ring/deque ordering**. Record the reversal if you see it.
3. The program **will not crash**. Expect a **majority of vertices with distances that are too large** (it explores longest-first), and correct answers on graphs where all paths are equal — which is why tiny tests pass.
4. Exact message: **TypeError: '<' not supported between instances of 'dict' and 'dict'**. After the fix, two equal-priority items must pop in insertion order.
5. Expect **heap-buffer-overflow ... READ of size 4 ... 0 bytes to the right of a 32-byte region** for a ring of 8 ints, with the frame naming **back()**. Note that the bug only fires when the ring is full.
6. Zero warnings for the correct version. After removing the mutex from **push**, expect **WARNING: ThreadSanitizer: data race** naming both **push** and **pop**, with the location inside the deque's internals.
7. TSan should report a **data race on buf_** even though the program on x86 **never produces a wrong answer**. Record both facts side by side — that pairing is the entire lesson of the topic.
8. You should reproduce the qualitative result: **padding alone is not faster than the naive version**, and **padding plus cached cursors is several times faster than both**. Absolute numbers will differ by machine; the ordering should not.

HANDNOTE SUMMARY CARD

PART ELEVEN - TOOLING, SAFETY AND DEBUGGING

topics 75-81

C++ STANDARD LIBRARY

std::queue> -- an ADAPTOR, holds no data
needs front/back/push_back/pop_front -> vector does NOT compile
hides push_front/pop_back/operator[] -> the FIFO guarantee is STRUCTURAL
10 M interleaved int ops: deque 86 ms | list 640 ms (7.4x) | ring 21 ms
no reserve() through the adaptor
std::priority_queue -- MAX-heap by default
*** std::greater -> MIN-heap *** (inverted vs std::sort. memorise it.)
Compare(a,b) = "a before b"; the heap pops what is ordered LAST
custom min-heap on cost: return a.cost > b.cost;
top() is CONST. no erase, no find, no decrease-key -> lazy deletion (T37)
reversed comparator in Dijkstra = no crash, wrong distances, passes tiny tests

PYTHON

collections.deque ends O(1), INDEXING O(n) (block walk, ~40 us at 1e6)
append/popleft ATOMIC under the GIL, but cannot BLOCK
deque(maxlen=n) SILENTLY DISCARDS the far end. len() sits at n forever.
queue.Queue(maxsize) blocking; put_nowait raises queue.Full
heapq MIN-heap only; push -priority for a max-heap
h[0] is the min; h is NOT sorted
tuple ties compare the NEXT element -> TypeError on dicts
-> heappush(h, (pri, next(counter), payload)) also gives FIFO ties
NEVER list.pop(0)

JAVA

ArrayDeque REJECTS null (null is the poll()/peek() empty sentinel)
PriorityQueue.iterator() and toString() are ARRAY order, NOT priority order
-- and small heaps often LOOK sorted. only repeated poll() is ordered.
add/remove/element THROW offer/poll/peek SIGNAL put/take BLOCK
(this is Topic 5's policy table, encoded in the API)
ArrayDeque is 2-4x java.util.Stack and beats LinkedList at every end op

SANITISERS - what finds what

-fsanitize=address	OoB, use-after-free, double free	2x time, 3x mem
-fsanitize=undefined	SIGNED index overflow (T10)	1.2x
-D_GLIBCXX_DEBUG	push while iterating a deque (T30)	up to 10x
-fsanitize=thread	data races, unordered atomics	5-15x, 5-10x mem

READING AN ASAN REPORT: "0 bytes to the right of a 32-byte region,
READ of size 4" -> capacity 8 ints, index 8 read. off by exactly one.
ASan CANNOT see: missing ~T(), reversed comparator, lost wakeup, memcpy
growth reordering, an unbounded queue.
TSan CANNOT see: `if` instead of `while` on a CV wait (no race exists!),
lost wakeups, deadlocks (unless detect_deadlocks=1), missing bounds.
A TSan warning on x86 that "never fails" is still REAL -> it fails on ARM.

BENCHMARKING - measurement spread exceeds implementation spread

- 1 INTERLEAVE batched flattens the list (1.4x) vs interleaved (7.4x)
- 2 ESCAPE the result asm volatile("" : : "g"(p) : "memory") or a checksum
- 3 WARM UP page faults, allocator growth, branch predictor
- 4 REPORT p50/p99/max, NEVER a mean -- the grow spike, the shuttle and the
futex sleep all live in the tail (0.1% of ops at 4100 ns = 65% of the time)
- 5 PIN to PHYSICAL cores (taskset); two hyperthreads hide false sharing

ALWAYS -02. -00 can reverse the ranking.

PART TWELVE

Interview and review

Recognition, not recall.

Part Twelve · Interview and review

82. Pattern recognition: phrasing to technique

Under time pressure you do not derive; you recognise. This topic is the mapping from the words a problem uses to the structure that solves it. Read the table, then practise saying the right-hand column out loud when you see the left.

The problem says	Reach for	Topic
"fewest steps", "minimum moves", "shortest path", unweighted	BFS with a queue	47, 48
"level by level", "each row of the tree", "k steps away"	BFS + size snapshot	50
"nearest of several", "rot spreads from all", "distance to any exit"	Multi-source BFS	49
"weights are only 0 and 1", "free moves and costly moves"	Deque, push_front/back	52
"maximum of every window of size k"	Monotonic deque	53, 54
"best of the last k states", $dp[i] = a[i] + \max(dp[j])$	Monotonic deque on dp	55
"dependencies", "prerequisites", "build order", "detect a cycle"	Kahn's, queue of indeg 0	57
"kth largest", "top k", streaming	Min-heap of size k	40
"merge k sorted lists/streams"	Heap of k heads	84
"shortest path with weights"	Heap + lazy deletion	37
"schedule by urgency", "most important first"	Heap — and ask about starvation	32, 36
"in order of arrival", "fair", "first come first served"	Queue; never a heap	2, 32
"implement X using only Y"	Composition; count moves per element	41, 42
"O(1) min alongside a queue"	Two stacks, each carrying its min	43
"least recently used", "cache eviction"	Hash map + linked list	45
"sliding window, window only moves right"	Two pointers; often no structure at all	46
"rate limit", "N per second"	Queue of timestamps, or token bucket	85
"producer and consumer", "worker pool"	Bounded blocking queue	59, 66
"run this in 30 seconds", "retry later"	Heap on time, or a timer wheel	73
"process events in order per user"	Partition by key	71

THE KEY INSIGHT

Three questions decide almost every queue problem. **Which end do I need to touch?** (one → queue, both → deque, middle → neither). **Is the order arrival or value?** (arrival → queue, value → heap). **Is the key a small integer?** (yes → buckets, and the log factor disappears).

THE TRAP

Pattern-matching on the *domain* rather than the *access pattern*. "Tasks with priorities" sounds like a heap and is often a queue with aging; "shortest path" sounds like Dijkstra and is BFS when all weights are 1. Read for the operations the problem needs, not the nouns it uses.

recognition – 3 questions: which end · arrival or value · small integer keys · answer them before writing any code

83. The master complexity table

Every structure in this book, with the numbers that decide between them. The constants matter as much as the exponents — a log factor is 20 at $n = 10^6$, but a cache miss is 200 cycles.

Structure	Push	Pop	Peek	Space/elem	Note
Shift array queue (7)	$O(1)^*$	$O(n)$	$O(1)$	sizeof(T)	Drain is $O(n^2)$ — 900 ms at $n=100k$
Head-index queue (8)	$O(1)^*$	$O(1)$	$O(1)$	grows forever	Space tracks throughput, not depth
Ring buffer (9)	$O(1)$	$O(1)$	$O(1)$	sizeof(T)	0 allocations; 2–4 ns/op
Growable ring (21)	$O(1)$ am.	$O(1)$	$O(1)$	$\leq 2 \times \text{sizeof}(T)$	Worst case $O(n)$: p99.9 spike 4100 ns
Linked queue (13)	$O(1)$	$O(1)$	$O(1)$	32 B for an int	Addresses stable; 5–20× scan cost
std::deque (14)	$O(1)$	$O(1)$	$O(1)$	$\sim 1.02 \times$	Index = 2 dependent loads
Deque (26)	$O(1)$ both	$O(1)$ both	$O(1)$	sizeof(T)	Same cost as the queue
Binary heap (33)	$O(\log n)$	$O(\log n)$	$O(1)$	sizeof(T)	20 cmp push / 40 pop at $n=10^6$
4-ary heap (38)	$O(\log_4 n)$	$O(\log_4 n)$	$O(1)$	sizeof(T)	1.48× faster measured
Bucket queue (39)	$O(1)$	$O(1)$ am.	$O(1)$	$O(P + n)$	Needs small integer keys
Two-stack queue (41)	$O(1)$	$O(1)$ am.	$O(1)$ am.	sizeof(T)	Worst case $O(n)$ per pop
Two-queue stack (42)	$O(1)/O(n)$	$O(n)/O(1)$	$O(1)$	sizeof(T)	No amortisation exists
Min-queue (43)	$O(1)$	$O(1)$ am.	min $O(1)$	$2 \times \text{sizeof}(T)$	Two min-stacks
Blocking queue (59)	$O(1)$	$O(1)$	—	sizeof(T)	22 ns uncontended, 1–5 μ s per sleep
SPSC ring (61)	$O(1)$	$O(1)$	—	sizeof(T)	2.3 ns/op with cached cursors

```
// Algorithm costs, same table, one line each:
// BFS (grid or graph) .....  $O(V + E)$       queue peak  $O(\text{frontier})$ 
// multi-source BFS .....  $O(V + E)$           once, not k times
// 0-1 BFS .....  $O(V + E)$                    vs  $O(E \log V)$  for a heap
// Dijkstra + heap + lazy delete ..  $O(E \log V)$  heap holds  $O(E)$  entries
// Dial's (weights  $\leq C$ ) .....  $O(E + V \cdot C)$ 
// Kahn's topological sort .....  $O(V + E)$     cycle detection free
// sliding window min/max .....  $O(n)$          each index in once, out once
// window DP .....  $O(n)$                      vs  $O(n \cdot k)$  naive
// heapify .....  $O(n)$                        vs  $O(n \log n)$  by pushes
// top-k, min-heap of size k .....  $O(n \log k)$   $O(k)$  space, streaming
// merge k sorted lists .....  $O(N \log k)$      $N$  = total elements
```

THE KEY INSIGHT

Three entries in this table have a worst case that differs from their amortised cost: the growable ring, the two-stack queue, and the min-queue. In an interview, volunteer that distinction before being asked — and in a latency-sensitive system, treat the worst case as the real number.

THE TRAP

Quoting $O(1)$ for a linked queue and stopping there. It is $O(1)$ with a malloc inside it: 25–60 ns per operation against the ring's 2–4, and a tail whose length depends on the allocator's mood. Complexity classes rank algorithms; constants decide implementations.

master table – 3 structures with worst $\neq$ amortised $\cdot \log_2 10^6 = 20 \cdot$ cache miss ≈ 200 cycles $\cdot$ malloc $\approx 15\text{--}30$ ns

84. Ten canonical problems, one insight each

The problems that recur across interview sets. For each, the one sentence that turns it from a puzzle into a template you already know.

Problem	Technique	The insight
Binary tree level order	BFS + snapshot	Read <code>q.size()</code> once per level, before the inner loop (50)
Rotting oranges	Multi-source BFS	Count fresh cells up front; the answer is the level count when it hits 0 (49)
Number of islands	BFS per unvisited cell	The outer loop is the source enumeration; total work is still $O(RC)$
Word ladder	BFS on an implicit graph	Neighbours are generated, not stored: $26 \cdot L$ candidates per word (47)
Course schedule	Kahn's	<code>order.size() != V</code> is the cycle test, free (57)
Sliding window maximum	Monotonic deque	Indices, not values; domination is permanent (53)
Jump game VI	Window DP	Expire, read, compute, then insert — in that order (55)
Merge k sorted lists	Heap of k heads	The heap holds one node per list, so it is size k, not N
Kth largest in a stream	Min-heap of size k	The root is the weakest of the best set (40)
Implement queue with stacks	Two stacks	Shuttle only when <code>out</code> is empty (41)

```
// Merge k sorted lists -- the one template not yet written in this book.
struct Item { int val; int list_id; int idx; };
struct ByVal { bool operator()(const Item& a, const Item& b) const {
    return a.val > b.val; } }; // > for a MIN-heap (T76)

std::vector<int> merge_k(const std::vector<std::vector<int>>& lists) {
    std::priority_queue<Item, std::vector<Item>, ByVal> pq;
    for (int i = 0; i < (int)lists.size(); ++i)
        if (!lists[i].empty()) pq.push({lists[i][0], i, 0});
    std::vector<int> out;
    while (!pq.empty()) {
        Item it = pq.top(); pq.pop();
        out.push_back(it.val);
        if (it.idx + 1 < (int)lists[it.list_id].size())
            pq.push({lists[it.list_id][it.idx + 1], it.list_id, it.idx + 1});
    }
    return out;
}
// O(N log k), not O(N log N): the heap never holds more than k items.
// N = 10^6 across k = 100 lists: 2*10^7 comparisons vs 4*10^7 for a full
// sort of the concatenation -- and the merge is streaming.
```

THE KEY INSIGHT

Nine of these ten are a template from Parts Five and Seven with the input renamed. The interview skill being tested is recognising which template applies, and the fastest route to that is having written each one at least twice from memory.

THE TRAP

Solving "merge k sorted lists" by concatenating and sorting. It is $O(N \log N)$, it needs all N in memory, and it cannot stream — and it is accepted by most judges, which is why the habit survives until the interviewer asks about a 100 GB input.

canonical set — 9 of 10 are templates renamed · merge k: $O(N \log k)$, heap size k · concatenate-and-sort: $O(N \log N)$ and non-streaming

85. Design a rate limiter

The most common queue-flavoured system design question, and a good one because the naive answer — a queue of timestamps — is correct, obvious, and wrong at scale for a reason you can state precisely.

```
// APPROACH 1: sliding window log. A queue of timestamps per key.
class SlidingLog {
    std::deque<std::int64_t> ts_; // milliseconds, ascending
    int limit_; std::int64_t window_ms_;
public:
    bool allow(std::int64_t now) {
        while (!ts_.empty() && ts_.front() <= now - window_ms_)
            ts_.pop_front(); // expire -- Topic 53's step
        if ((int)ts_.size() >= limit_) return false;
        ts_.push_back(now);
        return true;
    }
};
// EXACT. Memory = limit_ timestamps PER KEY.
// 100 req/min, 10 M users -> 10^9 timestamps * 8 B = 8 GB. Unusable.

// APPROACH 2: token bucket. Two numbers per key, no queue at all.
struct Bucket { double tokens; std::int64_t last_ms; };
bool allow(Bucket& b, std::int64_t now, double rate_per_ms, double burst) {
    b.tokens = std::min(burst, b.tokens + (now - b.last_ms) * rate_per_ms);
    b.last_ms = now; // refill lazily: no timer
    if (b.tokens < 1.0) return false;
    b.tokens -= 1.0;
    return true;
}
// 16 bytes per key -> 10 M users = 160 MB. Allows a burst of `burst`, which
// is usually what you want. This is what nginx limit_req and most gateways
// implement.
```

```
// APPROACH 3: sliding window counter -- the practical compromise.
// allowed = prev_window_count * overlap_fraction + current_window_count
// two integers per key, no burst-at-the-boundary bug of fixed windows,
// within ~1% of the exact answer. Redis: two INCR + EXPIRE per request.
//
// DISTRIBUTED: the counter must be shared. Options, in order of cost:
// * Redis INCR + EXPIRE ..... 1 round trip (~0.3 ms), exact-ish
// * local buckets, N nodes ..... each node gets limit/N; wastes capacity
// when traffic is skewed across nodes
// * local + periodic sync ..... approximate, no per-request round trip
// And the response matters as much as the decision: 429 with Retry-After
// and X-RateLimit-Remaining, so clients back off instead of retrying into
// the spiral of Topic 66.
```

fixed window's boundary bug -- why sliding exists

```
limit 100/min    |--- minute 1 ---|--- minute 2 ---|
requests        100 at :59   100 at :00
                ^^^^^^^^^^^
                200 requests in 2 seconds, both windows legal
```

token bucket: refill rate 100/60 s, burst 100 -> the second batch finds an empty bucket and is rejected. No boundary. Two numbers per key.

THE KEY INSIGHT

The token bucket replaces a queue of events with an *integral* of the rate: instead of remembering when each request happened, remember how much budget has accrued since the last one. Any "how many in the last T" question can be answered this way when approximation is acceptable, and the memory drops from $O(\text{limit})$ to $O(1)$ per key.

THE TRAP

Queueing rejected requests "so they get processed eventually". You have turned a rate limiter into an unbounded queue and reintroduced every failure in Part Nine. A rate limiter's job is to *reject*; if the work must survive, that is a job queue with its own bounded capacity and its own backpressure — a different component, deliberately chosen.

rate limiting – sliding log exact, 8 B×limit per key · token bucket 16 B per key · counter within ~1 % · always return 429 + Retry-After

86. Wrong answers, and how to catch yourself giving one

The failures that cost interviews and production incidents are not exotic. They are the same dozen, and each has a check you can run in your own head before the words leave your mouth.

The wrong answer	The check that catches it
"Two stacks give an amortised $O(1)$ stack from queues"	Does the expensive work make future work cheaper? If not, it is not amortised (42)
"A priority queue is a fair queue with priorities"	Can a low-priority item be overtaken forever? Then say "starvation" first (32)
"Just add a queue to handle the spike"	Is $\lambda_{\text{avg}} < \mu$? If not, the queue only schedules the failure (2)
"We'll make the queue bigger"	Queue depth is latency: depth/ μ seconds of p99 (64)
"85 % utilisation leaves headroom"	Compute $p/(1-p)$ at 0.85 and at 0.94 (65)
"The queue is exactly-once"	Name the moment the ack can be lost. It always exists (70)
"Kafka gives us ordering"	Ordering <i>per partition</i> . What is the key? (71)
"It's $O(1)$, we use a linked list"	$O(1)$ with a malloc inside: 25–60 ns vs 2–4 (13)
"BFS and DFS are interchangeable here"	Does the answer need <i>fewest</i> steps? Only FIFO gives that (3)
"The sanitisers are clean, so it's correct"	Which class of bug does each tool actually model? (79, 80)
"std::greater gives a max-heap"	Say it out loud: greater → min-heap (76)
"The benchmark says the list is fine"	Was it interleaved, at -O2, with percentiles? (81)

```
// The five sentences worth having ready, verbatim:
//
// 1. "Amortised  $O(1)$ , worst case  $O(n)$  on the operation that triggers the
//     growth or the shuttle -- and here is why the average still holds."
// 2. "This gives ordering per key, not global ordering. Global ordering
//     would cap throughput at one consumer."
// 3. "I'd bound this queue at  $\lambda$  times the acceptable wait, and shed
//     beyond it with a counter I can alarm on."
// 4. "Delivery is at least once, so the handler has to be idempotent on
//     the job id."
// 5. "I'd mark visited at enqueue time, so each vertex is pushed exactly
//     once and the queue holds  $O(V)$ , not  $O(E)$ ."
```

THE KEY INSIGHT

Every one of these checks is a question about a *guarantee*, not about code. The strongest signal you can give — in an interview or a design review — is naming what your structure does not promise before anyone asks. It is also, in production, the habit that prevents the incident.

THE TRAP

Learning these as facts to recite. They are checks to *run*: when you next reach for a queue, say aloud which end you touch, whether the order is arrival or value, and what happens when it fills. If you cannot answer the third, you have not finished designing.

self-check – 3 questions (which end, arrival or value, what happens when full) · state what you do NOT guarantee · volunteer the worst case

PRACTICE SET — Topics 82-86

A. Conceptual

1. Give the three questions that decide almost every queue problem, and the structure each answer points to.
2. A problem says "minimum number of moves" and the moves have costs 0 and 1. Which technique, and why not Dijkstra?
3. Name the three structures in the master table whose worst case differs from their amortised cost.
4. Why is "merge k sorted lists" $O(N \log k)$ rather than $O(N \log N)$?
5. State the boundary bug of a fixed-window rate limiter, with a concrete example at 100 requests per minute.
6. What does a token bucket store instead of a queue of events, and what is the memory per key?
7. Why is queueing rejected requests a category error for a rate limiter?
8. For each of these claims, give the one-line check: "just add a queue", "we'll make it bigger", "the queue is exactly-once".
9. Why is naming what your structure does *not* guarantee a strong signal in a design review?
10. A candidate says a two-queue stack is amortised $O(1)$. Give the exact counter-argument.

B. Tracing and analysis

1. A sliding-window log rate limiter allows 200 requests per minute for 5 M users. Compute worst-case memory at 8 bytes per timestamp, and the same for a token bucket at 16 bytes per key.
2. Token bucket with rate 10/s and burst 20, starting full. Requests arrive at $t = 0$ (30 of them), then 15 more at $t = 1.0$ s. How many are allowed at each point?
3. Merge 200 sorted lists totalling 4×10^6 elements: compute the comparison count for the heap merge and for concatenate-and-sort.
4. At $n = 10^6$, compare total comparisons for: heapify, n pushes, and a size-10 min-heap top-k pass.
5. A service at $\rho = 0.85$ with $\mu = 2000/\text{s}$. Compute W_q , then W_q after a 12 % traffic increase, and the ratio.
6. A Kafka topic keyed by country has 12 partitions; one country is 55 % of traffic. Give the hottest consumer's share and the factor above average.
7. For a bounded queue at $\lambda = 3000/\text{s}$ with a 40 ms wait budget, give the capacity, and the rejection rate if μ is only 2600/s sustained.

C. Code and design tasks — build it, break it, observe, fix

1. Implement all three rate limiters (sliding log, token bucket, sliding window counter). Drive them with the same 10^6 -request trace and report, for each: allowed count, memory, and disagreement with the exact sliding log.
2. **Break it:** implement a fixed-window counter limiter at 100/min. Construct the boundary attack (100 requests at :59, 100 at :00) and record how many are allowed in the two-second span. Then run the same trace through the token bucket and record the difference.

3. **Break it:** make the rate limiter queue rejected requests instead of returning 429, with an unbounded queue. Run 60 seconds at $1.5\times$ the limit and record queue depth, memory, and the age of the oldest queued request.
4. Implement merge-k with a heap and with concatenate-and-sort. Compare runtime and peak RSS at $N = 4\times 10^6$, $k = 200$.
5. Write, from memory and without looking back, the ten templates of Topic 84. Then diff each against the book and record which lines you got wrong. Repeat any template you missed twice more.
6. Take any three "wrong answers" from Topic 86 and write a five-line failing test or a measurement that demonstrates each one concretely.

ANSWER KEY — Topics 82-86

A. Conceptual

1. **Which end do I touch?** One → queue; both → deque; middle → not a queue (map + list, or a heap). **Is the order arrival or value?** Arrival → queue; value → heap. **Are the keys small integers?** Yes → bucket queue, and the log factor disappears.
2. **0-1 BFS with a deque**, $O(V+E)$. Dijkstra's heap costs $O(E \log V)$ for an ordering that collapses to two buckets when only two weights exist — the log factor buys nothing.
3. The **growable ring** ($O(n)$ on the reallocation), the **two-stack queue** ($O(n)$ on the shuttle), and the **min-queue** built on it.
4. Because the heap holds **one element per list** — at most k — not all N . Each of the N elements enters and leaves a heap of size k , so each costs $\log k$.
5. Two adjacent windows are each independently legal, so **100 requests at :59 and 100 at :00 pass — 200 in two seconds** against a stated limit of 100 per minute.
6. It stores **the accrued budget**: a token count and the timestamp of the last refill — an integral of the rate rather than a log of events. **16 bytes per key**, versus 8 bytes $\times$ limit for the log.
7. Because the limiter's purpose is to **reject**. Queueing the rejects reintroduces unbounded depth, latency growth and the retry spiral of Part Nine. If the work must survive, that is a separate, bounded job queue — a deliberate second component.
8. "Just add a queue" → **is $\lambda_{\text{avg}} < \mu$?** "Make it bigger" → **depth/ μ is added p99 latency**. "Exactly-once" → **name the moment the ack can be lost**.
9. Because it demonstrates you understand the **boundary of the guarantee**, which is where every incident happens. Anyone can state what a structure does; knowing what it refuses to do is what prevents relying on it.
10. Amortisation requires that **expensive work leaves the structure in a cheaper state**. Here the rotation is discarded: the next pop rotates the remaining $n-1$ elements from scratch, so the cost is $O(n)$ **every** time.

B. Tracing and analysis

1. Log: $200 \times 8 \text{ B} = 1600 \text{ B per user} \times 5 \times 10^6 = \mathbf{8.0 \text{ GB}}$. Token bucket: $16 \text{ B} \times 5 \times 10^6 = \mathbf{80 \text{ MB}}$ — a **100×** reduction.
2. At $t=0$ the bucket holds 20 tokens → **20 allowed, 10 rejected**. By $t=1.0 \text{ s}$ it has refilled by $10 \times 1.0 = 10$ tokens (capped at 20, currently $0+10 = 10$) → **10 allowed, 5 rejected**.
3. Heap merge: $N \log_2 k = 4 \times 10^6 \times \log_2 200 (\approx 7.64) = \approx \mathbf{3.06 \times 10^7}$. Full sort: $N \log_2 N = 4 \times 10^6 \times 21.9 = \approx \mathbf{8.77 \times 10^7}$ — roughly **2.9×** more, and it cannot stream.
4. Heapify: **$<10^6$ swaps** (under one per element). n pushes: up to $n \cdot \log_2 n = 10^6 \times 20 = \mathbf{2 \times 10^7}$. Top- k with $k=10$: $n \cdot \log_2 10 = 10^6 \times 3.32 = \mathbf{3.3 \times 10^6}$, in **40 bytes** of heap.
5. $Wq = \rho / (\mu(1-\rho)) = 0.85 / (2000 \times 0.15) = \mathbf{2.83 \text{ ms}}$. After +12 %: $\rho = 0.952$, $Wq = 0.952 / (2000 \times 0.048) = \mathbf{9.92 \text{ ms} - 3.5 \times}$.
6. Hot partition: $55 \% + (45 \% / 12 = 3.75 \%) = \mathbf{58.75 \%}$. Average share is 8.33 %, so the hottest consumer carries **7.05×** the average — and adding consumers cannot help it.
7. Capacity = $3000 \times 0.040 = \mathbf{120 \text{ items}}$. Sustained rejection = $(3000 - 2600) / 3000 = \mathbf{13.3 \%}$ of arrivals.

C. Code and design tasks — what to verify

1. The sliding window counter should differ from the exact log by **under ~1 %** of decisions. Token bucket will differ more on bursty traces (it permits a burst by design) — that is a feature; state it rather than treating it as error. Memory should differ by roughly the ratio in B1.
2. Fixed window must allow **200 requests in the two-second span**. The token bucket with burst 100 should allow **100 then essentially none** until refill. Record both numbers side by side.
3. At $1.5\times$ the limit for 60 s, expect depth to grow to **$0.5 \times \text{limit} \times 60$** and the oldest request's age to approach **the full 60 seconds** — by which time every client has timed out. That age, not the depth, is the number that makes the point.
4. Heap merge should use **$O(k)$ extra memory** and stream; the sort version holds all N — expect roughly **16 MB versus a few KB** of working set for the merge structure at $N = 4 \times 10^6$ ints, and a runtime ratio near **2-3 $\times$** .
5. Score yourself by **lines wrong per template**. The most common misses: the size snapshot in level order, the order of the three steps in window DP, and the comparator direction in merge-k.
6. Each demonstration must produce **a number or a failing assertion**, not an argument. For example: a starvation test showing the oldest low-priority item's age growing without bound, or a `std::greater` heap printing an ascending pop order.

HANDNOTE SUMMARY CARD

PART TWELVE - INTERVIEW AND REVIEW

topics 82-86

THE THREE QUESTIONS - ask before writing any code

1. WHICH END? one -> queue | both -> deque | the middle -> map + list / heap
2. ARRIVAL OR VALUE? arrival -> queue | value -> heap
3. SMALL INTEGER KEYS? yes -> bucket queue, the log factor disappears

PHRASE -> TECHNIQUE

"fewest steps", unweighted	BFS + queue	47,48
"level by level"	BFS + q.size() SNAPSHOT	50
"nearest of several sources"	multi-source BFS	49
"weights are 0 and 1"	deque, push_front / push_back	52
"max of every window of k"	monotonic deque (INDICES)	53,54
"best of the last k states"	monotonic deque on dp[]	55
"prerequisites / build order"	Kahn's; size != V => cycle	57
"top k" / "kth largest"	MIN-heap of size k	40
"merge k sorted"	heap of k heads, $O(N \log k)$	84
"shortest path, weighted"	heap + lazy deletion	37
"fair" / "in arrival order"	queue, NEVER a heap	2,32
"implement X using only Y"	composition; count moves/elem	41,42
"LRU / cache eviction"	hash map + linked list	45
"N per second"	token bucket	85
"producer / consumer"	BOUNDED blocking queue	59,66
"run in 30 s" / "retry later"	heap on time, or timer wheel	73
"in order per user"	partition by key	71

WORST CASE != AMORTISED (volunteer this before being asked)

growable ring (reallocation) | two-stack queue (shuttle) | min-queue
all three: amortised $O(1)$, worst case $O(n)$ on ONE operation

RATE LIMITER

sliding log exact, deque of timestamps, 8 B x limit PER KEY -> 8 GB at
200/min x 5 M users. correct and unusable.
token bucket tokens += (now - last) * rate, cap at burst; 16 B per key
-> 80 MB. stores the INTEGRAL of the rate, not the events.
window counter prev*overlap + current; ~1% error, 2 ints, 2 Redis INCRs
fixed window BUG: 100 at :59 + 100 at :00 = 200 in 2 s, both windows legal
respond 429 + Retry-After + X-RateLimit-Remaining. NEVER queue the rejects.

WRONG ANSWER -> THE CHECK

"two queues give an amortised stack"	does expensive work make the next operation cheaper? no -> not amortised
"priority queue = fair + priorities"	can an item be overtaken forever? say STARVATION first
"add a queue for the spike"	is $\lambda_{avg} < \mu$?
"make the queue bigger"	depth / μ = added p99
"85% util = 15% headroom"	$\rho/(1-\rho)$: 0.85 -> 5.7, 0.94 -> 15.7
"exactly-once"	name the moment the ack is lost
"Kafka gives ordering"	per PARTITION. what is the key?
" $O(1)$, it's a linked list"	$O(1)$ + malloc: 25-60 ns vs 2-4
"sanitisers are clean"	which bug class does each tool model?
"std::greater = max-heap"	greater -> MIN-heap
"the benchmark says list is fine"	interleaved? -02? percentiles?

FIVE SENTENCES TO HAVE READY

"Amortised $O(1)$, worst case $O(n)$ on the growth, and here is why."
"Ordering per key, not global -- global would cap throughput at one consumer."
"Bound it at $\lambda \times$ acceptable wait, shed beyond, alarm on the counter."
"At-least-once, so the handler is idempotent on the job id."
"Mark visited at ENQUEUE: each vertex pushed once, queue holds $O(V)$."

MASTER CHEATSHEET

Everything in this book, compressed to what you would want on one desk. The three cards below cover the discipline, the code, and the numbers; the tables that follow cover recognition and testing.

The core discipline

THE THREE QUESTIONS

- 1 WHICH END? one -> queue | both -> deque | the middle -> map+list or heap
- 2 ARRIVAL OR VALUE? arrival -> queue | value -> heap
- 3 SMALL INTEGER KEYS? yes -> bucket queue; the log factor vanishes

THE FIFO CONTRACT

two positions only: front (out), back (in)
NOT offered: element k, search, max, membership, removal from the middle
refusing them is what allows blocked storage, ring wrapping, and one thread per end

FIVE OPERATIONS, TWO PRECONDITIONS

enqueue(x) pre: !full dequeue() pre: !EMPTY front() pre: !EMPTY
empty() size() front()/back() references die at the NEXT MUTATION

WHEN A QUEUE IS THE WRONG ANSWER

highest priority next -> heap	element k -> vector / deque
"is x already queued?" -> queue+set	most-recently-used -> map + linked list
both ends -> deque	survives a crash -> broker / WAL
ordering across N consumers	-> one partition per key

THE TWO LAWS

- 1 every queue must have a maximum size
 - 2 consider whether that maximum should be ZERO (a direct handoff)
- and: if $\lambda_{avg} > \mu$, NO capacity helps -- the queue schedules the failure

The four templates worth memorising

1. THE RING $N = 2^k$, monotonic UNSIGNED indices, raw storage
try_enqueue(x) { if (tail_ - head_ == N) return false;
 new (slot(tail_ & (N-1))) T(x); ++tail_; return true; }
try_dequeue() { if (head_ == tail_) return nullopt;
 T* p = slot(head_ & (N-1)); opt o(std::move(*p));
 p->~T(); ++head_; return o; }
front = slot(head_ & (N-1)) back = slot((tail_-1) & (N-1))
size = tail_ - head_ full = tail_ - head_ == N
ORDER: check -> CONSTRUCT -> publish | check -> move -> DESTROY -> release
two runs: h = head_ & (N-1); first = min(size, N-h)
 (slot(h), first) + (slot(0), size-first) <- grow, writev, DMA
2. BFS
dist[src] = 0; q.push(src);
while (!q.empty()) { u = q.front(); q.pop();
 for (v : adj[u]) if (dist[v] == -1) { dist[v] = dist[u]+1; q.push(v); } }
MARK AT ENQUEUE -> each vertex pushed once, queue holds $O(V)$ not $O(E)$
levels: sz = q.size() SNAPSHOT, then loop sz times
multi-source: seed the queue with every source at distance 0
0-1 weights: deque, push_front for 0, push_back for 1
path: parent[] array + reverse. never queue>.
3. MONOTONIC DEQUE (window maximum)
if (dq.front() <= i - k) dq.pop_front(); // expire by POSITION
while (a[dq.back()] <= a[i]) dq.pop_back(); // domination is PERMANENT
dq.push_back(i); if (i >= k-1) emit a[dq.front()];
HOLD INDICES. min: flip to >=. window DP: EXPIRE -> READ -> COMPUTE -> INSERT
4. BOUNDED BLOCKING QUEUE
push: unique_lock lk(m_);
 not_full_.wait(lk, []{ return q_.size() < cap_ || closed_; });
 q_.push_back(x); lk.unlock(); not_empty_.notify_one();
pop: not_empty_.wait(lk, []{ return !q_.empty() || closed_; }); ...
TWO condition variables. closed_ in BOTH predicates. ALWAYS the predicate
form -- a cv has no state and a notify with no waiter is discarded.

The numbers

LAYOUT COST 1 M ints, x86-64 glibc, -O2
shift array 4.0 MB ~20 allocs dequeue 0(n) drain $O(n^2)$, 900 ms @100k
head index grows ~20 0(1) space tracks THROUGHPUT
ring 4.0 MB 1 0(1) 2-4 ns/op, 0 allocations
linked list 32 MB 1 000 000 0(1) 8x memory, 5-20x scan
std::deque 4.1 MB 7 813 0(1) index = 2 dependent loads

OPERATION COST

index step runtime % 12.0 ns | branch 3.1 | mask ($N=2^k$) 2.9
malloc 15-30 ns | context switch 1-3 us
cache miss ~200 cycles | mutex lock/unlock ~20-25 ns
futex sleep/wake 1-5 us | exception throw ~1-2 us
heap at $n=1e6$ push ~20 cmp, pop ~40 cmp; 4-ary 1.48x faster
heapify vs pushes 295 ms vs 1180 ms at $n=1e7$ (4.0x)
std::queue 10 M deque 86 ms | list 640 ms (7.4x) | hand ring 21 ms
SPSC ring shared line 32 M ops/s | padded 21 M | padded+cached 440 M
window max, $n=1e6$, $k=1000$ naive 1.2 s | multiset 380 ms | deque 4 ms

CAPACITY

$L = \lambda * W$ distribution-free, needs only steady state
capacity = $\lambda * W_{max}$ <- a LATENCY budget, not a memory budget
 $M/M/1 \ W_q = \rho / (\mu (1 - \rho))$
rho 0.50 0.70 0.80 0.90 0.95 0.99 ($\mu = 1000/s$)
Wq 1.0 2.3 4.0 9.0 19.0 99.0 ms
no knee -- a hyperbola. 85% util is NOT 15% headroom.
bufferbloat: added ms = bytes * 8 / link_bps. 1 MB at 10 Mbit/s = 839 ms.

SANITISERS

ASan OOB + use-after-free 2x / 3x "0 bytes to the right of a
UBSan signed overflow (indices!) 1.2x 32-byte region, READ of size 4"
_GLIBCXX_DEBUG iterator misuse up to 10x = capacity 8 ints, index 8
TSan data races 5-15x / 8x
NONE of them find: missing ~T(), reversed comparator, lost wakeup,
memcpy growth reordering, an unbounded queue.

Section map

Part	Subject	The one thing to carry away
1 (1-6)	Foundation	The contract's power is what it refuses to offer; every queue needs an overflow policy
2 (7-15)	Representation	Ring for bounded depth, blocked for stable references, never head-index in a long-running process
3 (16-24)	Core operations	Construct-then-publish, destroy-then-release, and four assertions
4 (25-31)	The deque	Same cost as a queue; end insertion keeps references and kills iterators
5 (32-40)	Heaps	Not a queue; ties are not FIFO; heapify is $O(n)$ because the loop runs upward
6 (41-46)	Composition	Two stacks make a good queue; two queues make a bad stack
7 (47-58)	Patterns	Mark visited at enqueue; the monotonic deque is $O(n)$ because domination is permanent
8 (59-63)	Concurrency	Predicate-form waits; one writer per index; kill true sharing before false sharing
9 (64-66)	Capacity	Depth is latency; bound everything and count the rejections
10 (67-74)	Real systems	Four queues before <code>accept()</code> ; delivery is at-least-once; partition key chooses your orderings
11 (75-81)	Tooling	<code>std::greater</code> gives a min-heap; sanitisers find memory bugs, not logic bugs
12 (82-86)	Review	Say what you do not guarantee, before you are asked

PATTERN RECOGNITION

The left column is what the problem, ticket, or interviewer actually says. The right column is what to reach for. Read this page before a practice session and again the night before an interview.

The problem says	Reach for	Cost	Topic
"fewest steps", "minimum moves", unweighted	BFS with a queue	$O(V+E)$	47, 48
"level by level", "each row", "k steps away"	BFS + <code>q.size()</code> snapshot	$O(n)$	50
"nearest exit", "spreads from all sources"	Multi-source BFS	$O(V+E)$	49
"free moves and costly moves", weights 0/1	Deque: <code>push_front</code> / <code>push_back</code>	$O(V+E)$	52
"the path itself, not just its length"	<code>parent[]</code> array + reverse	$O(V)$	51
"maximum/minimum of every window of size k"	Monotonic deque of indices	$O(n)$	53, 54
"best of the previous k states"	Monotonic deque over <code>dp[]</code>	$O(n)$	55
"first non-repeating in a stream"	Queue + frequency table, lazy front	$O(1)$ am.	56
"prerequisites", "build order", "is there a cycle"	Kahn's: queue of in-degree 0	$O(V+E)$	57
"kth largest", "top k", streaming input	Min-heap of size k	$O(n \log k)$	40
"merge k sorted lists or streams"	Heap holding k heads	$O(N \log k)$	84
"shortest path with arbitrary weights"	Heap + lazy deletion	$O(E \log V)$	37
"weights are integers at most C"	Dial's / bucket queue	$O(E+VC)$	39
"most urgent first"	Heap — then ask about starvation	$O(\log n)$	32, 36
"fair", "in arrival order", "first come first served"	Queue; a heap is wrong	$O(1)$	2, 32
"equal priorities must be FIFO"	Key = (priority, sequence)	+8 B	36
"implement a queue using only stacks"	Two stacks, shuttle when out is empty	$O(1)$ am.	41
" $O(1)$ minimum alongside the queue"	Two min-stacks	$O(1)$	43
"least recently used", "evict the coldest"	Hash map + <code>std::list</code> splice	$O(1)$	45
"window only ever moves right"	Two integers; no structure at all	$O(1)$ space	46
"round robin", "eliminate every kth"	Queue rotation — or a recurrence	$O(n)$	58
"producer and consumer", "worker pool"	Bounded blocking queue	~22 ns	59
"one producer, one consumer, hot path"	SPSC ring + cached cursors	~2.3 ns	61, 62
"how deep should the queue be?"	$\lambda \times$ acceptable wait	—	64
"it falls over at peak"	Bound, shed, deadline-check, count	—	66

"N requests per second"	Token bucket (16 B per key)	O(1)	85
"run this in 30 seconds", "retry with backoff"	Timer wheel, or heap on time	O(1)	73
"events must be ordered per customer"	Partition by that key	—	71
"jobs sometimes run twice"	timeout < retry_after; idempotent handler	—	70
"one bad message stalls everything"	DLQ + the ability to skip an offset	—	72, 74

The universal test matrix

Run every one of these against any queue you write, with an element type that allocates (`std::string` or `std::vector<int>`), not just `int`.

Case	What it catches
dequeue on empty	Missing precondition; the <code>T{}</code> corruption of Topic 5
front/back on empty	Reads of unconstructed storage
Single element in and out	The <code>tail_ = nullptr</code> bug (13); head/tail confusion
Fill to exact capacity	Full/empty ambiguity (10); off-by-one in <code>full()</code>
Overflow by one	Overflow policy actually applied; <code>[[nodiscard]]</code>
Wrap past the end	Masking, <code>back()</code> , and the memcpy-growth reorder (12)
Grow while wrapped	Two-run move; the single most-missed test in the book
Drain to empty, then refill	Index renormalisation; the two-stack shuttle condition (41)
Alternating push/pop, 10^6 times	Slow leaks; monotonic index arithmetic
Destruct with elements still queued	Destructor destroys size, not N (22)
Copy and move the queue	Rule of five; double free
Element type that throws on construction	Construct-then-publish ordering (16, 26)
Differential test against <code>std::queue</code>	Everything else, 10^6 random operations

REVISION TRACKER

Fill this in by hand as you go. The columns match a Day 0 / +1 / +7 / +30 / +90 schedule: first pass, a next-day review of the summary card only, the practice set, then three spaced reviews. A part is "done" when you can reproduce its summary card from memory and your answers to Section B match the key.

Topic group	First	Rev 1 +1d	Practice	Rev 2 +7d	Rev 3 +30d	Rev 4 +90d
1. Foundation (1-6)						
2. Representation (7-15)						
3. Core operations (16-24)						
4. The deque (25-31)						
5. Heaps (32-40)						
6. Composition (41-46)						
7. Patterns (47-58)						
8. Concurrency (59-63)						
9. Capacity (64-66)						
10. Real systems (67-74)						
11. Tooling (75-81)						
12. Review (82-86)						
Master cheatsheet						
Four templates, from memory						
Ten canonical problems (84)						

What each column means

Column	The activity
First	Read the part end to end. Type out every code artifact rather than reading it. Expect 60–120 minutes for a large part.
Rev 1 (+1d)	Summary card only, 10 minutes. Cover the right-hand side and recall the numbers.
Practice	The full practice set, timed, then the answer key. Sections B and C are where the learning is; A is a warm-up.
Rev 2 (+7d)	Reproduce the summary card on blank paper. Diff against the book and note only what you missed.
Rev 3 (+30d)	Re-do Section B of the practice set cold. Re-read only the topics whose questions you got wrong.
Rev 4 (+90d)	Master cheatsheet plus the pattern table. Then write the four templates from memory, compiling each.

HOW TO USE THIS BOOK

This is Volume 2 of a core computer science series, and it is built to be *worked* rather than read. Reading it front to back in an afternoon will produce a pleasant sense of understanding that evaporates within a week. The structure below is what actually transfers.

The reading order

If you are learning queues for the first time, go in order. Parts One through Four are cumulative: the ring built in Part Three is the object that Part Four extends, Part Eight makes concurrent, and Part Eleven benchmarks. Do not skip Part Two — every number in the rest of the book traces back to a layout decision made there.

If you are preparing for interviews, read Parts One, Five, Six and Seven, then Part Twelve. Write out the four templates in the master cheatsheet until you can produce each from memory in under five minutes. Then come back for Parts Eight through Ten — those are what separate a candidate who can code a BFS from one who can be trusted with a system.

If you are debugging something in production right now, start at Part Nine, then Part Ten, then Topic 66. If it is a C++ correctness problem, start at Part Eleven and run the sanitisers before reading anything.

The practice sets are the book

Section A checks that you can state a mechanism. Section B checks that you can compute with it — those are the questions that reveal whether "amortised $O(1)$ " means anything to you yet. Section C is where the real work is: every set contains at least two tasks that ask you to break working code deliberately, observe the failure, and only then fix it.

Those tasks matter more than they look. The lesson of Topic 17 is not "call the destructor" — it is that omitting it produces *no symptom at all* until 400 MB have leaked. You cannot learn that from a warning box. You learn it by deleting one line, running valgrind, and reading the number. The same is true of the ASan report in Topic 79, the silent element loss with two producers in Topic 61, and the livelock in Topic 23. Do them.

The numbers

Every measurement in this book was chosen because it changes a decision. They come from a mix of published figures, standard-library source, and the kind of benchmark described in Topic 81 — and they are all from someone else's machine. **Re-measure the ones you intend to rely on.** The ratios (list is $7\times$ a deque; padding without cursor caching is slower than no padding; heapify is $4\times$ repeated pushes) are stable across hardware. The absolute nanoseconds are not.

What to do when you finish

Fill in the tracker honestly. Then pick one thing from your own codebase — a job queue, a worker pool, a cache, a retry loop — and answer the three questions of Topic 82 about it out loud: which end do you touch, is the order arrival or value, and what happens when it fills. If the third answer is "it grows", you have found your first thing to fix, and you now know exactly what to replace it with.

Everything About Queues · Core Computer Science · Volume 2 · 86 topics, twelve parts, C++17 throughout. Written to be broken, measured, and argued with.

